

IB Math Analysis and Approaches HL Exploration

A Comparative Study of First-order and Second-order Optimizers for
Training Large-Scale AI Models

Personal Code: lgv218

Number of Pages: 30

Table of Contents

1 – Introduction	3
2 – Background on Optimization	3
2.1 Why Iterative Optimizers?	3
2.2 Complexity via Big O Notation	4
2.3 Local vs. Global Minima	4
3 – Optimization Method	5
3.1 First-Order (Gradient-Based) Method: Gradient Descent	5
3.2 Second-Order (Hessian-Based) Method: Newton–Raphson	6
3.3 Worked Example: Gradient Descent vs. Newton–Raphson	7
4 – Empirical Evaluation	9
4.1 Methodology Overview	9
4.1.1 Program Setup	9
4.1.2 Benchmark Functions	10
4.1.3 Random Initialization Strategy	12
4.1.4 Preliminary Trials & Dimension Limits	12
4.2 Efficiency of Optimizers (Runtime Analysis)	13
4.2.1 Preliminary Observations	13
4.2.2 Detailed Runtime Graphs	15
4.2.3 Statistical Validation (Mann–Whitney Test)	18
4.3 Effectiveness of Optimizers (Success Analysis)	19
4.3.1 Failure Cases (NaN)	19
4.3.2 Relative Improvement of Final Function Value	21
4.4 Per-Iteration Analysis of Optimizers	23
4.4.1 Gradient Cosine Similarity	23
4.4.2 Step-Size Variation	24
4.4.3 Statistical Testing of Iteration-Level Metrics	26
4.5 Real-World AI Training: Gradient Descent vs. Newton–Raphson	27
5 – Conclusion	29
6 – Evaluation	30
References	33
Appendix A	38
Appendix B	54
Appendix C	65

1 – Introduction

I am passionate about training **AI** (*artificial intelligence*) models, which rely on **optimization techniques** to learn effective parameters (Sievers & Thornton, 2024). Although AI models typically perform well by reaching sufficiently low **local minima** in **high-dimensional** spaces (Kiem, 2020), I have often wondered why certain optimizers are chosen over others. In my projects, I frequently faced the challenge of selecting the “*right*” *optimizer* without a deep understanding of the underlying mathematics. Through this **exploration**, I intend to bridge that gap by studying these algorithms in depth—investigating *which approach works better under specific conditions and why*.

While **first-order methods** (e.g., *Gradient Descent*) dominate most recent AI applications due to their simplicity and efficiency, I have found that **second-order techniques** (e.g., *Newton Raphson*) boast a long history in optimization theory (Mehouachi & Kasmi, 2021). Thus, in this exploration, I aim to compare these **classic second-order methods** with **gradient-based approaches**, assessing their performance—especially in the **high-dimensional** settings typical of neural network (AI) training. My investigation explores the IB AA HL curriculum topics like vectors, calculus, numerical methods and statistics.

2 – Background on Optimization

2.1 Why Iterative Optimizers?

In many domains—whether **physics simulations**, *finance*, or **artificial intelligence**—we need to **find minima** of some function, aiming for the best (*global*) or at least a *good-enough* (*local*) solution. At first glance, one might think, “*Just set $\frac{dy}{dx} = 0$ or $\nabla f(\mathbf{x}) = \mathbf{0}$ to find the global minima!*” However, the **Abel–Ruffini theorem** (Weisstein, n.d.) shows that even for polynomial equations of degree five (which can appear in as few as two dimensions), no general closed-form solution exists—let alone for more complex higher-dimensional functions. As we scale up to **massive** real-world problems—such as training *ChatGPT-4* with **1.8 trillion parameters** (Schreiner, 2023) (where the optimizer must navigate a *trillion-dimensional* space!)—this naive approach becomes utterly **infeasible**, forcing us to rely on **iterative optimizers** instead.

Therefore, *iterative optimizers* are deployed to handle this complexity. They systematically traverse a high-dimensional parameter space, adjusting variables bit by bit until they (hopefully) converge to a suitable local minimum (or approach the global minimum) within a manageable timeframe (Erway, 2006). Although

this paper focuses on the **AI perspective**, the methods and insights presented are equally applicable to other fields where large-scale optimization is critical (Kochenderfer & Wheeler, 2019, 2).

2.2 Complexity via Big O Notation

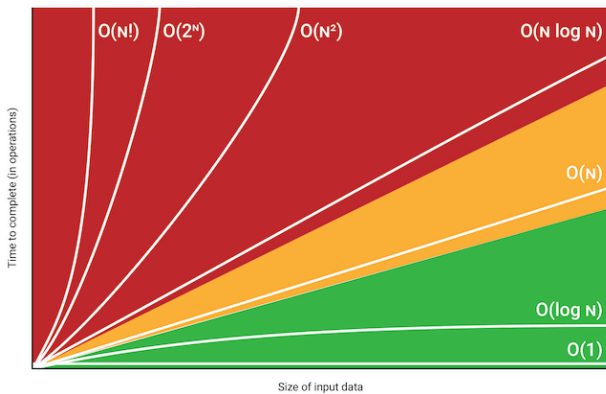


Figure 1: Comparison of Common Big O Complexity Classes (Gupta, 2021)

As problems become larger, not all optimization methods remain feasible. To compare how different algorithms scale as the number of parameters n increases, we use **Big O notation** (Programiz, 2024)—a standard tool in both mathematics and computer science for describing asymptotic computational costs. For instance, evaluating a gradient might require $\mathcal{O}(n)$ operations, whereas computing and solving systems involving a Hessian can escalate to $\mathcal{O}(n^2)$ or $\mathcal{O}(n^3)$ (Tibshirani, 2015). Even if a method is theoretically appealing, it may be prohibitively expensive in

very high-dimensional problems.

To illustrate this growth, **Figure 1** compares various Big O complexities. Notice how functions like $\mathcal{O}(n)$ or $\mathcal{O}(n \log n)$ grow more slowly than $\mathcal{O}(n^2)$, $\mathcal{O}(n^3)$, or worse. This matters because neural networks (especially at large scales) can have millions or billions of parameters, making frequent Hessian computations impractical. Thus, Big O notation is a key lens through which we'll judge the feasibility of the optimization methods examined later.

2.3 Local vs. Global Minima

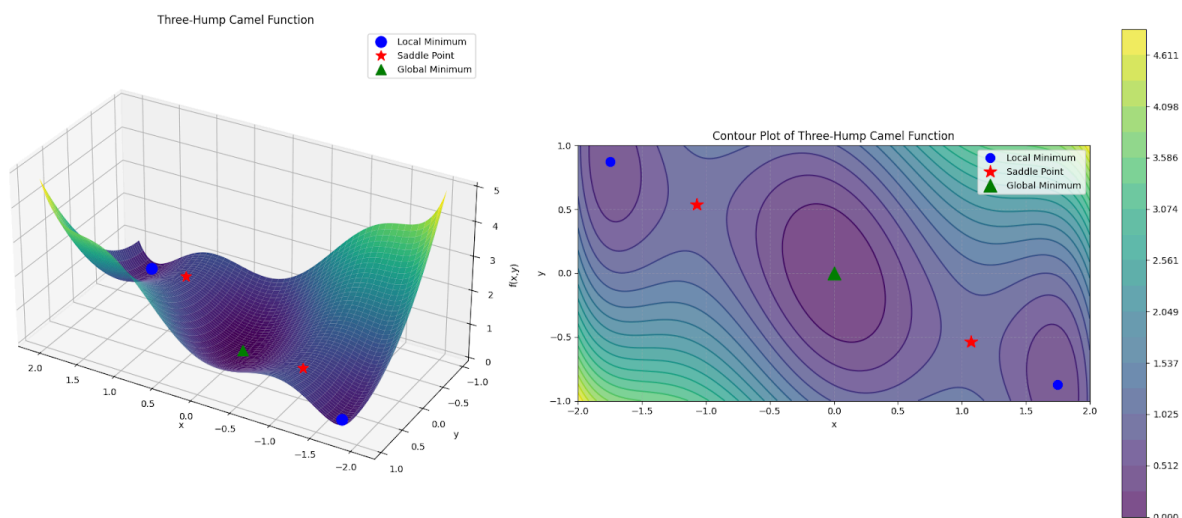


Figure 2: 3D Plot with Contour Plot Demonstrating Function's Landscape Features (Matplotlib)¹

¹ Note: All subsequent custom graphs have been generated by me programming inside the Matplotlib Python library (Hunter, 2007), which offers extensive capabilities for creating custom plots. The source code for these plots is available in Appendix C.

When optimizing a function, the “best” outcome is often the **global minimum**—the lowest point in the entire landscape. However, finding a point that appears low does not guarantee it is the absolute best; it could be a **local minimum**, a dip lower than its immediate surroundings but not necessarily the global optimum (MathWorks, n.d.).

Complicating matters are **saddle points** (see Figure 2), where the gradient vanishes yet the function curves upward in some directions and downward in others. Since these points are neither maxima nor minima, they can stall or mislead certain algorithms (Kamburova et al., 2023).

In principle, we want the **global minimum**, but in practice—especially for **large-scale neural networks**—reaching a suitable local minimum often suffices, given the complexity of these high-dimensional landscapes (Kiem, 2020).

3 – Optimization Method

3.1 First-Order (Gradient-Based) Method: Gradient Descent

When minimizing a function, one of the simplest strategies is to use its **gradient**—a vector pointing in the direction of steepest ascent. By moving in the opposite direction of the gradient, we “descend” toward lower values of the function, aiming to reach a minimum (Weisstein, n.d.).

From Partial Derivatives to Gradients

Consider a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ that depends on variables $(x_1, x_2, \dots, x_n)$. To understand how f changes, we look at **partial derivatives**, which measure the rate of change of f with respect to a single variable while holding the others fixed. Formally, for the i^{th} variable:

$$\frac{\partial f}{\partial x_i}(x_1, \dots, x_n) = \lim_{h \rightarrow 0} \frac{f(x_1, \dots, x_i + h, \dots, x_n) - f(x_1, \dots, x_i, \dots, x_n)}{h}$$

By calculating the partial derivatives for each variable, we can construct the gradient vector:

$$\nabla f(x) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(x) \\ \frac{\partial f}{\partial x_2}(x) \\ \vdots \\ \frac{\partial f}{\partial x_n}(x) \end{bmatrix}$$

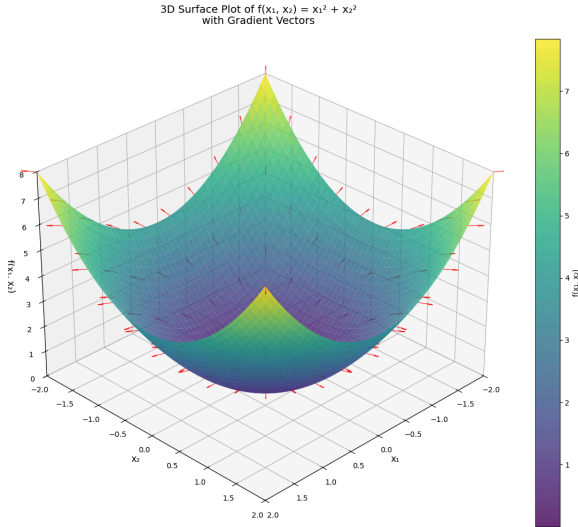
Mathematical Formulation

Given a current parameter vector $\mathbf{x}_t$, Gradient Descent updates it by moving in the opposite direction of the gradient, by following $-\nabla f(\mathbf{x})$ (Kochenderfer & Wheeler, 2019, 69):

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \alpha \nabla f(\mathbf{x}_t),$$

where $\alpha > 0$ is the **learning rate**, which determines how large a step we take each iteration. By continuously stepping opposite the gradient, we aim to reach a point where $\nabla f(\mathbf{x}) = 0$, ideally corresponding to a minimum of the function f (Kochenderfer & Wheeler, 2019).

Visualization



For visualization, **Figure 3** provides a 3D plot of the sphere function (see Table 1 below for definition), with red gradient arrows plotted at various points. Each arrow points toward the direction of steepest ascent (the direction of $\nabla f(\mathbf{x})$). Moving in the direction of $\nabla f(\mathbf{x})$ increases the function most rapidly, while moving in the direction of $-\nabla f(\mathbf{x})$ leads to the steepest descent (MathWorks, n.d.). Hence, following $-\nabla f(\mathbf{x})$ repeatedly is the core idea behind **Gradient Descent**.

Figure 3 - 3D Surface Plot of Sphere Function with Red Gradient Arrows (Matplotlib)

3.2 Second-Order (Hessian-Based) Method: Newton–Raphson

Having explored Gradient Descent, I will now introduce second-order methods, specifically **Newton Raphson**. This method differs from the first-order method by incorporating **second-order curvature information** through the **Hessian matrix**, which can potentially lead to faster convergence near a minimum (Tibshirani, 2015).

Introducing the Hessian

For a twice-differentiable function $\mathbb{R}^n \rightarrow \mathbb{R}$, the Hessian $H(\mathbf{x})$ at a point $\mathbf{x} \in \mathbb{R}^n$ is defined as the $n \times n$ matrix of second-order partial derivatives (Kochenderfer & Wheeler, 2019, 87):

$$H(\mathbf{x}) = \nabla^2 f(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

The Hessian provides a local quadratic approximation to f around $\mathbf{x}$. Near a local minimum $\mathbf{x}^*$, if $H(\mathbf{x}^*)$ is positive definite, methods using the Hessian (like Newton Raphson) can converge **quadratically** (Tibshirani, 2015), meaning they reduce the error at an accelerating rate.

Computational Complexity and Practical Limitations

While the Hessian provides valuable curvature information, its computation and use come with high **computational** and **memory** costs.

- **Computing Hessian Entries:** Calculating each second derivative is expensive. For an n -dimensional problem, the Hessian matrix has n^2 entries. Even if we exploit symmetry, the cost of forming the Hessian is still $\mathcal{O}(n^2)$ matrix (Tibshirani, 2015).
- **Matrix Inversion or Factorization:** Many second-order methods, like the Newton step, require solving a linear system that involves inverting or factorizing the Hessian, which has a computational complexity of $\mathcal{O}(n^3)$. This cubic complexity becomes impractical for high-dimensional problems, such as those encountered in modern neural networks with billions of parameters (Haramoto & Matsumoto, 2008).

Thus, while Newton's method can achieve **quadratic convergence** near the solution, the **high computational** and **memory requirements** make it impractical for large-scale problems.

Mathematical Definition:

The Newton Raphson method uses the **Hessian matrix** $H_k = \nabla^2 f(\mathbf{x}_k)$ and the **gradient** $\mathbf{g}_k = \nabla f(\mathbf{x}_k)$ at iteration k to compute the parameter update (Kochenderfer & Wheeler, 2019, 89):

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha H_k^{-1} \mathbf{g}_k$$

where α is a user-defined step size (learning rate). Similar to **Gradient Descent**, it moves in the opposite direction of the gradient to approach the minimum. However, if f is twice continuously differentiable and the **Hessian** H_k is **positive definite** near the solution, **Newton's method** can achieve **quadratic convergence** near a local minimum. This means that as the method approaches the minimum, the error decreases at an accelerating pace, leading to faster convergence compared to first-order methods (Tibshirani, 2015).

3.3 Worked Example: Gradient Descent vs. Newton–Raphson

In this subsection, to demonstrate how both **Gradient Descent** and **Newton Raphson** work, I will use the objective test function and demonstrate the calculation required of 1 single iteration:

$$f(x, y) = (x - 1)^2 + (y + 2)^2$$

We start with the initial point $(x_0, y_0) = (2, 2)$ and a learning rate $\alpha = 0.1$.

Gradient Descent - Worked Example

1. **Compute Gradient** at $(x_0, y_0) = (2, 2)$:

$$\nabla f(2, 2) = (2(x - 1), 2(y + 2)) = (2, 8)$$

2. **Update Parameters:**

$$x_1 = 2 - 0.1 \times 2 = 1.8, \quad y_1 = 2 - 0.1 \times 8 = 1.2$$

3. **Function Value** at $(1.8, 1.2)$:

$$f(1.8, 1.2) = (1.8 - 1)^2 + (1.2 + 2)^2 = 0.64 + 10.24 = 10.88$$

Brief Comment: We can see that Gradient Descent moves closer to the minimum, though at a relatively slow pace due to smaller, incremental steps.

Newton Raphson - Worked Example

1. **Compute Gradient and Hessian** at $(x_0, y_0) = (2, 2)$:

$$\nabla f(2, 2) = (2, 8), \quad H(2, 2) = \nabla^2 f(2, 2) = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$

2. **Update Parameters:**

$$\mathbf{x}_1 = \mathbf{x}_0 - H^{-1} \nabla f = \begin{bmatrix} 2 \\ 2 \end{bmatrix} - \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 2 \\ 8 \end{bmatrix}$$

$$x_1 = 2 - 0.5 \times 2 = 1, \quad y_1 = 2 - 0.5 \times 8 = -2$$

3. **Function Value** at $(1, -2)$:

$$f(1, -2) = (1 - 1)^2 + (-2 + 2)^2 = 0$$

Brief Comment: Newton Raphson takes a larger, more informed step using both the gradient and Hessian, enabling faster convergence. It reaches the global minimum in one iteration, demonstrating quadratic convergence. However, as shown in **Figure 5**, this may not be a fair test for Gradient Descent, since the objective test function is quadratic by nature, where Hessians naturally perform better (Tibshirani, 2015).

Plotting our Worked Example

Due to space constraints, I have only shown the example working for **one iteration**. However, in real scenarios, both methods would generally require more than one step to reach convergence. To clarify this, I decided to use **Matplotlib** (Hunter, 2007) to plot the progress of both **Gradient Descent** and **Newton Raphson**, showing the iterations taken until convergence, as seen in **Figure 5**:

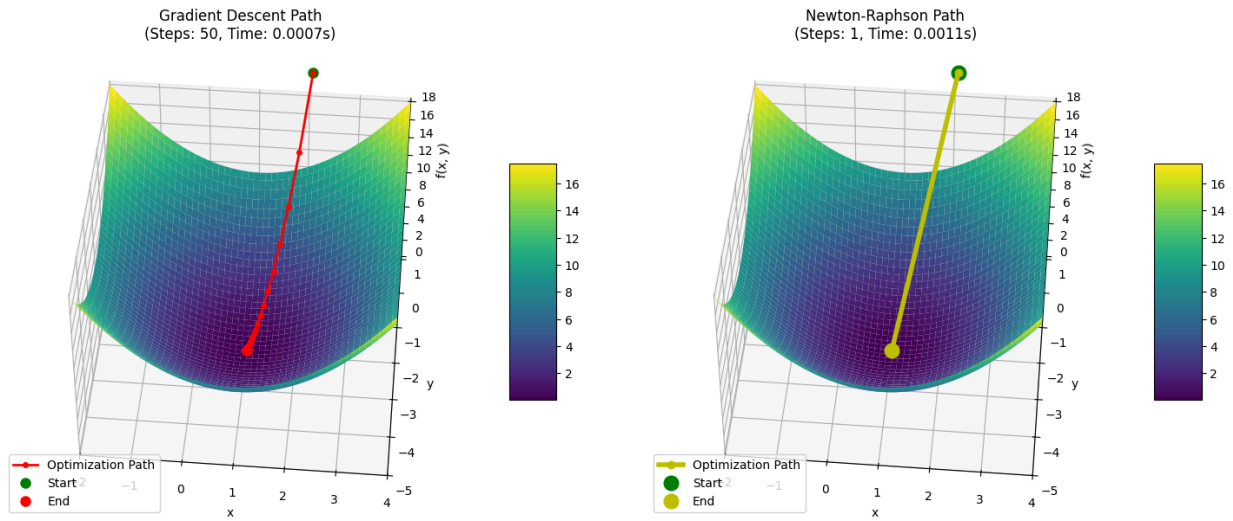


Figure 4: Comparison of Optimizers' Trajectories on Test Function Beyond 1st Iteration (Matplotlib)

As seen in **Figure 4**, Newton Raphson converges after just **one iteration**, while Gradient Descent requires about **50 iterations** to converge. Newton Raphson's rapid convergence is likely due to its quadratic convergence property, which is particularly effective on Hessian-friendly functions (like the paraboloid in Figure 5). Although Gradient Descent required more iterations, it surprisingly requires **less runtime overall**. I believe this is likely due to the **higher computational cost** associated with Newton Raphson's matrix inversion (Haramoto & Matsumoto, 2008) and Hessian calculations (Tibshirani, 2015), which have complexities of $\mathcal{O}(n^2)$ and $\mathcal{O}(n^3)$ respectively. This runtime difference is a common trend that I observed in **Section 4**, where Newton Raphson's computational demands make it less efficient than Gradient Descent, especially for larger problems.

4 – Empirical Evaluation

4.1 Methodology Overview

This section details how I gathered empirical data to compare the performance of **Gradient Descent** and **Newton Raphson** on a diverse set of benchmark functions. My goal is to simulate real-world, **high-dimensional** scenarios—like those found in **neural networks**—and to evaluate each optimizer's **convergence speed**, **reliability**, and **scalability**.

4.1.1 Program Setup

All empirical results in Sections 4.2 to 4.4 come from a Python (Python, n.d.) program I developed and ran on **Google Colab's high-RAM CPU instance** (Google, n.d.), leveraging:

- **NumPy** (Harris et al., 2020) for vector arithmetic.
- **SciPy** (Virtanen et al., 2020) for the built-in optimization routines.
- **Matplotlib** (Hunter, 2007) for plotting and visualizations.

The full code can be found in **Appendix A**. My workflow is as follows:

1. **Benchmark Functions and Optimizers**

- I defined five benchmark functions (detailed in Section 4.1.2) inside NumPy.
- For the optimizers, I relied on SciPy’s implementations of Gradient Descent and Newton Raphson to ensure **algorithmic reliability** and reduce any bias from custom coding.

2. **Iteration Cap**

- Initially, I allowed runs to continue indefinitely, but upon discovering that some tests lasted for hours, I imposed a **2,000-iteration limit**. This mirrors real-world constraints, where computational resources are finite.

3. **Dimensional Range**

- I tested parameter dimensions {2, 8, 32, 128, 512, 2048, 8192, 32768, 131,072}. While **131,072** might seem extreme, modern AI models can involve billions of parameters. Hence, high-dimensional environments are highly relevant to evaluate optimizer performance.
- Section 4.1.4 explains why certain optimizer (Newton Raphson) were ultimately capped at lower dimensions (e.g., 8192D) to remain tractable.

4. **Multiple Trials and Data Logging**

- For each dimension–function pair, I conducted **15 independent trials** to account for random initial conditions (detailed in Section 4.1.3), ensuring a fair comparison and allowing me to calculate reliable averages for each scenario.
- After every run, I programmed my program such that metrics such as runtime, iteration counts, and final function values were logged to CSV files. I then used **Matplotlib** to generate custom plots and performed **statistical tests** to validate our findings (detailed in Sections 4.2–4.4).

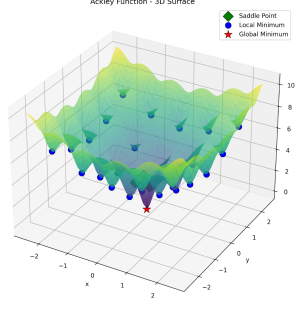
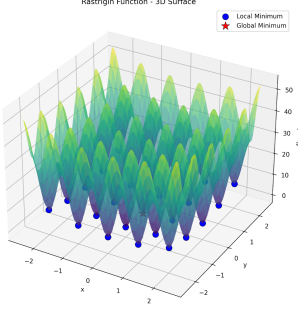
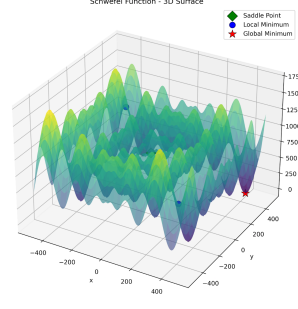
5. **Control of External Variables**

- All tests ran on the **same Google Colab CPU instance** (hence same compute power, fair test), and I minimized coding overhead by using **trusted libraries** like SciPy and NumPy for both the functions and optimizers when running the empirical experiments. Hence, this ensures a fair test, as any observed performance differences reflect the **optimizers’ true behavior** rather than hardware or my program’s inconsistencies.

Overall, this setup allowed me to **systematically** and **fairly** compare how each optimizer scales with dimension and to evaluate their performance under realistic conditions.

4.1.2 **Benchmark Functions**

To approximate the complexity of neural network loss surfaces, I selected **five benchmark functions**—Ackley, Rastrigin, Schwefel, Rosenbrock, and Sum of Squares—that all scale naturally to n -dimensional inputs. **Table 1** shows each function’s equation and a representative 3D surface plot, while **Table 2** summarizes their key properties (e.g., multimodality, saddle points) and clarifies **why** they are included.

Function Name	Ackley	Rastrigin	Schwefel
Equation	$f(x) = -20 \exp \left(-0.2 \sqrt{\frac{1}{n} \sum_{i=1}^n x_i^2} \right) - \exp \left(\frac{1}{n} \sum_{i=1}^n \cos(2\pi x_i) \right) + 20 + e$	$f(x) = 10n + \sum_{i=1}^n [x_i^2 - 10 \cos(2\pi x_i)]$	$f(\mathbf{x}) = 418.9829n - \sum_{i=1}^n x_i \sin(\sqrt{ x_i })$
3D Surface Plot			

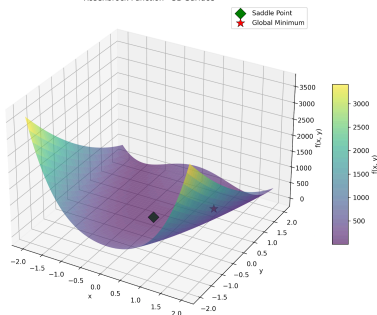
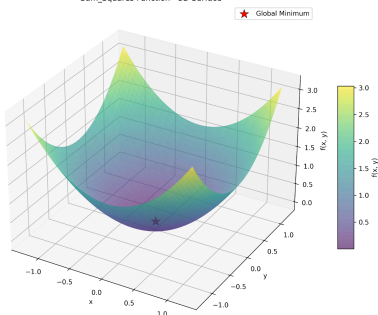
Function Name	Rosenbrock	Sum of Squares
Equation	$f(x) = \sum_{i=1}^{n-1} [100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2]$	$f(x) = \sum_{i=1}^n ix_i^2$
3D Surface Plot		

Table 1: Benchmark Functions: Equations and 3D Visualisations (Matplotlib) (Surjanovic et al., 2013)

Function Name	Characteristics	Purpose
Ackley	Highly multimodal, numerous local minima, presence of saddle points.	Tests whether an optimizer can escape local minima and navigate a highly “rugged” surface.
Rastrigin	Large number of local minima are arranged periodically.	Challenges optimizers to avoid getting trapped in periodic local dips.
Schwefel	Many local minima, global minimums are deceptively placed far from the origin.	Evaluates an optimizer’s ability to locate the true global minimum despite misleading local minima.
Rosenbrock	Narrow, curved valley leading to a global minimum; often ill-conditioned.	Examines steady convergence in regions where the gradient may mislead naive algorithms.
Sum of Squares	Simple convex function (similar to a Sphere function), easily scaled to large n .	Provides a controlled environment to measure raw convergence efficiency on a purely convex surface.

Table 2: Benchmark Functions: Key Characteristics and Purposes (Surjanovic et al., 2013)

Because **real neural network landscapes** are often a complex mix of steep valleys, flat plateaus, and scattered local minima (Geiger et al., 2021), testing on this **range of function types** (Table 1-2) provides a well-rounded indication of how each optimizer might fare in practice.

4.1.3 Random Initialization Strategy

My empirical testing aims to assess how each optimizer performs on the benchmark functions (see **Table 1**), so I must carefully define **initial conditions**. In real-world AI training, weights are typically initialized randomly—often uniformly or with small Gaussian noise—to avoid starting too close to any global minimum (Scabini et al., 2024). I mirror this practice by **programming my program** such that it **randomizes** each run’s starting point, rather than using a single **fixed coordinate** for **all runs**, which prevents any one optimizer from disproportionately benefiting if the initial point favors its approach.

Concretely, for each dimension, I draw each initial coordinate $x_i^{(0)}$ from a **uniform distribution**:

$$x_i^{(0)} \sim U(-r, r) \quad \text{for } r \in \{2, 5\}.$$

where r is chosen based on preliminary trials. Larger intervals (e.g., ≥ 100) frequently caused **divergence**, while very small ones (e.g., ≤ 1) trivialized convergence by starting too close to local minima. Selecting $\{2, 5\}^2$ thus provided a **middle ground**—challenging yet feasible.

To ensure **reliable** results, I ran each experiment **15 times** for every function–dimension pair and averaged the outcomes. This approach captures the statistical variation caused by random initialization, reflecting how **robust** an optimizer is when faced with varying starting conditions.

4.1.4 Preliminary Trials & Dimension Limits

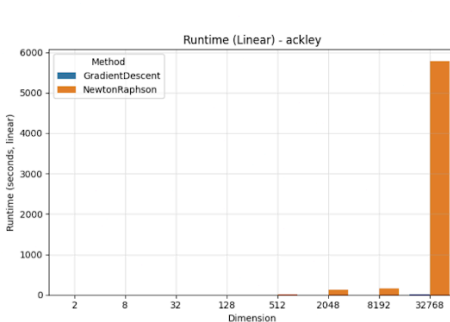


Figure 5: Ackley Runtime on Linear Scale (Matplotlib)

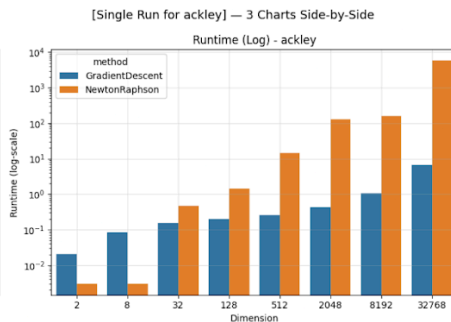


Figure 6: Ackley Runtime on Log Scale (Matplotlib)

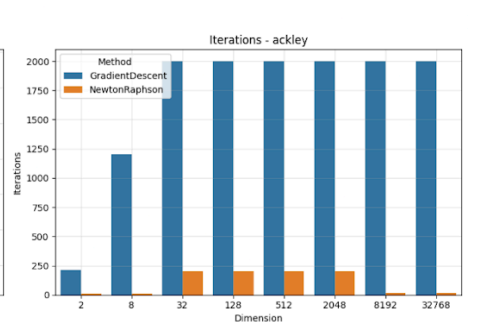


Figure 7: Ackley Iterations vs. Dimension (Matplotlib)

² We didn’t choose initialization values below 2 because, for functions like Sum of Squares, doing so would place the optimizer too close to the global minimum initially, making the test uninformative. However, this domain choice still caused **problems** for Sum of Squares in high dimensions (**Section 4.3.1**). Hence, we subsequently proposed **potential improvements** in **Section 6**.

Once I coded my initial program, I quickly discovered how **time-consuming** and **inefficient** Newton Raphson can be at higher dimensions. As seen in **Figure 5**, a single run at 32,768 dimensions lasted around **5,600 seconds** (over an hour), whereas Gradient Descent stayed below **10 seconds** for the same dimension. To illustrate this gap, I produced **Figure 6** with a logarithmic y-axis, revealing this dramatic divergence.

Initially, I assumed that **fewer iterations** would always mean **faster convergence**. However, when Newton Raphson exceeded **one hour** at 32,768 dimensions, I realized **iteration counts** can be **misleading**. Its $\mathcal{O}(n^3)$ matrix inversion cost (Haramoto & Matsumoto, 2008) often **negates** the benefit of fewer steps—for example, **Figure 7** shows it needs fewer than 50 iterations on Ackley yet still consumes **over 5,600 seconds** (**Figure 5**). This realization prompted me to **log actual runtime** for each method going forward—an essential metric in **real-world AI training** (Cueto-Mendoza & Kelleher, 2024)—as computational time is a more reliable performance indicator than iteration count.

Because of these results, I **capped** Newton Raphson runs at 8,192 dimensions, where a single run averaged about **300 seconds**—still feasible on my Google Colab CPU instance (Google, n.d.). Ironically, although Newton Raphson required **fewer iterations** overall (see **Figure 7**), its total runtime still far exceeded Gradient Descent. Hence, when **comparing efficiency** in Section 4.2, I only focus on comparing the **elapsed time** rather than iteration counts, given that “fewer steps” may still incur a **substantial** computational cost (as seen with Newton Raphson).

Curious about Gradient Descent’s behavior **beyond** 8,192 dimensions, I pushed it to 32,768 and 131,072. The findings of these extended runs—and how performance scales to even larger dimensionality for Gradient Descent—are presented in **Section 4.2** to **Section 4.4**.

4.2 Efficiency of Optimizers (Runtime Analysis)

This section (4.2 to 4.4) presents and analyzes the **empirical results** generated by the program I discussed in **Section 4.1**.

4.2.1 Preliminary Observations

One of my core aims is to evaluate each optimizer’s **efficiency**—especially in **high-dimensional** contexts, where AI training costs can **skyrocket**. As seen in Section 4.1.4, **iteration counts** can be misleading; Newton Raphson may converge in fewer steps yet incur a far higher **total runtime**. Therefore, I have primarily logged and analyzed **seconds to completion** to assess the efficiency and practical viability of the optimizers.

Averaged Across All 5 Functions

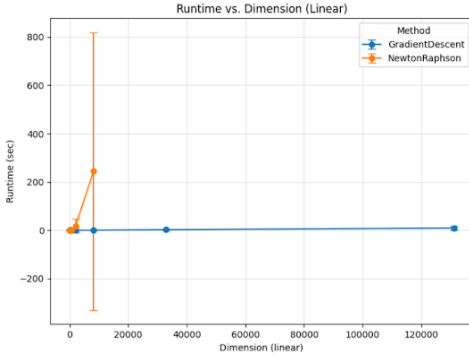


Figure 8: Mean Runtime vs. Dimension (Linear Scale) (Matplotlib)

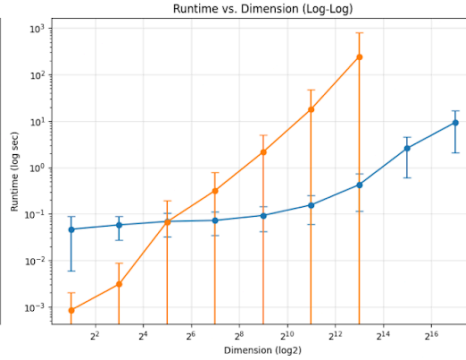


Figure 9: Mean Runtime vs. Dimension (Log-Log Scale) (Matplotlib)

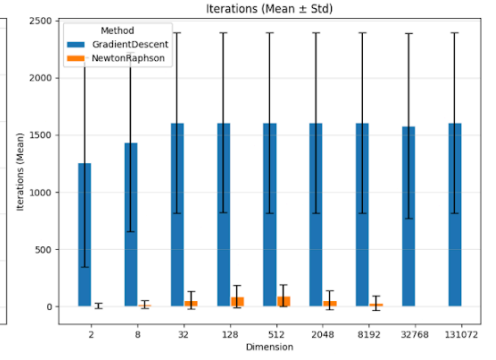


Figure 10: Mean Iteration Count vs. Dimension (with Standard Deviations) (Matplotlib)

To illustrate these comparisons, **Figure 8** plots runtime versus dimension on a linear scale, while **Figure 9** reuses the same data but applies a **logarithmic** axis to both dimensions and runtime, highlighting trends at large scales. **Figure 10** shows iterations versus dimension, again with σ (standard deviation) error bars.

From **Figure 9**, Newton Raphson is slightly faster than Gradient Descent at very small dimensions (e.g., 2^2 or 2^3), but its runtime **rises sharply** thereafter (though the increase appears less dramatic on a logarithmic scale). By contrast, Gradient Descent's **mean runtime** remained under **10 seconds** even at 131,072 dimensions, demonstrating that it **scales more gracefully**—a highly desirable trait for **AI training**, which may extend into the **billions** of parameters.

Though, for certain **low-dimensional** tasks (like physics simulations), Newton Raphson's fewer steps may still be advantageous, but in **large-scale** AI scenarios involving millions of parameters, its $\mathcal{O}(n^3)$ Hessian computations can become prohibitively expensive.

To capture **consistency**, I have programmed my code to **log** both the **mean** and **standard deviation** (σ) (National Library of Medicine, n.d.) of runtime across 15 trials:

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2},$$

reflecting the spread of performance under different initial conditions. Further, I initially considered reporting only **average runtimes**, but realized that also analyzing σ is useful because a **large** σ for an optimizer's runtime, for example, can indicate an optimizer having **inconsistent** efficiency, which is a significant problem in **AI training**. Because **stable** compute costs are crucial for AI training—especially given that large-scale training can incur costs in the **millions of USD** (Buchholz, 2024).

From these plots, **Gradient Descent** exhibits notably **lower** σ than Newton Raphson across most dimensions, implying it adapts more consistently to **random** initial conditions. This characteristic is

especially **valuable** in large-scale AI tasks, where initial conditions faced by optimizers frequently vary significantly.

However, I realised that **Figure 8** and **9** are plagued by the issue of aggregating all the data from **all five** benchmark functions at each dimension, which is **not ideal** for pinpointing function-specific nuances. Hence, in the **next subsection**, I employ **confusion matrices** (ScienceDirect, n.d.) to delve further into analysing how each optimizer’s efficiency performs on each function and dimension **individually**.

4.2.2 Detailed Runtime Graphs

Relative Standard Deviation

Moreover, a raw standard deviation σ can be **misleading** if one optimizer’s **mean runtime** $\bar{t}$ is **much larger or smaller** than another’s. For instance, $\sigma = 10$ seconds is **just 10%** of the mean if $\bar{t} = 100$ seconds, but it’s a **1,000%** deviation if $\bar{t} = 1$ second. To resolve this, I introduced **RelativeStd** (ScienceDirect, n.d.), calculated by:

$$\text{RelativeStd} = \frac{\sigma}{\bar{x}} \times 100\%,$$

This normalizes σ against the mean, ensuring **fairer** comparisons even when average runtimes differ by an order of magnitude or more. Throughout Sections 4.2–4.4, most figures therefore include both **mean** runtimes and **RelativeStd** metrics, giving a **clearer** picture of each optimizer’s performance at scale.

Initial Experiments with Graphs

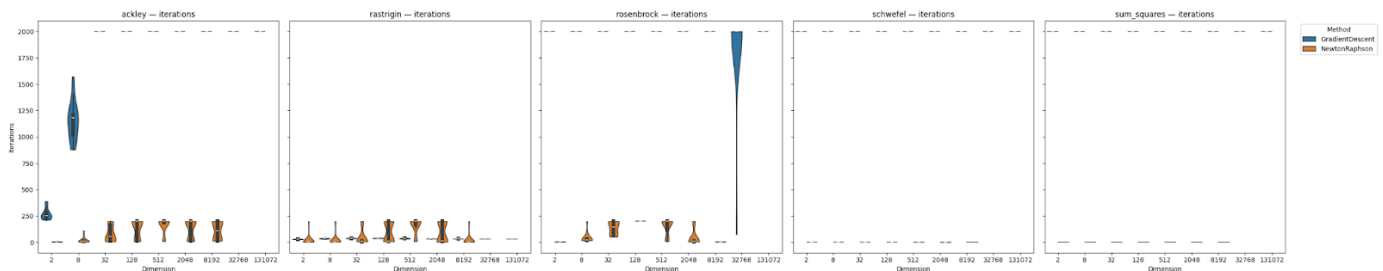


Figure 11: Violin Plots of Iterations per Dimension (All Functions) (Matplotlib)

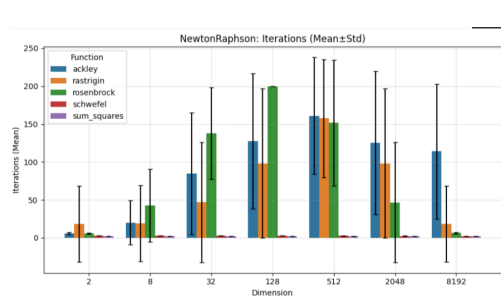


Figure 12: Newton-Raphson Iterations (Mean ± Std) Across Dimensions (Matplotlib)

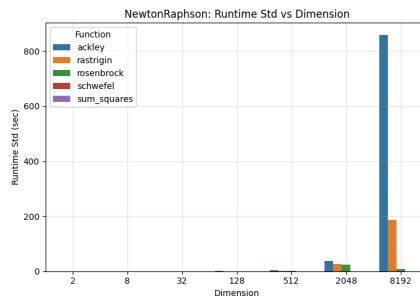


Figure 13: Newton-Raphson Runtime vs. Dimension (Matplotlib)

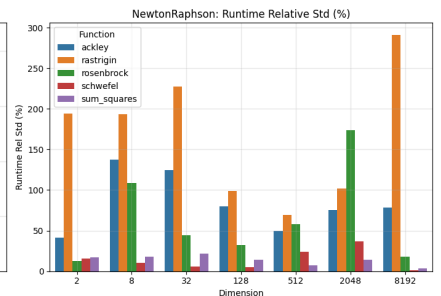


Figure 14: Newton-Raphson Runtime Variability (Relative Std) vs. Dimension (Matplotlib)

Initially, I tried using **bar charts** (Figures 12–14) and **violin plots** (Tanious & Manolov, 2022) (Figure 11) to visualize the runtime data for five functions (each repeated **15 times** across multiple dimensions) per optimizer. However, I found these plots became **cumbersome** and **visually cluttered** when dealing with so many different dimensions and functions. It also proved difficult to spot which runs or dimensions were **outliers** or showed **unusually high** variability.

Confusion-Matrix Layout

Hence, I adopted a **confusion-matrix-style** layout to display **mean** and **RelativeStd** side by side for each dimension and function. By organizing the data into a grid—functions as rows and dimensions as columns—this format clarifies which scenarios produce high or low runtimes and variability. It also makes direct comparisons between Gradient Descent and Newton Raphson more straightforward, highlighting exactly where one method excels or struggles.

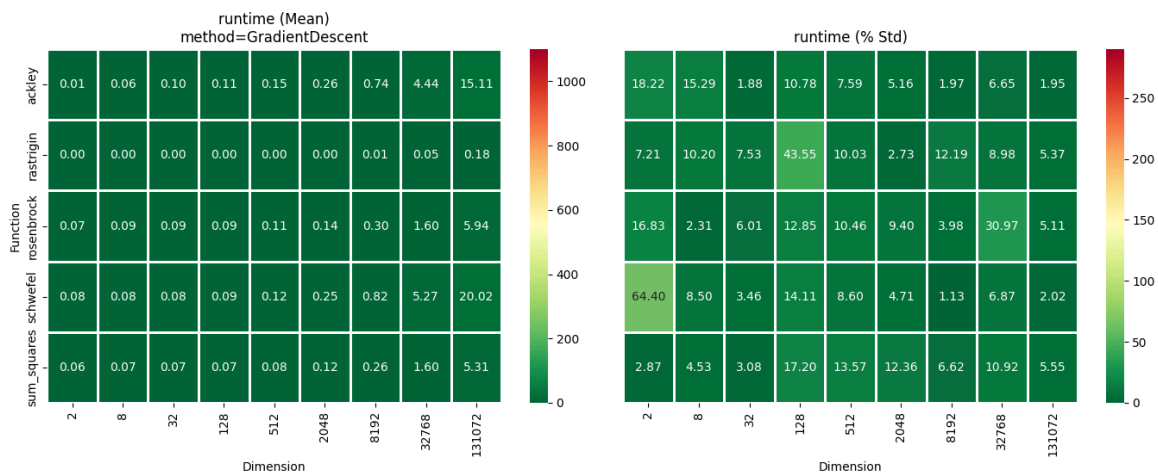


Figure 15: Confusion-Matrix of Mean Runtime and Relative Std for Gradient Descent (Matplotlib)

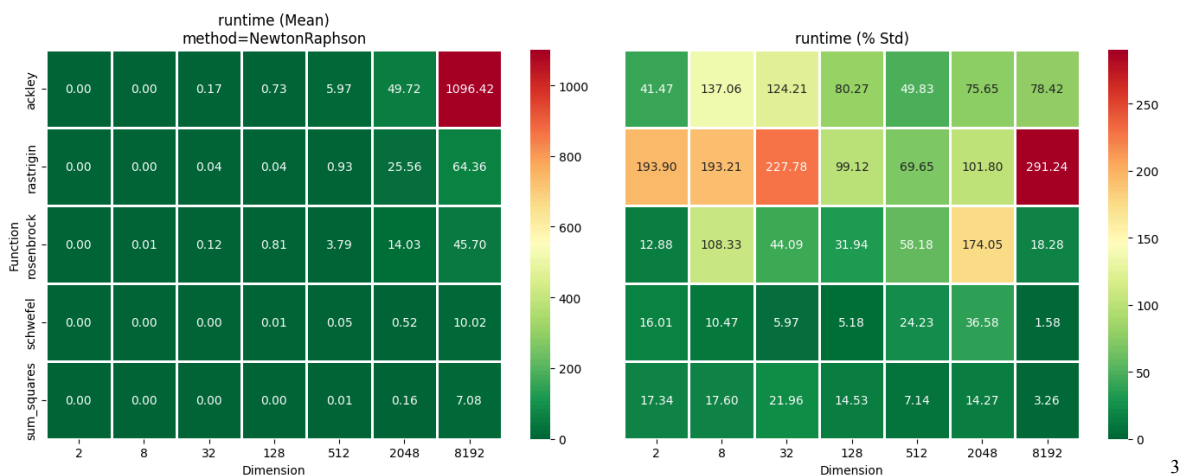


Figure 16: Confusion-matrix of Mean Runtime and Relative Std for Newton-Raphson (Matplotlib)

³ For all confusion plots in Sections 4.2–4.4, I kept their gradient bars' ranges uniform across both optimizers, ensuring that a “dark red” area reflects the same numeric range in each matrix, preserving consistency and reducing potential misinterpretations.

From **Figures 15–16**, several trends emerge. Consistent with **Section 4.2.1**, **Newton Raphson** becomes prohibitively slow at **higher dimensions**, in line with its $\mathcal{O}(n^3)$ Hessian inversion cost (Haramoto & Matsumoto, 2008). Meanwhile, **Gradient Descent** maintains a significantly **lower** runtime overall (Figure 16).

Interestingly, Newton Raphson is **computationally cheaper** than Gradient Descent on **Schwefel** and **Sum of Squares**, likely due to Newton’s rapid **quadratic convergence** property when applied to paraboloid function landscapes (Tibshirani, 2015), thanks to Newton’s Hessian-based updates (see **Section 3.2**). Hence, it achieves rapid convergence (less runtime, higher efficiency) on functions which landscape largely mimics a quadratic minima (see **Table 1** for Schwefel and Sum of Squares function landscape). However, at **higher dimensions**, this advantage diminishes and Newton Raphson still lags behind Gradient Descent for these functions.

Both optimizers slow down on the **Ackley** function, but Newton Raphson fares worse (higher runtime)—probably because **saddle points** and **flat regions** degrade the utility of Hessian-based updates, causing overshoot or stalling. Gradient Descent, on the other hand, likely performed worse (higher runtime) in efficiency because it stalled near minima as its updates become small, mirroring the behavior observed in **Section 3.3** (Figure 4).

Similar to our observations from Section 4.2.1, examining the **RelativeStd** (right-hand side of **Figure 15**) shows that **Gradient Descent** remains **remarkably consistent**, often below **15%**, whereas Newton Raphson’s **RelativeStd** is quite high for most functions (except for **Sum of Squares** and **Schwefel**). This implies that Gradient Descent is less sensitive to **varying initial conditions**⁴, which is preferable in large-scale AI training where one cannot guarantee a favorable starting point.

Even at lower dimensions (2D to 8D) where Newton Raphson’s **mean runtime** is sometimes lower than Gradient Descent, its overall higher **RelativeStd** (Figure 15 and 16) points to **inconsistent** performance—undesirable in applications like AI training (Zhou et al., 2020)⁵, where **predictability** and stable compute demands matter.

Overall, these observations suggest that **Gradient Descent** not only achieves a **lower mean runtime** at scale but also offers **more reliable** performance (low RelativeStd). While Newton Raphson may have a theoretical edge at **small** dimensions (see **Figure 9**), its **high** variability and unpredictability undercuts its practical value in most **real-world** applications (Zhou et al., 2020).

⁴ A higher RelativeStd value indicates greater variability among trials. For instance, if ten runs produce similar results, the RelativeStd remains low—an indication of consistency. Conversely, if those same runs produce widely scattered outcomes, the RelativeStd becomes high, revealing inconsistency. Throughout Sections 4.2–4.4, a lower RelativeStd is therefore preferred.

⁵ Though this paper did not specifically discuss Newton-Raphson and Gradient Descent, its arguments about consistency for different optimizers used in AI training still apply to our context. Hence, I am using it as a reference.

4.2.3 Statistical Validation (Mann–Whitney Test)

Initially, I considered using **t-tests** to compare the optimizers’ runtimes. However, the **violin plots** (Figure 11) showed that neither **Gradient Descent** nor **Newton Raphson** followed a **normal** distribution. Consequently, I turned to **nonparametric** methods—specifically the **Mann–Whitney U** ⁶ test (GraphPad, 2025)—to compare the two optimizers directly.

Mann–Whitney U Test

1. Hypothesis Setup

- H_0 : The distribution of runtimes for Newton Raphson is the **same** as that for Gradient Descent.
- H_1 : The two runtime distributions **differ** significantly.

2. Test Procedure

- **Combine and Rank All Data**

For two independent samples of sizes n_1 and n_2 , I pooled the runtimes from both optimizers and assigned **ranks** to each observation (lowest runtime gets rank 1, etc.).

- **Compute U**

Let R_1 be the sum of ranks for group 1 (e.g., Newton Raphson). The Mann–Whitney statistic U is (GraphPad, 2025):

$$U = R_1 - \frac{n_1(n_1+1)}{2}.$$

If U is **too small** or **too large** relative to its expected value under H_0 (“both groups share the same distribution”), we obtain a **low p -value**.

- **Interpret p -value**

- If $p < 0.05$, **reject H_0** $\Rightarrow$ The runtime distributions **differ** significantly.
- If $p \geq 0.05$, **fail to reject H_0** $\Rightarrow$ No statistically significant difference is detected.

3. Rationale and Advantages

This nonparametric approach accounts for **non-normal** data and potential **outliers**, so relying purely on means or raw standard deviations (both easily skewed) is unnecessary. A **low p -value** (< 0.05) indicates a strong difference between the two optimizers, whereas a **high p -value** (≥ 0.05) suggests we cannot conclude a significant difference.

⁶ Initially, I also tried testing some multi-group comparisons using Kruskal–Wallis as well—e.g., across all benchmark functions. But these were omitted as I ultimately found dimension-by-dimension results most informative, so I focus here on Table 4 only.

I then ran these tests in Python (via NumPy) and the respective results are:

Dimension	n_{m1} (Newton)	n_{m2} (Gradient)	U_{stat}	p -value	Conclusion
2	75	75	52.0	3.25×10^{-25}	Significant ($p < 0.05$)
8	75	75	448.0	6.36×10^{-19}	Significant ($p < 0.05$)
32	75	75	1966.0	1.47×10^{-3}	Significant ($p < 0.05$)
128	75	75	2547.0	3.19×10^{-1}	Not Sig ($p \geq 0.05$)
512	75	75	3645.0	1.76×10^{-3}	Significant ($p < 0.05$)
2048	75	75	5145.0	1.86×10^{-18}	Significant ($p < 0.05$)
8192	75	75	5625.0	4.13×10^{-26}	Significant ($p < 0.05$)

Table 3: Mann–Whitney by Dimension ($\leq 8192D$)⁷

With the exception of **128D** ($p \geq 0.05$), most p -values fall **below 0.05**, implying **rejecting** H_0 . This matches the **confusion-matrix** findings (Figures 15-16) that **Gradient Descent** consistently outperforms **Newton Raphson**, indicating they do **not** share the same runtime distribution. Moreover, the **very low** p -values suggest these differences stem from **genuine** performance gaps rather than random noise.

Hence, this statistical confirmation aligns with observations in **Section 4.2.2**, where **Gradient Descent** showed both **faster** runtimes and **lower** variability at higher dimensions. Taken together, these Mann–Whitney results provide robust evidence that **Gradient Descent** is more **efficient**, reinforcing its suitability for large-scale AI training, where **computational cost** is a critical concern.

4.3 Effectiveness of Optimizers (Success Analysis)

4.3.1 Failure Cases (NaN)

experiment_num	timestamp	function	dimension	method	success	iterations	runtime	f_initial	f_final	initial_distance_to_minimum	final_distance_to_minimum
1	20250117_105833	sum_squares	131072	GradientDescent	False	2000	4.97354531288147	912999.0749794078	nan	3.7345715272787166	nan
2	20250117_105839	sum_squares	131072	GradientDescent	False	2000	5.730066776275635	863877.2091724935	nan	3.635001310448228	nan
3	20250117_105844	sum_squares	131072	GradientDescent	False	2000	4.939152479171753	676503.2731234576	nan	3.2124060319424688	nan
4	20250117_105850	sum_squares	131072	GradientDescent	False	2000	5.371228218078613	994058.6462337162	nan	3.8970954391005055	nan
5	20250117_105855	sum_squares	131072	GradientDescent	False	2000	4.818073987960815	1186037.3508206825	nan	1.255337636472533	nan
6	20250117_105901	sum_squares	131072	GradientDescent	False	2000	5.70385217666626	1108702.5954728127	nan	1.111094225331319	nan
7	20250117_105906	sum_squares	131072	GradientDescent	False	2000	5.312781572341919	936565.1217918136	nan	3.777267459041705	nan
8	20250117_105912	sum_squares	131072	GradientDescent	False	2000	5.582723140716553	427572.6831920028	nan	2.5492610588469016	nan
9	20250117_105917	sum_squares	131072	GradientDescent	False	2000	4.999147891998291	563629.0646460954	nan	2.937778484434307	nan
10	20250117_105923	sum_squares	131072	GradientDescent	False	2000	5.348104000091553	554325.5458388906	nan	2.9031717842740523	nan
11	20250117_105928	sum_squares	131072	GradientDescent	False	2000	5.577195167541504	581386.0591957989	nan	2.979221494813494	nan
12	20250117_105934	sum_squares	131072	GradientDescent	False	2000	5.133015155792236	1217180.894266979	nan	3.312502189016412	nan
13	20250117_105939	sum_squares	131072	GradientDescent	False	2000	5.500269174575806	1557365.9529762617	nan	1.8760189213090115	nan
14	20250117_105945	sum_squares	131072	GradientDescent	False	2000	5.164628982543945	685035.2937166573	nan	3.235396293247048	nan
15	20250117_105950	sum_squares	131072	GradientDescent	False	2000	5.546295881271362	1551644.3202112527	nan	1.86563117904869	nan

Figure 17: Raw Experiment Logs Showing NaN Failures

⁷ We only compare up to 8,192 dimensions here because this is the highest dimension tested for the Newton Raphson method. While gradient descent was tested up to 131,072 dimensions, our comparison stops at 8,192 to maintain consistency across both methods.

When I created **Figure 19**, I noticed an unexpected outcome: several runs ended in **NaN (Not a Number)**. To find the cause, I examined the **raw data** in **Figure 17** and observed that some run's f_{final} (the function value at run completion) had effectively reached **infinity**, which causes NaN due to arithmetic overflow (ScienceDirect, n.d.). Since optimizers aim to **lower** the function value, any run ending with $f_{\text{final}} = \text{NaN}$ is considered a **failure**.

Reflecting on this, I realized the issue parallels **gradient explosion** (Pascanu et al., 2013) in real-world AI training, where step sizes or gradients become so large that the loss function skyrockets and training diverges. This underscores why methods like **adaptive learning rates** (Xie et al., 2019) and **gradient clipping** (Zhang et al., 2019) are commonly used—without them, certain parameter configurations or large gradients can push the optimizer's steps into **overflow territory** (ScienceDirect, n.d.), causing training runs to fail rather than converge, which is very costly.⁸

I was initially concerned that **Gradient Descent** might simply be “broken” for certain functions. Closer inspection of **Figure 17** revealed multiple instances—most notably all 15 Gradient Descent trials on **sum_squares** at **131,072** dimensions—where the runs failed with $f_{\text{final}} = \text{NaN}$. To better understand this pattern, I plotted a **confusion matrix** indicating the percentage of NaN outcomes per optimizer:

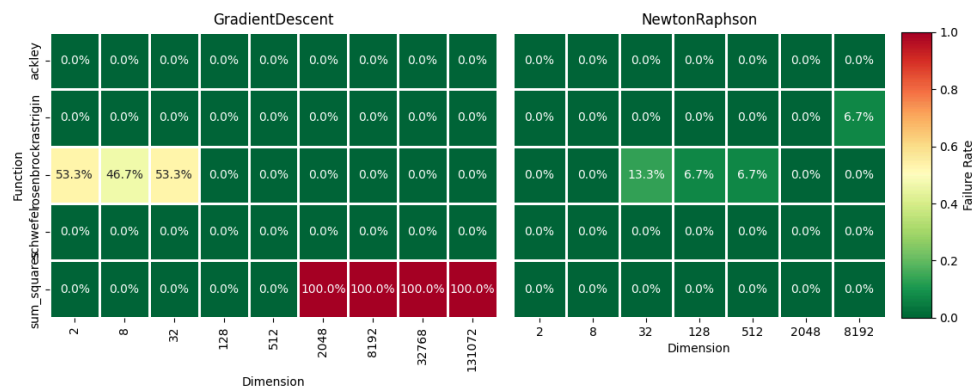


Figure 18: Confusion-matrix of Failure Rates Across Dimensions and Functions (Matplotlib)

After graphing, I observed a **striking** outcome: for **Gradient Descent** on **sum_squares** from **2,048D** to **131,072D**, **100%** of the runs failed. Reflecting on **Figure 17**, it became apparent that this failure is largely due to f_{initial} (function value at start of each run) being **extremely large**—the direct result of my **initial point generation** from **Section 4.1.3**, which permits coordinates to lie between **2** and **5**. Since **sum_squares** (see **Table 1**) scales with the square of each coordinate, high dimensions naturally produce **massive** starting sums. For instance, at **131,072D**, f_{initial} is:

⁸ This is mentioned in Section 6 as a future extension of our optimizer investigation, because gradient clipping and adaptive learning rates are techniques used in real-world AI training. However, we will not explore these here since they are outside the scope of this exploration.

⁹ Red signifies higher percentages of failed runs, while green indicates fewer failures, which is preferable.

- **Lower bound** ($x_i = 2$): $\sum_{i=1}^{131,072} (2^2) = 131,072 \times 4 = 524,288$,
- **Upper bound** ($x_i = 5$): $\sum_{i=1}^{131,072} (5^2) = 131,072 \times 25 \approx 3.28 \times 10^6$.

Consequently, in my empirical experiments (see **Figure 17**), given these extreme f_{initial} values, **Gradient Descent** likely encountered **extreme initial gradients** and subsequently failed to converge (with f_{final} being infinitely huge, indicating run failure).

Such **large** f_{initial} values are **unrealistic** in typical neural network training, where initial f_{initial} are usually more **conservative** (Scabini et al., 2024). Hence, I do **not** attribute these failures to an inherent weakness in Gradient Descent but rather to my **experimental design**. Instead, in future work, I plan to refine the **initial-distance constraints** for each benchmark function, instead of using a single range of $[2, 5]$ for all.¹⁰

Interestingly, **Newton Raphson** did **not** fail under the same conditions at **2,048D** or **8,192D**, possibly because its Hessian-based method adapts more naturally to **bowl-shaped** functions like **sum_squares**. One could argue this indicates greater stability in principle. However, since **sum_squares** is fundamentally **quadratic**, Newton Raphson’s strong performance here is **not** surprising—it converges quickly in strictly quadratic landscapes (Tibshirani, 2015). Therefore, I do **not** conclude that Gradient Descent is *objectively worse*; these high-dimensional failures are better explained by my **initialization scheme**, which does **not** reflect real neural-network practices.

4.3.2 Relative Improvement of Final Function Value

In this subsection, I analyse **how much** each optimizer reduces the function value from its **initial** to its **final** state. Since the ultimate goal of optimizers is to make f_{final} as low as possible, I employ a **normalized** metric—rather than raw differences—to fairly compare optimizer improvements. For instance, going from $f_{\text{initial}} = 100$ to $f_{\text{final}} = 80$ is a drop of 20, but only a 20% improvement; meanwhile, moving from 10 to 0 represents a full 100% improvement. By focusing on a **ratio** rather than an absolute difference, I avoid distorting the results when initial values are especially large.

$$\text{RelativeImprovement} = \frac{f_{\text{initial}} - f_{\text{final}}}{|f_{\text{initial}}|}.$$

Hence, to **implement** this approach, I programmed my program such that it records this “Relative Improvement” measure for each run.

In **Figures 19** and **20**, I **filtered out** any NaN runs or “unsuccessful” runs that ended in implausibly large final values from analysis—primarily those linked to **sum_squares** failures at high dimensions (see **Figure 18** and **Section 4.3.1**). Though these omissions might seem to favor Gradient Descent, I consider them a **necessary** correction, given the **sum_squares** issues stemming from my **experimental design** flaw rather than an optimizer deficiency.

¹⁰ This methodology limitation will be elaborated further in Section 6.

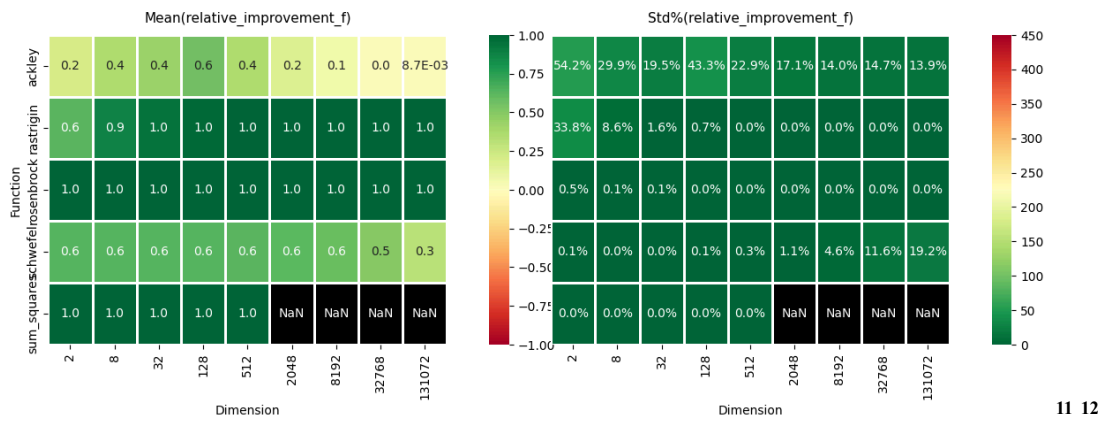


Figure 19: Confusion-Matrix of Gradient Descent's Relative Function Value Improvement (Matplotlib)

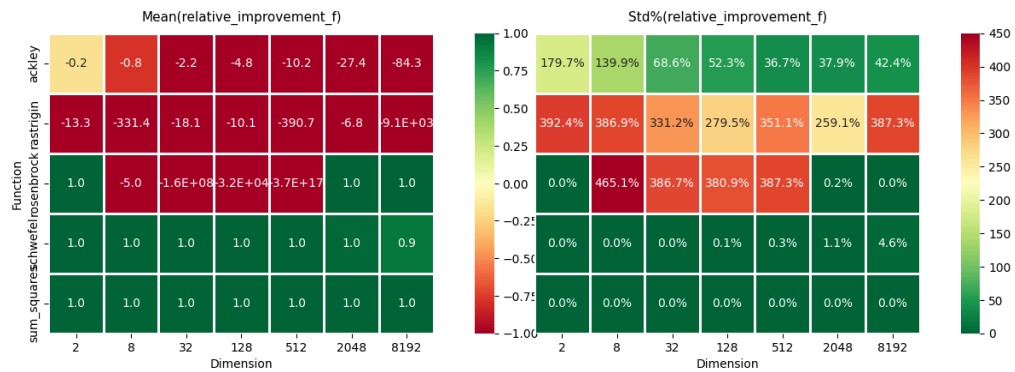


Figure 20: Confusion-Matrix of Newton-Raphson's Relative Function Value Improvement (Matplotlib)

Observing **Figures 19** and **20**—where the color scale remains consistent—shows that **Gradient Descent** frequently achieves **100%** improvement, whereas **Newton Raphson** occasionally ends with **extremely large** final values (sometimes 3.7×10^{17} times worse than the initial point, as on Rosenbrock at **512D**). Although Newton Raphson performs well on certain **Hessian-friendly** functions (e.g., **Schwefel** and **sum_squares**), the broader evidence suggests **Gradient Descent** is more **consistently** effective across most test functions. This is highly beneficial for **AI training**, where an optimizer must handle diverse function landscapes (Geiger et al., 2021) and successfully minimise the function, which Gradient Descent demonstrates.

Additionally, **Gradient Descent** maintains a **lower** RelativeStd across the board, mirroring the **runtime** results in **Figures 15** and **16**, where it exhibited a **lower RelativeStd**—indicating performance that is **less sensitive** to varying initial conditions. In **Figures 19–20**, for instance, Gradient Descent continues to achieve significantly **lower** RelativeStd. This consistency is crucial in **large-scale** AI contexts, where **reliability** under varied initial conditions is paramount (Zhou et al., 2020).

¹¹ In these confusion matrices, a value of +1.0 (indicating a 100% improvement in the function) is labeled in green. Conversely, negative values (for example, -1.0, which means the function value doubled) are labeled in red to highlight a worse outcome.

¹² For confusion matrices from Section 4.3-4.4, I unfortunately resorted to scientific E-notation in them instead of standard notation. Because customizing number formats in Matplotlib is extremely complex in Python. Additionally, I restricted values to one decimal place to avoid clutter—the extra digits would otherwise overwhelm the limited space.

4.4 Per-Iteration Analysis of Optimizers

In this section, I examine how each optimizer behaves **per timestep**, aiming to determine whether one might be **superior** for particular problem characteristics. As noted in **Sections 2–3**, **Gradient Descent** and **Newton Raphson** are **iteration-based** optimizers, meaning they operate through multiple iterations. Consequently, analyzing their **per-iteration** performance metrics is very useful because this allows us to directly look into how they truly perform.

4.4.1 Gradient Cosine Similarity

A key metric I analysed is the **gradient cosine similarity**, which indicates how frequently the optimizer changes its direction when traversing a function landscape per-iteration:

$$\cos(\theta) = \frac{\nabla f(\mathbf{x}_t) \cdot \nabla f(\mathbf{x}_{t+1})}{\|\nabla f(\mathbf{x}_t)\| \|\nabla f(\mathbf{x}_{t+1})\|}.$$

In **AI training** contexts, **rapidly changing** directions can hinder stable training, so ideally we want $\cos(\theta)$ near **1.0**, implying **small** directional shifts (Xing et al., 2018). Initially, I considered using the actual angle θ , but I chose $\cos(\theta)$ instead, as analyzing raw angles with mean and RelativeStd would be misleading. In contrast, $\cos(\theta)$ provides a normalized, scale-independent measure, making it more suitable for fair comparisons.

Accordingly, I **programmed** my code to **calculate and log** the gradient cosine similarity at each iteration. I then computed the **average** $\cos(\theta)$ for each run and plotted the results below. Similar to **Section 4.3.2**, I filtered out **failed runs** to keep the confusion matrices from being skewed. Like before, these failed runs stem from my **experimental design flaw**, rather than the optimizers themselves. Hence, I will analyse the results below as if they are fully **representative** of each optimizer’s typical behavior.

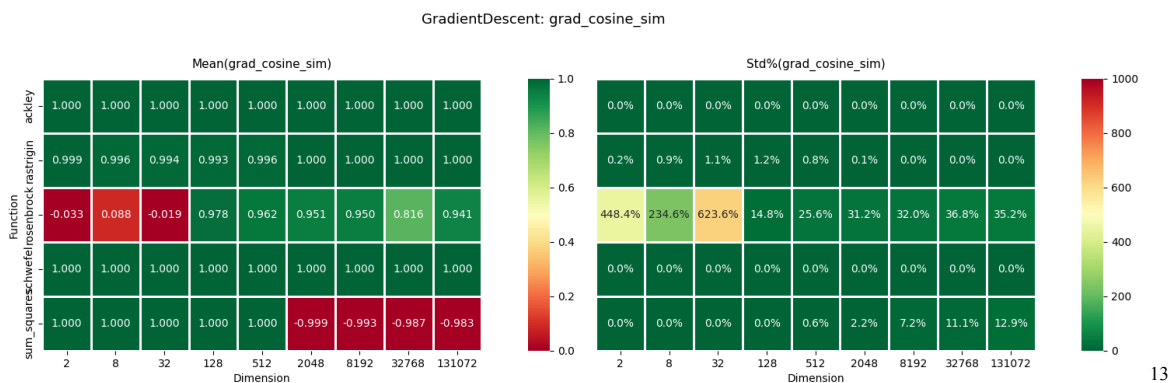


Figure 21: Confusion-Matrix of Gradient Descent’s Gradient Cosine Similarity (Matplotlib)

¹³ Green corresponds to $\cos(\theta)=1.0$, indicating minimal directional change—ideal for AI training, which favors gradual updates. Red appears at 0 or below, reflecting near-orthogonal or reversed gradient directions.

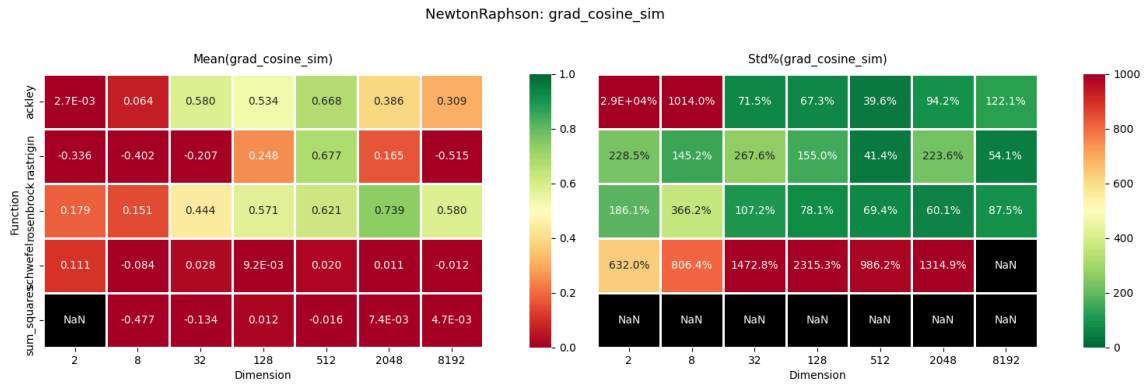


Figure 22: Confusion-Matrix of Newton Raphson's Gradient Cosine Similarity (Matplotlib)

Gradient Descent:

I excluded the “red boxes” in **Figure 21** pertaining to **sum_squares** from **8192D** to **131,072D**, as they stem from an **experimental design** flaw. Apart from these outliers, **Gradient Descent's** mean cosine similarity hovers near **1.000** (to three decimals) despite being faced with vastly different function landscapes. This property is **highly** desirable for AI training, because **smaller, steady** updates are typically preferred over **large, erratic jumps**—allowing the model to learn **gradually** from the broad distribution of its training data (Xing et al., 2018). Moreover, consistent with **Section 4.2.3**, Gradient Descent again exhibits **low** RelativeStd (see **Figure 21**), indicating consistent good performance despite faced with varying initial conditions (Zhou et al., 2020). Overall, this points to **Gradient Descent** being the **better** choice in scenarios like AI-training which demands smooth, incremental convergence.

Newton Raphson:

By contrast, I notice **unstable** mean cosine similarities for **Newton Raphson**, with the angle consistently near **90 degrees** ($\cos(\theta) = 0$). This is not ideal—particularly for neural network training—because it indicates that it's changing very **rapidly**. As mentioned before, **AI model training** prefers **slow** updates to avoid divergence.

The **RelativeStd** value is also **very high** (see Figure 22), suggesting initial conditions strongly affect Newton Raphson—a negative trait for AI training, as it implies **inconsistency**. I believe these properties arise from Newton Raphson's **quadratic convergence** (relying on Hessians) (Tibshirani, 2015). As noted in **Section 3.3**, Hessians can shift abruptly, enabling Newton Raphson to find minima quickly but also causing failures in varied landscapes or around saddle points (see **Section 4.3**). Consequently, these observations reinforce that **Newton Raphson** is poorly suited for large-scale AI scenarios, given its inherent **inconsistencies**—performance varies too much due to varying initial conditions (Zhou et al., 2020).

4.4.2 Step-Size Variation

Another performance metric I explored is the **step size** per iteration for both optimizers, indicating **how far** each method moves in parameter space at every update. Formally, the step size is defined as:

$$\Delta x_t = \|\mathbf{x}_{t+1} - \mathbf{x}_t\|.$$

By examining this measure, I aimed to gain **additional insights** into each optimizer’s behavior. Accordingly, I programmed my code to **calculate and log** the step size at each iteration, then averaged those values for each run. In **neural network** contexts, **smaller, incremental** updates are typically preferred, allowing the model to learn **gradually** from a broad distribution of data (Nar & Sastry, 2018). Hence, in **Figure 23** and **24**, I used a **red–green** color scheme: **red** indicates a **larger** mean step size, while **green** denotes a **smaller** one.

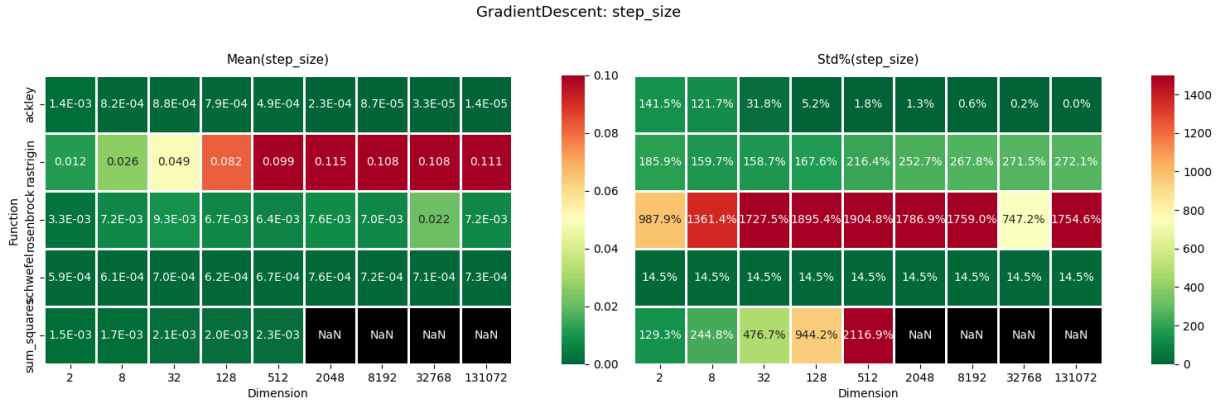


Figure 23: Confusion-Matrix of Gradient Descent Step Sizes (Matplotlib)

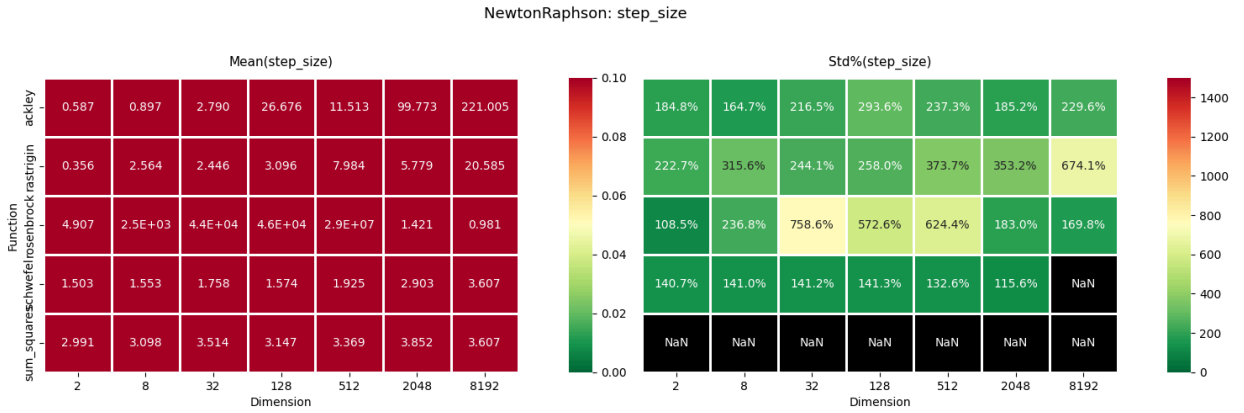


Figure 24: Confusion-Matrix of Newton Raphson Step Sizes (Matplotlib)

From these plots, **Gradient Descent** generally shows a **lower** mean step size than **Newton Raphson** (see **Figures 23–24**). Interestingly, Gradient Descent’s consistency sometimes **lags** behind Newton Raphson, as indicated by a **higher** RelativeStd in certain cases—suggesting that **both** methods’ step sizes depend on **initial conditions** (Zhou et al., 2020). However, since Newton Raphson’s RelativeStd is typically **worse** overall (i.e., higher RelativeStd, meaning lower consistency), we cannot conclude which optimizer is *definitively* more consistent. For Gradient Descent, a **notable exception** of consistency arises in the **Rosenbrock** function. I believe the **inconsistency** here is due to Rosenbrock’s **curved valley** (see **Table 1**); in that landscape, Gradient Descent likely takes **larger** steps in steep regions but **slows dramatically** in flatter terrains or at saddle points, hence the **initial condition** matters substantially—even for Gradient Descent, which otherwise demonstrated **remarkably** low RelativeStd across most benchmarks.

By contrast, **Newton Raphson** tends to have a **higher** overall step size, often converging in **fewer** iterations (see **Figures 7 and 10**). I attribute this to **Newton** being a **Hessian-based** method, “jumping” more aggressively toward local minima—especially when the Hessian is well-conditioned in a **quadratic** space (like **sum_squares**, effectively a simple bowl, explaining Newton’s notably **higher** mean step sizes in **Figure 24**).

Overall, I conclude that **Gradient Descent** is generally **preferable** due to its smaller step sizes, which is desirable in **AI training** (Nar & Sastry, 2018). However, this advantage is **not** absolute, as an overly small step can lead to **gradient vanishing** during AI training. Furthermore, **both** optimizers appear **inconsistent**, given their mixed and high RelativeStd results. This is **expected**: in functions like **Rosenbrock** (Table 1), the **initial condition** heavily influences step sizes, meaning an optimizer starting near the global minimum or a saddle may dramatically alter **how quickly** or **effectively** it progresses.

4.4.3 Statistical Testing of Iteration-Level Metrics

We rely on the **same** Mann–Whitney testing (GraphPad, 2025) approach and hypothesis setup as in **Section 4.2.3**, this time applying it to **per-iteration** metrics (gradient cosine similarity and step size). Our goal is to see whether different functions or optimizers produce **statistically different** per-iteration distributions.

Below are the Mann–Whitney results for **gradient cosine similarity** (Table 4) and **step size** (Table 5), each examined across all tested dimensions (up to 8,192D). Every comparison is **highly significant** ($p < 0.05$), reinforcing our earlier observations that **Newton Raphson** and **Gradient Descent** exhibit **markedly different** per-iteration behaviors.

dimension	p -value	conclusion
2	4.67×10^{-15}	Significant ($p < 0.05$)
8	8.91×10^{-17}	Significant ($p < 0.05$)
32	8.66×10^{-17}	Significant ($p < 0.05$)
128	4.12×10^{-25}	Significant ($p < 0.05$)
512	7.81×10^{-24}	Significant ($p < 0.05$)
2048	1.89×10^{-9}	Significant ($p < 0.05$)
8192	3.27×10^{-10}	Significant ($p < 0.05$)

Table 4: Mann–Whitney Test Results for Gradient Cosine Similarity

dimension	p -value	conclusion
2	9.18×10^{-17}	Significant ($p < 0.05$)
8	8.21×10^{-18}	Significant ($p < 0.05$)
32	9.18×10^{-17}	Significant ($p < 0.05$)
128	4.13×10^{-26}	Significant ($p < 0.05$)
512	4.13×10^{-26}	Significant ($p < 0.05$)
2048	2.24×10^{-10}	Significant ($p < 0.05$)
8192	2.24×10^{-10}	Significant ($p < 0.05$)

Table 5: Mann–Whitney Test Results for Step Size

Overall, these **extremely low** p -values (Tables 4–5) **decisively reject** H_0 (that the distributions are similar), thereby **strongly supporting** the arguments I made in **Sections 4.4.1 and 4.4.2**: the per-iteration metrics (gradient cosine similarity and step size) vary **significantly** depending on which optimizer is used and on the function landscape. Even the **highest** p -value (1.89×10^{-9} at **2048D** in Table 4) remains **remarkably small**, underscoring how **Newton Raphson** and **Gradient Descent** diverge **significantly** in their per-iteration

behavior. Hence, my claim that **Gradient Descent** is superior is justified, because the **Mann–Whitney** test demonstrates these differences arise from **genuine** performance gaps rather than random noise.

4.5 Real-World AI Training: Gradient Descent vs. Newton–Raphson

In this section, I decided to test both **Gradient Descent** and **Newton Raphson** in a real-world scenario to better evaluate them. That is, we are testing how both optimizers fare when used in actual AI training. Specifically, I implemented a **simple CNN** (O'Shea & Nash, 2015) in **PyTorch** (Paszke et al., 2019), training on the **MNIST** dataset (from Yann Le Cun) (Le Cun, n.d.). To accelerate training, I ran experiments on an **Nvidia A100 GPU** (a specialised AI training chip) (Nvidia, n.d.) via Google Colab (Google, n.d.), which is far faster than the CPU environment used in **Sections 4.2–4.4**. The **full code** for these runs is provided in **Appendix B**, and the trained models are stored on HuggingFace (Hugging Face, 2025).

For analysis, I have logged the training loss and time required for both optimizers.

Statistic	Gradient Descent	Newton Raphson
Initial Loss	2.29	2.30
Final Loss	0.15	6.58
Total Steps	7,500 (Finished)	~372 (Failed to Finish)
Total Time (s)	61.03	3,391.56
Steps per Second	122.89	0.11

Table 6: Key Metrics of AI Training

Comment on Training Efficiency

Initially, I intended to train a **million-parameter** model with both **Gradient Descent** and **Newton Raphson**, as modern AI models can span from **billions** to **trillions** of parameters. This means the chosen optimizer would have to **navigate** an extremely large—potentially **trillion-dimensional**—space, which would be an **ideal** test to simulate real-world scenarios where optimizers must be **deployable**. However, I soon realized this was **not** feasible, because **Newton Raphson** grows **extremely slow** as parameter counts increase. Consequently, I scaled down to a **10,000-parameter** model for a direct comparison between Newton Raphson and Gradient Descent.

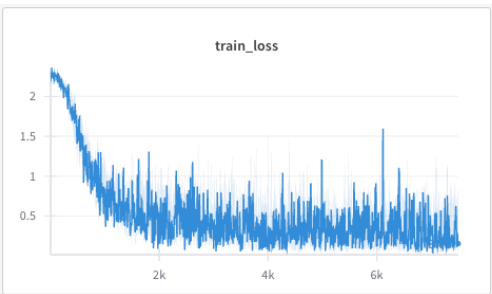


Figure 25: Gradient Descent Training-Loss Curve
(Weights & Biases)

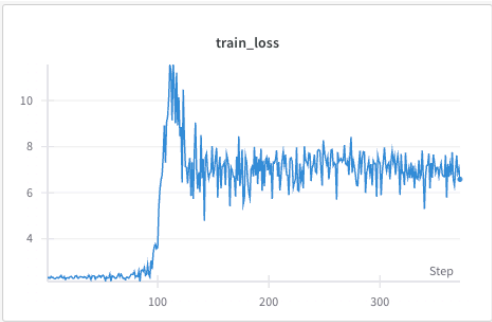


Figure 26: Newton–Raphson Training-Loss Curve
(Weights & Biases)

From **Table 6**, we observe that **Gradient Descent** is *impressively fast*, processing **122.9 steps per second**¹⁴ and finishing in **61 seconds** total. In contrast, **Newton Raphson** is *painfully slow*, performing **0.11 steps per second**—which is expected, given Hessian inversions’ costs are $\mathcal{O}(n^3)$ (Haramoto & Matsumoto, 2008)—which is **more than 1000 times** slower than Gradient Descent. This finding mirrors **Section 4.2** (Figure 8), where Newton’s cost soared rapidly with dimension. Considering our observations in **Sections 4.2** and **4.5** where Newton is significantly more inefficient than Gradient Descent, I can confidently conclude that for training huge AI models at higher dimensions (e.g., **ChatGPT-4** with **1.8 trillion** parameters) (Schreiner, 2023), Newton Raphson would be **unbearably** expensive to deploy in practice.

Unfortunately, Newton’s efficiency is so poor that this effectively puts the “final nail in the coffin” for use of Newton-Rhapson in large-scale AI training. Observing how long the Newton run took (nearly an hour as shown in **Table 6**), I decided to **stop** the run at **372** steps out of the planned **7,500** (3,400 seconds, costing about USD 1 on Colab for an A100). Moreover, I believe this decision was **justified**, given there was **no sign** of the training loss improving (**Figure 26**).

Comment on Training Loss

Before the AI model using Gradient Descent was successfully trained (Figure 25), I had to **tune** the learning rate—showcasing the real-world **nuances** of AI optimization. Because in real world scenarios, like AI training, it’s not just about *which* optimizer one picks; tuning hyperparameters, such as an **appropriate** learning rate is also crucial (IBM, 2024). As these hyperparameters determine if an optimizer can be successfully used.¹⁵

- **Gradient Descent** initially failed when the learning rate was **1.0**; the **training loss** (i.e., model error) stalled around $\ln(20) \approx 2.3$. By **gradually** lowering the learning rate, I settled on **0.01**, which allowed the model to **converge** successfully. The initial training loss ($\ln(20)$) is a typical baseline for an untrained CNN model; eventually, **Gradient Descent** reduced this to **0.15**, indicating a successful run.
- **Newton Raphson**, however, kept failing, even after I repeatedly **shrunk** its learning rate to **0.001**, **0.0001**, and **0.00001**. Its training loss soared above the usual baseline ($\ln(20) \approx 2.3$), leveling off around **6.5** (Figure 26). Hence, we can **decisively** say it failed. This failure is likely caused by Newton Raphson’s **aggressive** direction updates (see **Section 4.4.1**, Figure 22), stemming from its Hessian-based nature, and large step sizes (see **Section 4.4.2**, Figure 24) that’s **undesirable** for AI training. As mentioned before, AI training typically favors **smaller** step sizes, so that the model can **gradually** learn from a huge dataset instead. Although Newton Raphson’s **quadratic convergence** allows it to find minima very rapidly under ideal (bowl-shaped) conditions, it proves **unhelpful** for

¹⁴ AI models rely on iteration-based optimizers like Gradient Descent, meaning more steps per second is preferable. For instance, if training requires 10,000 steps, a higher step rate finishes the training faster, saving computational cost.

¹⁵This is discussed further in Section 6 as a possible future extension to this exploration. In addition, the learning rate variable appears in both Section 3.1 and 3.2; however, because learning isn’t the focus of this exploration, I didn’t explore it in detail.

slow, step-by-step learning in real-world AI tasks—hence Newton Raphson is **decisively worse** than Gradient Descent for AI training.

Overall, **Gradient Descent** emerges as the **better** option, being **significantly more efficient** (Table 6) in practice—a result consistent with the trends noted in **Section 4.2**. Moreover, **Gradient Descent’s** successful training loss reduction (Figure 25) aligns with the observations from **Section 4.3**, further confirming its superiority.

5 – Conclusion

Based on the **empirical findings** from this exploration, we can conclude which optimizer—Gradient Descent or Newton Raphson—performs better overall:

1. Efficiency

From Section 4.2, we observed that **Gradient Descent** is decisively more **efficient** than Newton Raphson, as its runtime remained **under 15 seconds** (even at **131,072D**), while Newton Raphson’s grew prohibitively long. Moreover, Gradient Descent displayed a significantly **lower** RelativeStd value, indicating **consistent** efficiency (good for AI training). Our **Mann–Whitney** test confirmed that these runtime differences between the two optimizers are **statistically** significant rather than just random noise.

Our Section 4.5’s real-world test further strengthened this conclusion about efficiency, with Gradient Descent running more than **1,000 times** faster than Newton Raphson—a paramount advantage given the **millions of USD** often required for large-scale AI training (Buchholz, 2024).

2. Effectiveness

In **Section 4.3.2**, we observed that **Gradient Descent** consistently outperforms **Newton Raphson** across various benchmark functions, often achieving up to a **100%** improvement in f_{final} . Moreover, **Gradient Descent’s** RelativeStd for its improvements remains **consistently lower** than Newton Raphson’s, indicating not only **superior effectiveness** in high-dimensional minimization but also **greater consistency**—crucial for reliable optimization.

Turning to **Section 4.4**, our **per-iteration** analysis reinforced the conclusion that Gradient Descent is **better suited** for AI training. In **Section 4.4.1**, for instance, we found it has a **smaller** gradient cosine similarity (and lower RelativeStd), implying more stable, incremental updates (desired for AI training). Similarly, **Section 4.4.2** highlights Gradient Descent’s **smaller** average step sizes (desired for AI training) compared to Newton Raphson, preventing overshoot or instability. Both findings were

substantiated by **Mann–Whitney** tests, which confirmed that these differences are **statistically significant**—not merely random variation.

Most importantly, our **empirical** results in **Section 4.5** (the real-world test, AI training) validate these observations: **only Gradient Descent** successfully converged, reducing the training loss to a **low** value, while Newton Raphson failed to converge.

Hence, to a large extent, I believe our empirical observations in this exploration **clearly** demonstrate that **Gradient Descent** is **superior** for AI training, given its **greater** effectiveness in minimizing various high-dimensional functions and its **significant** efficiency, which helps **reduce** overall computational costs.

6 – Evaluation

Methodology Improvement - Refined Initial-Point Strategies

In our original design (Section 4.1.3), we applied the same initial bounds $[2, 5]$ for all five benchmark functions. As discussed in Section 4.3.1, this “one-size-fits-all” approach proved inappropriate for functions like Sum of Squares, where it produced very large initial function values (Figure 17), often causing Gradient Descent to fail. This highlights a flaw in our experimental setup—namely, that applying identical bounds to functions with markedly different characteristics can obscure an optimizer’s true performance.

Function	Minimum Distance ($d_{\min}$)	Maximum Distance (d_{max})
Ackley	10	30
Rastrigin	2	5
Schwefel	1	2
Sum of Squares	1	2
Rosenbrock	0.5	1.5

Table 7: Proposed Function-specific Bounds

To address this issue in future work, I plan to **tailor** the minimum and maximum distances (`min_dist` and `max_dist`) for each function, rather than imposing a single range across the board. Table 7 illustrates how these bounds would be adjusted depending on the function’s behaviour across different inputs. For instance, **Rosenbrock** and **Sum of Squares** will have tighter bounds than **Ackley**, thus preventing artificially high f_{initial} values (e.g., Sum of Squares at 131,072D). By adopting function-specific initialization ranges, we can better isolate each optimizer’s performance and produce more representative results that align with real-world AI training scenarios.

Future Extension - Different Domain-Specific Experiments

So far, this investigation has focused on how the chosen optimizers perform in an AI training context. However, I am also interested to learn about different optimizer requirements in other real-world domains:

1. **CFD (Computational Fluid Dynamics):** Solving large-scale PDEs may reveal how first-order methods (like gradient descent) compare to second-order methods (like Newton-Raphson) when it comes to stability in fluid-flow simulations. (Jameson & Qu, 2010)
2. **Finance:** Portfolio optimization with noisy market data could test each method's robustness under uncertainty. This might be especially relevant for second-order methods if they adapt more finely to volatile input signals (Cornuéjols et al., 2018).
3. **Computational Biology:** Highly non-convex problems such as protein folding could highlight differences between Hessian-based techniques and gradient-descent-type updates in handling complex energy landscapes (Reali et al., 2017).

These real-world scenarios are integral to many fields. By exploring different real-world domains, we can better understand which optimizers excel under different applications, each with distinct mathematical requirements. This guides more tailored and effective optimization strategies. For example, in CFD problems—where the number of dimensions is often smaller—Newton Raphson may outperform Gradient Descent in terms of computational efficiency, as illustrated in Section 4.2.2, given the lower dimensional requirements.

Future Extension - Alternate Benchmark Functions

While Ackley, Rastrigin, Schwefel, Rosenbrock, and Sum of Squares offer a range of diverse landscapes, I plan to include more challenging functions (e.g., **Griewank**, **Lévy**) (Surjanovic et al., 2013) in future explorations to investigate whether they provoke distinct optimizer behaviors—particularly regarding multiple basins and deceptive valleys.

Future Extension - More Advanced Optimizers

Currently, we are comparing only Newton-Raphson and Gradient Descent. However, there are more sophisticated algorithms used in AI training, such as:

1. **Quasi-Newton Methods (BFGS, L-BFGS):**
These methods (Kochenderfer & Wheeler, 2019, 91) approximate Hessians instead of computing them directly, mitigating the $\mathcal{O}(n^3)$ cost associated with Newton-Raphson's matrix inversion (Haramoto & Matsumoto, 2008) while still leveraging some second-order advantages.
2. **First-Order Enhancements (Adam, RMSProp):**
Widely used in neural network training, these methods (Kochenderfer & Wheeler, 2019, 75-79) build on basic gradient descent with features like adaptive learning rates (Xie et al., 2019). Comparing them against both gradient descent and Newton-Raphson would clarify whether second-order approaches can surpass typical first-order improvements in practice.

Future Extension - Hyperparameter Tuning in Neural Networks

Reflecting on Sections 4.3.1 and 4.5, I realized that choosing the optimizer is not the only key factor for successfully training AI models. Because hyperparameter settings—such as **adaptive learning rates** (Xie et al., 2019) and **gradient clipping** (Zhang et al., 2019)—can be just as influential. Hence, in future work, I plan to investigate these tuning strategies alongside second-order methods, assessing whether they can match or even surpass well-tuned first-order approaches.

Future Extension - Scaling Beyond Current Hardware

If I gain access to stronger computational resources, I intend to push beyond 131,072 dimensions for Section 4.2 to 4.4 or experiment with larger AI models (e.g., over 100k parameters) for Section 4.5. This will help determine whether Newton Raphson remains viable at such scales, ultimately validating (or refuting) the notion that second-order methods become unwieldy for massively high-dimensional optimization tasks.

References

- Buchholz, K. (2024, August 26). *The Extreme Cost Of Training AI Models*. Forbes. Retrieved February 11, 2025, from <https://www.forbes.com/sites/katharinabuchholz/2024/08/23/the-extreme-cost-of-training-ai-models/>
- Cornuéjols, G., Peña, J., & Tütüncü, R. (2018). *Optimization Methods in Finance*. Cambridge University Press.
- Cueto-Mendoza, E., & Kelleher, J. (2024, October 28). A framework for measuring the training efficiency of a neural architecture. *Artificial Intelligence Review*, 57. <https://doi.org/10.1007/s10462-024-10943-8>
- Cun, Y. L. (n.d.). *Index of Mnist*. Yann Le Cun. Retrieved February 11, 2025, from <http://yann.lecun.com/exdb/mnist/>
- Erway, J. B. (2006). Iterative methods for large-scale unconstrained optimization. *UC San Diego Electronic Theses and Dissertations*. <https://escholarship.org/uc/item/65t4c5zd>
- Geiger, M., Petrini, L., & Wyart, M. (2021, August 15). Landscape and training regimes in deep learning. *Physics Reports*, 924, 1-18. <https://doi.org/10.1016/j.physrep.2021.04.001>
- Google. (n.d.). *Google Colaboratory*. Google Colaboratory. Retrieved February 11, 2025, from <https://colab.google/>
- GraphPad. (2025). *GraphPad*. Interpreting results: Mann-Whitney test. Retrieved February 11, 2025, from https://www.graphpad.com/guides/prism/latest/statistics/how_the_mann-whitney_test_works.htm
- Gupta, S. (2021, March 15). *Big O — Time Complexity Graph Simplified* | by Ajak Cyer | *Medium*. Ajak Cyer. Retrieved February 3, 2025, from <https://ajakcyer97.medium.com/big-o-time-complexity-graph-simplified-798f3b67877a>
- Haramoto, H., & Matsumoto, M. (2008, July 22). A p-adic algorithm for computing the inverse of integer matrices. *Journal of Computational and Applied Mathematics*, 225(1), 320-322. <https://doi.org/10.1016/j.cam.2008.07.044>

- Harris, C. R., Millman, K. J., Walt, S. J. v. d., Gommers, R., Virtanen, P., & Cornapeau, D. (2020, September 17). Array programming with NumPy. *Nature methods*, 585, 357-362.
<https://doi.org/10.1038/s41586-020-2649-2>
- Hugging Face. (2025). *Hugging Face Hub*. Hugging Face Hub. Retrieved February 11, 2025, from <https://huggingface.co/>
- Hunter, J. D. (2007, May-June). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, 9(3), 90-95. <https://doi.org/10.1109/MCSE.2007.55>
- IBM. (2024, November 27). *What is learning rate in machine learning?* IBM. Retrieved February 11, 2025, from <https://www.ibm.com/think/topics/learning-rate>
- Jameson, A., & Qu, K. (2010, December). Optimization Methods in Computational Fluid Dynamics. *Encyclopedia of Aerospace Engineering*. <http://dx.doi.org/10.1002/9780470686652.eae059>
- Kamburova, D., Marinov, R., & Zlateva, N. (2023, July 29). Saddle points in completely regular topological spaces. <https://doi.org/10.48550/arXiv.2306.14001>
- Kiem, R. (2020, February 6). *Understanding Local Minima in Neural-Network Training*. All About Circuits. Retrieved February 11, 2025, from <https://www.allaboutcircuits.com/technical-articles/understanding-local-minima-in-neural-network-training/>
- Kochenderfer, M. J., & Wheeler, T. A. (2019). *Algorithms for Optimization*. MIT Press.
- MathWorks. (n.d.). *What Is Global Optimization?* MathWorks. Retrieved February 11, 2025, from <https://www.mathworks.com/help/gads/what-is-global-optimization.html>
- Mehouachi, F. B., & Kasmi, C. (2021, April 8). Exact Stochastic Second Order Deep Learning. *Machine Learning*. <https://doi.org/10.48550/arXiv.2104.03804>
- Nar, K., & Sastry, S. S. (2018, May 22). Step Size Matters in Deep Learning. *NeurIPS, 2018*.
<https://doi.org/10.48550/arXiv.1805.08890>
- National Library of Medicine. (n.d.). 2. *Common Terms and Equations*. NLM - Finding and Using Health Statistics. Retrieved February 11, 2025, from <https://www.nlm.nih.gov/oet/ed/stats/02-900.html>

Nvidia. (n.d.). *Cloud & Data Center - Nvidia A100 Tensor Core GPU*. Nvidia. Retrieved February 11, 2025, from <https://www.nvidia.com/en-us/data-center/a100/>

OpenAI LLC. (n.d.). *ChatGPT o1 pro*. ChatGPT. Retrieved February 11, 2025, from <https://chatgpt.com/?model=o1-pro>

I have used O1 for assisting me in coding the program for this IA and writeup. Sample usage:

<https://chatgpt.com/share/67ab5a96-ae84-8003-9964-174e6b24e0a9>

O'Shea, K., & Nash, R. (2015, December 26). An Introduction to Convolutional Neural Networks. *Neural and Evolutionary Computing*. <https://doi.org/10.48550/arXiv.1511.08458>

Pascanu, R., Mikolov, T., & Bengio, Y. (2013, February 16). On the difficulty of training Recurrent Neural Networks. <https://doi.org/10.48550/arXiv.1211.5063>

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., & Gimelshein, N. (2019, December 3). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *In Advances in Neural Information Processing Systems*, 32, 8024-8035. <https://doi.org/10.48550/arXiv.1912.01703>

Programiz. (2024). *Asymptotic Analysis: Big-O Notation and More*. Programiz. Retrieved February 11, 2025, from <https://www.programiz.com/dsa/asymptotic-notations#:~:text=Big>

Python. (n.d.). *Python*. Python. Retrieved February 11, 2025, from <https://www.python.org/>

Reali, F., Priami, C., & Marchetti, L. (2017). Optimization Algorithms for Computational Systems Biology. *Sec. Optimization*, 3. <https://doi.org/10.3389/fams.2017.00006>

Scabini, L., Baets, B. D., & Bruno, I. M. (2024, September 28). Improving deep neural network random initialization through neuronal rewiring. *Neurocomputing*, 599. <https://doi.org/10.1016/j.neucom.2024.128130>

Schreiner, M. (2023, July 11). *GPT-4 architecture, datasets, costs and more leaked*. the decoder. Retrieved February 11, 2025, from <https://the-decoder.com/gpt-4-architecture-datasets-costs-and-more-leaked/#:~:text=The%20key%20points%3A>

- ScienceDirect. (n.d.). *Arithmetic Overflow*. ScienceDirect. Retrieved February 11, 2025, from <https://www.sciencedirect.com/topics/computer-science/arithmetic-overflow>
- ScienceDirect. (n.d.). *Confusion Matrix*. ScienceDirect. Retrieved February 11, 2025, from <https://www.sciencedirect.com/topics/engineering/confusion-matrix>
- ScienceDirect. (n.d.). *Relative Standard Deviation*. ScienceDirect. Retrieved February 11, 2025, from <https://www.sciencedirect.com/topics/computer-science/relative-standard-deviation>
- Sievers, B., & Thornton, M. A. (2024, February 21). Deep social neuroscience: the promise and peril of using artificial neural networks to study the social brain. *Social cognitive and affective neuroscience*, 19(1). <https://doi.org/10.1093/scan/nsae014>
- Surjanovic, S., Bingham, D., & Simon Fraser University. (2013). *Virtual Library of Simulation Experiments: Test Functions and Datasets*. Optimization Test Problems. Retrieved February 11, 2025, from <https://www.sfu.ca/~ssurjano/optimization.html>
- Tanious, R., & Manolov, R. (2022, September). Violin Plots as Visual Tools in the Meta-Analysis of Single-Case Experimental Designs. *Methodology European Journal of Research Methods for the Behavioral and Social Sciences*, 18(3), 221-238. <http://dx.doi.org/10.5964/meth.9209>
- Tibshirani, R. (2015, March). *Convex Optimization: Spring 2015*. Ryan Tibshirani. Retrieved February 11, 2025, from <https://www.stat.cmu.edu/~ryantibs/convexopt-S15/lectures/14-newton.pdf#:~:text=%E2%80%A2%20Computation%3A%20each%20Newton%20iteration,dimensional%20vectors>
- Tibshirani, R. (2015, March 2). *Convex Optimization: Spring 2015*. Ryan Tibshirani - CMU Website. Retrieved February 11, 2025, from <https://www.stat.cmu.edu/~ryantibs/convexopt-S15/scribes/14-newton-scribed.pdf#:~:text=For%20example%2C%20let%20us%20consider,n%203>
- Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., & Cornapeau, D. (2020, February 3). SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17, 261-272. <https://doi.org/10.1038/s41592-019-0686-2>

- Weisstein, E. W. (n.d.). *Abel's Impossibility Theorem*. MathWorld--A Wolfram Web Resource. Retrieved February 11, 2025, from <https://mathworld.wolfram.com/AbelsImpossibilityTheorem.html>
- Weisstein, E. W. (n.d.). *Method of Steepest Descent*. MathWorld--A Wolfram Web Resource. Retrieved February 11, 2025, from <https://mathworld.wolfram.com/MethodofSteepestDescent.html#:~:text=An%20algorithm%20%20for%20finding,the%20local%20downhill%20%2020>
- Xie, C., Koyejo, O., Gupta, I., & Lin, H. (2019, November 29). Local AdaAlter: Communication-Efficient Stochastic Gradient Descent with Adaptive Learning Rates. <https://doi.org/10.48550/arXiv.1911.09030>
- Xing, C., Arpit, D., Tsirigotis, C., & Bengio, Y. (2018, February 24). A Walk with SGD. <https://doi.org/10.48550/arXiv.1802.08770>
- Zhang, J., He, T., Sra, S., & Jadbabaie, A. (2019, May 28). Why gradient clipping accelerates training: A theoretical justification for adaptivity. <https://doi.org/10.48550/arXiv.1905.11881>
- Zhou, Y., Karimi, B., Yu, J., Xu, Z., & Li, P. (2020, December 6). Towards better generalization of adaptive gradient methods. *NeurIPS, 2020*, 810-821. <https://dl.acm.org/doi/10.5555/3495724.3495793>

Appendix A

This segment, previously introduced in Section 4.1 (Methodology Overview), contains the code responsible for generating and logging the empirical data into a CSV file. That CSV file is then processed by the plotting scripts in Appendix C.

```
!pip install numpy pandas matplotlib seaborn gitpython PyGithub
#####
# 1) Imports & Basic Setup
#####
import os
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import time
import random
from typing import List, Dict, Tuple, Callable, Optional
from datetime import datetime
import math

# For Git
import git
from github import Github

#####
# 1.1) Retrieve GitHub Token from Colab Secrets
#####
try:
    from google.colab import userdata
    GH_TOKEN = userdata.get("github") # 'github' must match your secret's name exactly
    if GH_TOKEN:
        import os
        os.environ["github"] = GH_TOKEN
        print("GitHub token set from colab secrets.")
    else:
        print("No GitHub token found in colab secrets.")
except ImportError:
    print("Not in Colab or no userdata available.")
except Exception as e:
    print(f"Error reading colab secret: {e}")
#####
#          GIT MANAGER          #
#####

class GitManager:
    """Handles GitHub operations to push results to a random-named branch."""
    def __init__(self, username: str, repo_name: str):
        self.username = username
        self.repo_name = repo_name
```

```

# after your above block sets os.environ["github"], read it here:
self.token = os.environ.get('github')
self.repo_url = f"https://x-access-token:{self.token}@github.com/{username}/{repo_name}.git"
self.branch_id = f"optimization_{random.randint(1000, 9999)}"
self.timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
self.repo = None

def setup_repo(self) -> str:
    import git

    try:
        root_dir = "optimization_results"
        os.makedirs(root_dir, exist_ok=True)

        with git.Git().custom_environment(GIT_SSL_NO_VERIFY='true'):
            try:
                self.repo = git.Repo(root_dir)
                origin = self.repo.remote('origin')
                origin.set_url(self.repo_url)
            except git.exc.InvalidGitRepositoryError:
                self.repo = git.Repo.init(root_dir)
                origin = self.repo.create_remote('origin', self.repo_url)

            config_writer = self.repo.config_writer()
            config_writer.set_value("http", "sslVerify", "false")
            config_writer.release()

            origin.fetch()
            origin.pull('main')
            new_branch = self.repo.create_head(self.branch_id, origin.refs.main)
            new_branch.checkout()

            experiment_dir = os.path.join(root_dir, f"experiment_{self.timestamp}")
            os.makedirs(experiment_dir, exist_ok=True)
            print(f"Successfully created/checked out branch: {self.branch_id}")
            return experiment_dir

    except Exception as e:
        print(f"Error setting up repository: {str(e)}")
        raise

def push_results(self):
    try:
        env = {
            'GIT_SSL_NO_VERIFY': 'true',
            'GIT_TERMINAL_PROMPT': '0',
            'GIT_USERNAME': 'x-access-token',
            'GIT_PASSWORD': self.token
        }

        with self.repo.git.custom_environment(**env):
            print(f"\nPushing results from experiment_{self.timestamp}...")
            self.repo.index.add('*')
            self.repo.index.commit(f"Results from experiment_{self.timestamp}")
            self.repo.remotes.origin.push(self.branch_id)

            g = Github(self.token)
            repo = g.get_repo(f"{self.username}/{self.repo_name}")
            pr = repo.create_pull(
                title=f"Results from experiment_{self.timestamp}",

```

```

        body=f"Automated results from {self.branch_id}",
        head=self.branch_id,
        base="main"
    )
    print(f"Created PR at: {pr.html_url}")

except Exception as e:
    print(f"Error pushing results: {str(e)}")
#####
#     HELPER METRICS     #
#####

def distance_to_minimum(x: np.ndarray, x_min: Optional[np.ndarray]) -> float:
    if x_min is None:
        return float('nan')
    return float(np.linalg.norm(x - x_min))

def step_size(x_curr: np.ndarray, x_prev: np.ndarray) -> float:
    return float(np.linalg.norm(x_curr - x_prev))

def grad_cosine_similarity(g_curr: np.ndarray, g_prev: np.ndarray) -> float:
    norm_curr = np.linalg.norm(g_curr)
    norm_prev = np.linalg.norm(g_prev)
    if norm_curr < 1e-15 or norm_prev < 1e-15:
        return float('nan')
    return float(np.dot(g_curr, g_prev)/(norm_curr*norm_prev))

def relative_improvement(current_val: float, prev_val: float) -> float:
    if abs(prev_val) < 1e-15:
        return 0.0
    return float((prev_val - current_val)/abs(prev_val))
#####
#     OPTIMIZERS     #
#####

class GradientDescent:
    """Gradient Descent optimizer with iteration-level logging."""
    def __init__(self, learning_rate=0.001, max_iter=1000, tol=1e-6):
        self.learning_rate = learning_rate
        self.max_iter = max_iter
        self.tol = tol

    def optimize(self, func: Callable, grad: Callable, x0: np.ndarray,
                 x_min: Optional[np.ndarray] = None):
        import pandas as pd
        x_curr = x0.copy()
        path = []
        fvals = []
        rows = []

        g_curr = grad(x_curr)
        f_curr = func(x_curr)
        dist_curr = distance_to_minimum(x_curr, x_min)

        step_sz = np.nan
        grad_cos_sim = np.nan
        rel_imp_f = np.nan
        rel_imp_d = np.nan
        rel_imp_g = np.nan # new

```

```

dim = len(x0)
for i in range(self.max_iter):
    # decide if we store x or not
    store_x = x_curr.tolist() if dim < 128 else None

    path.append(x_curr.copy())
    fvals.append(f_curr)

    rows.append({
        'method': 'GradientDescent',
        'dimension': dim,
        'iteration': i,
        'x': store_x,
        'f_val': f_curr,
        'dist_to_min': dist_curr,
        'step_size': step_sz,
        'grad_norm': float(np.linalg.norm(g_curr)),
        'grad_cosine_sim': grad_cos_sim,
        'rel_improvement_f': rel_imp_f,
        'rel_improvement_dist': rel_imp_d,
        'rel_improvement_grad': rel_imp_g # newly added column
    })

    if np.linalg.norm(g_curr) < self.tol:
        break

    # save old states for next iteration
    x_prev = x_curr.copy()
    f_prev = f_curr
    dist_prev = dist_curr
    g_prev = g_curr.copy()
    grad_norm_prev = float(np.linalg.norm(g_prev))

    # do update
    x_curr = x_curr - self.learning_rate * g_prev
    g_curr = grad(x_curr)
    f_curr = func(x_curr)
    dist_curr = distance_to_minimum(x_curr, x_min)

    step_sz = step_size(x_curr, x_prev)
    grad_cos_sim = grad_cosine_similarity(g_curr, g_prev)
    rel_imp_f = relative_improvement(f_curr, f_prev)
    rel_imp_d = relative_improvement(dist_curr, dist_prev)

    grad_norm_curr = float(np.linalg.norm(g_curr))
    rel_imp_g = relative_improvement(grad_norm_curr, grad_norm_prev)

success = (np.linalg.norm(g_curr) < self.tol)

iter_df = pd.DataFrame(rows)
return {
    'x_final': x_curr,
    'f_final': f_curr,
    'n_iter': i+1,
    'success': success,
    'path': path,
    'f_path': fvals,
    'iter_df': iter_df
}

```

```

class NewtonRaphson:
    """Newton-Raphson with approximate Hessian + iteration-level logging."""
    def __init__(self, max_iter=200, tol=1e-6):
        self.max_iter = max_iter
        self.tol = tol

    def optimize(self, func: Callable, grad: Callable, hess_approx: Callable,
                 x0: np.ndarray, x_min: Optional[np.ndarray] = None):
        import pandas as pd
        x_curr = x0.copy()
        path = []
        fvals = []
        rows = []

        g_curr = grad(x_curr)
        f_curr = func(x_curr)
        dist_curr = distance_to_minimum(x_curr, x_min)

        step_sz = np.nan
        grad_cos_sim = np.nan
        rel_imp_f = np.nan
        rel_imp_d = np.nan
        rel_imp_g = np.nan # new

        dim = len(x0)
        for i in range(self.max_iter):
            store_x = x_curr.tolist() if dim < 128 else None

            path.append(x_curr.copy())
            fvals.append(f_curr)

            rows.append({
                'method': 'NewtonRaphson',
                'dimension': dim,
                'iteration': i,
                'x': store_x,
                'f_val': f_curr,
                'dist_to_min': dist_curr,
                'step_size': step_sz,
                'grad_norm': float(np.linalg.norm(g_curr)),
                'grad_cosine_sim': grad_cos_sim,
                'rel_improvement_f': rel_imp_f,
                'rel_improvement_dist': rel_imp_d,
                'rel_improvement_grad': rel_imp_g # newly added
            })

            if np.linalg.norm(g_curr) < self.tol:
                break

            H = hess_approx(x_curr)
            try:
                p = np.linalg.solve(H, g_curr)
            except np.linalg.LinAlgError:
                break

            x_prev = x_curr.copy()
            f_prev = f_curr
            dist_prev = dist_curr
            g_prev = g_curr.copy()

```

```

        grad_norm_prev = float(np.linalg.norm(g_prev))

        x_curr = x_curr - p
        g_curr = grad(x_curr)
        f_curr = func(x_curr)
        dist_curr = distance_to_minimum(x_curr, x_min)

        step_sz = step_size(x_curr, x_prev)
        grad_cos_sim = grad_cosine_similarity(g_curr, g_prev)
        rel_imp_f = relative_improvement(f_curr, f_prev)
        rel_imp_d = relative_improvement(dist_curr, dist_prev)

        grad_norm_curr = float(np.linalg.norm(g_curr))
        rel_imp_g = relative_improvement(grad_norm_curr, grad_norm_prev)

    success = (np.linalg.norm(g_curr) < self.tol)

    iter_df = pd.DataFrame(rows)
    return {
        'x_final': x_curr,
        'f_final': f_curr,
        'n_iter': i+1,
        'success': success,
        'path': path,
        'f_path': fvals,
        'iter_df': iter_df
    }

# dims = [2, 8, 128, 512, 2048, 8192, 32768, 131072] # you can skip big dims if files are too huge
class OptimizerConfig:
    """Allows you to control which optimizers are enabled for each dimension."""
    def __init__(self):
        self.enabled_optimizers = {
            2: ['GradientDescent', 'NewtonRaphson'],
            8: ['GradientDescent', 'NewtonRaphson'],
            32: ['GradientDescent', 'NewtonRaphson'],
            128: ['GradientDescent', "NewtonRaphson"],
            512: ['GradientDescent', "NewtonRaphson"], # 20 seconds
            2048: ['GradientDescent', "NewtonRaphson"], # 206 seconds
            8192: ['GradientDescent', "NewtonRaphson"], # 299 seconds
            32768: ['GradientDescent'], # 2731-5000+ seconds
            131072: ['GradientDescent'], # too long
        }

    def get_enabled(self, dimension: int) -> List[str]:
        return self.enabled_optimizers.get(dimension, [])

#####
# EXTENDED TEST FUNCTIONS
#####

class TestFunctions:
    """Extended test functions that work with any dimension."""
    @staticmethod
    def get_global_minimum(func_name: str, dimension: int = 2) -> tuple:
        global_minima = {
            'ackley': (np.zeros(dimension), 0.0),
            'rastrigin': (np.zeros(dimension), 0.0),
            'rosenbrock': (np.ones(dimension), 0.0),
            'sphere': (np.zeros(dimension), 0.0),
            'schwefel': (420.9687 * np.ones(dimension), 0.0),

```

```

        'sum_squares': (np.zeros(dimension), 0.0),
        'michalewicz': (None, None),
    }
    return global_minima.get(func_name.lower(), (None, None))

@staticmethod
def ackley(x: np.ndarray) -> float:
    n = len(x)
    sum_sq = np.sum(x**2)
    sum_cos = np.sum(np.cos(2*np.pi*x))
    return (
        -20.0*np.exp(-0.2*np.sqrt(sum_sq/n))
        - np.exp(sum_cos/n)
        + 20.0 + np.e
    )

@staticmethod
def ackley_gradient(x: np.ndarray) -> np.ndarray:
    n = len(x)
    sum_sq = np.sum(x**2)
    sum_cos = np.sum(np.cos(2.0*np.pi*x))
    if sum_sq < 1e-15:
        term1 = 0.0*x
    else:
        term1 = (20.0*0.2/np.sqrt(n*sum_sq))*np.exp(-0.2*np.sqrt(sum_sq/n))*x
    term2 = (2.0*np.pi/n)*np.exp(sum_cos/n)*np.sin(2.0*np.pi*x)
    return term1 + term2

@staticmethod
def ackley_hessian(x: np.ndarray) -> np.ndarray:
    eps = 1e-8
    n = len(x)
    H = np.zeros((n,n))
    base_grad = TestFunctions.ackley_gradient(x)
    for j in range(n):
        x_jp = x.copy()
        x_jp[j] += eps
        grad_p = TestFunctions.ackley_gradient(x_jp)
        H[:, j] = (grad_p - base_grad)/eps
    return 0.5*(H + H.T)

@staticmethod
def rastrigin(x: np.ndarray) -> float:
    n = len(x)
    return 10.0*n + np.sum(x**2 - 10.0*np.cos(2.0*np.pi*x))

@staticmethod
def rastrigin_gradient(x: np.ndarray) -> np.ndarray:
    return 2.0*x + 20.0*np.pi*np.sin(2.0*np.pi*x)

@staticmethod
def rastrigin_hessian(x: np.ndarray) -> np.ndarray:
    n = len(x)
    diag_cos = np.cos(2.0*np.pi*x)
    return 2.0*np.eye(n) + 40.0*(np.pi**2)*np.diag(diag_cos)

@staticmethod
def schwefel(x: np.ndarray) -> float:
    n = len(x)
    return 418.9829*n - np.sum(x*np.sin(np.sqrt(np.abs(x))))

```



```

@staticmethod
def schwefel_gradient(x: np.ndarray) -> np.ndarray:
    eps = 1e-15
    sqrt_abs_x = np.sqrt(np.abs(x)+eps)
    term1 = np.sin(sqrt_abs_x)
    term2 = x*np.cos(sqrt_abs_x)/(2.0*sqrt_abs_x)
    return -(term1 + term2)

@staticmethod
def schwefel_hessian(x: np.ndarray) -> np.ndarray:
    eps = 1e-8
    n = len(x)
    H = np.zeros((n,n))
    grad_fn = TestFunctions.schwefel_gradient
    base_grad = grad_fn(x)
    for j in range(n):
        x_ij = x.copy()
        x_ij[j] += eps
        H[:, j] = (grad_fn(x_ij) - base_grad)/eps
    return 0.5*(H + H.T)

@staticmethod
def sum_squares(x: np.ndarray) -> float:
    i_idx = np.arange(1, len(x)+1)
    return np.sum(i_idx*(x**2))

@staticmethod
def sum_squares_gradient(x: np.ndarray) -> np.ndarray:
    i_idx = np.arange(1, len(x)+1)
    return 2.0*i_idx*x

@staticmethod
def sum_squares_hessian(x: np.ndarray) -> np.ndarray:
    n = len(x)
    i_idx = np.arange(1, n+1)
    return 2.0*np.diag(i_idx)

@staticmethod
def sphere(x: np.ndarray) -> float:
    return np.sum(x**2)

@staticmethod
def sphere_gradient(x: np.ndarray) -> np.ndarray:
    return 2.0*x

@staticmethod
def sphere_hessian(x: np.ndarray) -> np.ndarray:
    n = len(x)
    return 2.0*np.eye(n)

@staticmethod
def rosenbrock(x: np.ndarray) -> float:
    return np.sum(100.0*(x[1:]-x[:-1]**2)**2 + (1.0-x[:-1])**2)

@staticmethod
def rosenbrock_gradient(x: np.ndarray) -> np.ndarray:
    n = len(x)
    grad = np.zeros(n)
    grad[0] = -400.0*x[0]*(x[1]-x[0]**2) - 2.0*(1.0-x[0])

```

```

    grad[-1] = 200.0*(x[-1]-x[-2]**2)
    if n>2:
        grad[1:-1] = (
            200.0*(x[1:-1]-x[:-2]**2)
            -400.0*x[1:-1]*(x[2:]-x[1:-1]**2)
            -2.0*(1.0-x[1:-1])
        )
    return grad

@staticmethod
def rosenbrock_hessian(x: np.ndarray) -> np.ndarray:
    eps = 1e-8
    n = len(x)
    H = np.zeros((n,n))
    def grad_rb(xx):
        return TestFunctions.rosenbrock_gradient(xx)
    base_grad = grad_rb(x)
    for j in range(n):
        x_jp = x.copy()
        x_jp[j] += eps
        grad_p = grad_rb(x_jp)
        H[:, j] = (grad_p - base_grad)/eps
    return 0.5*(H + H.T)
#####
#   RUN EXPERIMENTS
#####

def pick_function_components(func_name: str):
    func_name = func_name.lower()
    if func_name == 'ackley':
        return (TestFunctions.ackley,
                TestFunctions.ackley_gradient,
                TestFunctions.ackley_hessian)
    elif func_name == 'rastrigin':
        return (TestFunctions.rastrigin,
                TestFunctions.rastrigin_gradient,
                TestFunctions.rastrigin_hessian)
    elif func_name == 'rosenbrock':
        return (TestFunctions.rosenbrock,
                TestFunctions.rosenbrock_gradient,
                TestFunctions.rosenbrock_hessian)
    elif func_name == 'sphere':
        return (TestFunctions.sphere,
                TestFunctions.sphere_gradient,
                TestFunctions.sphere_hessian)
    elif func_name == 'schwefel':
        return (TestFunctions.schwefel,
                TestFunctions.schwefel_gradient,
                TestFunctions.schwefel_hessian)
    elif func_name == 'sum_squares':
        return (TestFunctions.sum_squares,
                TestFunctions.sum_squares_gradient,
                TestFunctions.sum_squares_hessian)
    else:
        raise ValueError(f"Unsupported function: {func_name}")

def generate_starting_points(
    func_name: str,
    dimension: int,

```

```

n_points: int,
min_dist: float,
max_dist: float,
seed: int = 42
):
    np.random.seed(seed)
    starts = []

    x_min, _ = TestFunctions.get_global_minimum(func_name, dimension)
    if x_min is None:
        for _ in range(n_points):
            x0 = np.random.uniform(-2, 2, size=dimension)
            starts.append(x0)
        return starts

    for _ in range(n_points):
        direction = np.random.randn(dimension)
        direction /= (np.linalg.norm(direction) + 1e-12)
        dist = np.random.uniform(min_dist, max_dist)
        x0 = x_min + dist*direction
        starts.append(x0)
    return starts

def run_experiment_once(
    run_id: int,
    func_name: str,
    dimension: int,
    x0: np.ndarray,
    optimizer_name: str,
    all_optimizers: dict,
    save_dir: str = "results_csv"
):
    # pick function
    f, grad, hess_approx = pick_function_components(func_name)
    method_subdir_map = {
        "NewtonRaphson": "newton",
        "GradientDescent": "gradient"
    }
    method_subdir = method_subdir_map.get(optimizer_name, optimizer_name.lower())

    x_min, f_min = TestFunctions.get_global_minimum(func_name, dimension)
    # compute initial grad norm
    init_grad_norm = float(np.linalg.norm(grad(x0)))

    optimizer = all_optimizers[optimizer_name]

    start_time = time.time()
    if optimizer_name == "GradientDescent":
        result = optimizer.optimize(f, grad, x0, x_min=x_min)
    elif optimizer_name == "NewtonRaphson":
        result = optimizer.optimize(f, grad, hess_approx, x0, x_min=x_min)
    else:
        raise ValueError(f"Unknown optimizer '{optimizer_name}'")
    end_time = time.time()
    elapsed = end_time - start_time

    path = result.get('path', [])
    f_path = result.get('f_path', [])
    iter_df = result.get('iter_df')

```

```

if len(path) > 0:
    x_initial = path[0]
    f_initial = f_path[0]
    x_final = path[-1]
    f_final = f_path[-1]
else:
    x_initial = x0
    x_final = result.get('x_final', x0)
    f_initial = f(x_initial)
    f_final = f(x_final)

# After you compute x_initial, x_final, etc. do:
if dimension in [2, 8, 32]:
    x_init_str = x_initial.tolist()
    x_final_str = x_final.tolist()
else:
    # For dimension >= 128, store empty strings (or None)
    x_init_str = ""
    x_final_str = ""

# final grad norm
final_grad_norm = float(np.linalg.norm(grad(x_final)))

if x_min is not None:
    dist_init = float(np.linalg.norm(x_initial - x_min))
    dist_final = float(np.linalg.norm(x_final - x_min))
else:
    dist_init = float('nan')
    dist_final = float('nan')

# Overwrite success if final solution is worse than initial
success = result['success']
if f_final > f_initial + 1e-12:
    success = False
if dist_final > dist_init + 1e-12:
    success = False

# 4) relative improvements
rel_imp_f = relative_improvement(f_final, f_initial)
rel_imp_d = relative_improvement(dist_final, dist_init)
rel_imp_g = relative_improvement(final_grad_norm, init_grad_norm)

# Suppose we've already computed:
# success = ...
# f_initial, f_final
# dist_init, dist_final

# 1) Compute improvement fraction for function and distance
f_improvement_fraction = 0.0
dist_improvement_fraction = 0.0

if abs(f_initial) > 1e-15:
    f_improvement_fraction = 1.0 - (f_final / f_initial)
if abs(dist_init) > 1e-15:
    dist_improvement_fraction = 1.0 - (dist_final / dist_init)

# 2) Define thresholds in multiples of 15% up to 90% => [0.15, 0.30, 0.45, 0.60, 0.75, 0.90]
thresholds = [0.15, 0.30, 0.45, 0.60, 0.75, 0.90]

```

```

# 3) For each threshold, define success_f_<THRESH>, success_dist_<THRESH>
#     e.g. success_f_15 => (f_improvement_fraction >= 0.15)
success_f_15 = (f_improvement_fraction >= 0.15)
success_f_30 = (f_improvement_fraction >= 0.30)
success_f_45 = (f_improvement_fraction >= 0.45)
success_f_60 = (f_improvement_fraction >= 0.60)
success_f_75 = (f_improvement_fraction >= 0.75)
success_f_90 = (f_improvement_fraction >= 0.90)

success_dist_15 = (dist_improvement_fraction >= 0.15)
success_dist_30 = (dist_improvement_fraction >= 0.30)
success_dist_45 = (dist_improvement_fraction >= 0.45)
success_dist_60 = (dist_improvement_fraction >= 0.60)
success_dist_75 = (dist_improvement_fraction >= 0.75)
success_dist_90 = (dist_improvement_fraction >= 0.90)

# build local dir
dim_dir = os.path.join(save_dir, f"{dimension}D")
os.makedirs(dim_dir, exist_ok=True)
method_dir = os.path.join(dim_dir, method_subdir)
os.makedirs(method_dir, exist_ok=True)
func_dir = os.path.join(method_dir, func_name.lower())
os.makedirs(func_dir, exist_ok=True)

csv_name = f"run_{method_subdir}_{run_id}.csv"
csv_path = os.path.join(func_dir, csv_name)

if iter_df is not None:
    iter_df.to_csv(csv_path, index=False)

# Reorder summary columns =>
# experiment_num, timestamp, function, dimension, method, success,
# iterations, runtime, f_initial, f_final, initial_distance_to_minimum,
# final_distance_to_minimum, initial_grad_norm, final_grad_norm,
# x_initial, x_final
row = {
    # existing fields
    'experiment_num': run_id,
    'timestamp': datetime.now().strftime("%Y%m%d_%H%M%S"),
    'function': func_name,
    'dimension': dimension,
    'method': optimizer_name,
    'success': success, # gradient-based
    'iterations': result['n_iter'],
    'runtime': elapsed,
    'f_initial': f_initial,
    'f_final': f_final,
    'initial_distance_to_minimum': dist_init,
    'final_distance_to_minimum': dist_final,
    'initial_grad_norm': init_grad_norm,
    'final_grad_norm': final_grad_norm,
    'relative_improvement_f': rel_imp_f,
    'relative_improvement_dist': rel_imp_d,
    'relative_improvement_grad': rel_imp_g,

    # Now add new success flags
    'success_f_15': success_f_15,
    'success_f_30': success_f_30,
    'success_f_45': success_f_45,

```

```

        'success_f_60': success_f_60,
        'success_f_75': success_f_75,
        'success_f_90': success_f_90,

        'success_dist_15': success_dist_15,
        'success_dist_30': success_dist_30,
        'success_dist_45': success_dist_45,
        'success_dist_60': success_dist_60,
        'success_dist_75': success_dist_75,
        'success_dist_90': success_dist_90,

        # keep x_initial, x_final last
        'x_initial': x_init_str,
        'x_final': x_final_str
    }

    return row

def run_experiments(
    func_names: List[str],
    dimensions: List[int],
    n_experiments: int,
    all_optimizers: dict,
    optimizer_config,
    save_dir: str = "results_csv",
    min_dist: float = 1.0,
    max_dist: float = 5.0
):
    os.makedirs(save_dir, exist_ok=True)
    summary_path = os.path.join(save_dir, "summary.csv")

    # create summary with new col order:
    # experiment_num, timestamp, function, dimension, method, success,
    # iterations, runtime, f_initial, f_final, initial_distance_to_minimum,
    # final_distance_to_minimum, initial_grad_norm, final_grad_norm,
    # x_initial, x_final
    if not os.path.exists(summary_path):
        with open(summary_path, 'w') as f:
            f.write(
                "experiment_num,timestamp,function,dimension,method,success,"
                "iterations,runtime,"
                "f_initial,f_final,"
                "initial_distance_to_minimum,final_distance_to_minimum,"
                "initial_grad_norm,final_grad_norm,"
                "relative_improvement_f,relative_improvement_dist,relative_improvement_grad,"
                # now add your new success columns
                "success_f_15,success_f_30,success_f_45,success_f_60,success_f_75,success_f_90,"
                "success_dist_15,success_dist_30,success_dist_45,success_dist_60,success_dist_75,success_dist_90,"
                "x_initial,x_final\n"
            )

    for func_name in func_names:
        for dim in dimensions:
            enabled = optimizer_config.get_enabled(dim)
            starts = generate_starting_points(func_name, dim, n_experiments, min_dist, max_dist)

            for method in enabled:
                for run_id in range(1, n_experiments+1):

```

```

        x0 = starts[run_id-1]
        row = run_experiment_once(
            run_id=run_id,
            func_name=func_name,
            dimension=dim,
            x0=x0,
            optimizer_name=method,
            all_optimizers=all_optimizers,
            save_dir=save_dir
        )

    # 2) updated .write(...) line:
    with open(summary_path, 'a') as f:
        f.write(
            f"{row['experiment_num']},{row['timestamp']},{row['function']},{row['dimension']},{row['method']},{row['success']},{row['iterations']},{row['runtime']},{row['f_initial']},{row['f_final']},{row['initial_distance_to_minimum']},{row['final_distance_to_minimum']},{row['initial_grad_norm']},{row['final_grad_norm']},{row['relative_improvement_f']},{row['relative_improvement_dist']},{row['relative_improvement_grad']},{row['success_f_15']},{row['success_f_30']},{row['success_f_45']},{row['success_f_60']},{row['success_f_75']},{row['success_f_90']},{row['success_dist_15']},{row['success_dist_30']},{row['success_dist_45']},{row['success_dist_60']},{row['success_dist_75']},{row['success_dist_90']},{row['x_initial']},{row['x_final']}\n"
        )

    print(f"Experiments complete. Summary => {summary_path}")

def run_experiments_in_dir(
    base_dir: str,
    func_names: List[str],
    dimensions: List[int],
    n_experiments: int,
    all_optimizers: dict,
    optimizer_config,
    min_dist: float = 1.0,
    max_dist: float = 5.0
):
    save_dir = os.path.join(base_dir, "results_csv")
    os.makedirs(save_dir, exist_ok=True)

    summary_path = os.path.join(save_dir, "summary.csv")
    # Where you do:
    if not os.path.exists(summary_path):
        with open(summary_path, 'w') as f:
            f.write(
                "experiment_num,timestamp,function,dimension,method,success,"
                "iterations,runtime,"
                "f_initial,f_final,"
                "initial_distance_to_minimum,final_distance_to_minimum,"
                "initial_grad_norm,final_grad_norm,"
            )

```

```

        "relative_improvement_f,relative_improvement_dist,relative_improvement_grad,"
        # now add your new success columns
        "success_f_15,success_f_30,success_f_45,success_f_60,success_f_75,success_f_90,"
        "success_dist_15,success_dist_30,success_dist_45,success_dist_60,success_dist_75,success_dist_90,"
        "x_initial,x_final\n"
    )

for func_name in func_names:
    for dim in dimensions:
        enabled_methods = optimizer_config.get_enabled(dim)
        starts = generate_starting_points(func_name, dim, n_experiments, min_dist, max_dist)
        for method in enabled_methods:
            for run_id in range(1, n_experiments+1):
                x0 = starts[run_id-1]
                row = run_experiment_once(
                    run_id=run_id,
                    func_name=func_name,
                    dimension=dim,
                    x0=x0,
                    optimizer_name=method,
                    all_optimizers=all_optimizers,
                    save_dir=save_dir
                )
                with open(summary_path, 'a') as f:
                    f.write(
                        f"{row['experiment_num']},{row['timestamp']},{row['function']},"
                        f"{row['dimension']},{row['method']},{row['success']},"
                        f"{row['iterations']},{row['runtime']},"
                        f"{row['f_initial']},{row['f_final']},"
                        f"{row['initial_distance_to_minimum']},{row['final_distance_to_minimum']},"
                        f"{row['initial_grad_norm']},{row['final_grad_norm']},"
                        f"{row['relative_improvement_f']},{row['relative_improvement_dist']},{row['relative_improvement_grad']}"
                    ),"

                    # New success columns for function
                    f"{row['success_f_15']},{row['success_f_30']},{row['success_f_45']},"
                    f"{row['success_f_60']},{row['success_f_75']},{row['success_f_90']},"
                    # New success columns for distance

                    f"{row['success_dist_15']},{row['success_dist_30']},{row['success_dist_45']},"
                    f"{row['success_dist_60']},{row['success_dist_75']},{row['success_dist_90']},"
                    # Keep x_initial, x_final last
                    f"\n{row['x_initial']}\n,\n{row['x_final']}\n\n"

                )
            print(f"\nAll experiments done. Summary => {summary_path}")
#####
#
#         MAIN FUNCTION
#####

def main():
    from google.colab import userdata
    username = "Ice-Citron"
    repo_name = "AAH-IA"
    git_manager = GitManager(username, repo_name)

    local_repo_dir = git_manager.setup_repo()
    print(f"Local repo directory: {local_repo_dir}")

```



```

# example
func_names = ['ackley', 'rastrigin', 'rosenbrock', 'schwefel', 'sum_squares']
dims = [2, 8, 32, 128, 512, 2048, 8192, 32768, 131072] # you can skip big dims if files are too
huge
n_experiments = 15

gd = GradientDescent(learning_rate=0.001, max_iter=2000, tol=1e-4)
nr = NewtonRaphson(max_iter=200, tol=1e-4)

all_optimizers = {
    'GradientDescent': gd,
    'NewtonRaphson': nr
}

optimizer_cfg = OptimizerConfig()

run_experiments_in_dir(
    base_dir=local_repo_dir,
    func_names=func_names,
    dimensions=dims,
    n_experiments=n_experiments,
    all_optimizers=all_optimizers,
    optimizer_config=optimizer_cfg,
    min_dist=2.0,
    max_dist=5.0
)

try:
    git_manager.push_results()
except Exception as e:
    print(f"Failed to push: {e}")

print("Done with main().")

if __name__ == "__main__":
    main()

import wandb
wandb.finish()

```

Appendix B

(Section 4.5: This script below implements a simple CNN on the MNIST dataset, employing a custom Gradient Descent optimizer. It leverages PyTorch for model definition, data loading, and backward passes, and logs training progress via Weights & Biases.)

```
!pip install torch wandb huggingface_hub
#####
# 0) Install & Setup Hugging Face Hub
#####
!pip install huggingface_hub

from huggingface_hub import HfApi, HfFolder

# Attempt to load HF_TOKEN from colab secrets
try:
    from google.colab import userdata
    HF_TOKEN = userdata.get("HF_TOKEN")
    if HF_TOKEN:
        import os
        os.environ["HF_TOKEN"] = HF_TOKEN
        print("Set HF_TOKEN from colab secrets.")
    else:
        print("No HF_TOKEN found in colab secrets.")
except:
    pass

hf_token = os.environ.get("HF_TOKEN", None)
if hf_token:
    HfFolder.save_token(hf_token)
    print("HF token set in huggingface_hub")
else:
    print("No HF token. Upload to HF will fail unless you manually login.")

#####
# 1) Imports & Basic Setup
#####
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
import math

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

#####
# 2) Larger MLP (~9.6k params)
#####
class LargeMLP(nn.Module):
    """
    We'll downsample MNIST to 16x16 => input dim=256
    Then 2 hidden layers of 32 each => total ~9.6k parameters.
    """
```

```

def __init__(self):
    super().__init__()
    self.fc1 = nn.Linear(256, 32)
    self.fc2 = nn.Linear(32, 32)
    self.fc3 = nn.Linear(32, 10)

def forward(self, x):
    x = x.view(x.size(0), -1)
    x = torch.relu(self.fc1(x))
    x = torch.relu(self.fc2(x))
    x = self.fc3(x)
    return x

def compute_global_grad_norm(model):
    total_sq = 0.0
    for p in model.parameters():
        if p.grad is not None:
            g = p.grad.data
            total_sq += g.norm(2).item()**2
    return math.sqrt(total_sq)

#####
# 3) Simple Optimizers
#####
class NaiveGradientDescent(torch.optim.Optimizer):
    def __init__(self, params, lr=0.01):
        defaults = dict(lr=lr)
        super().__init__(params, defaults)

    def step(self, closure=None):
        if closure is None:
            raise RuntimeError("NaiveGradientDescent needs a closure returning the loss (tensor).")
        loss = closure()
        loss.backward()
        for group in self.param_groups:
            lr = group['lr']
            for p in group['params']:
                if p.grad is None:
                    continue
                p.data.sub_(lr * p.grad)
        return loss

#####
# 4) DataLoaders
#####
def get_mnist_loaders(batch_size=64):
    transform = transforms.Compose([
        transforms.Resize((16,16)),
        transforms.ToTensor(),
        transforms.Normalize((0.5,), (0.5,))
    ])
    train_ds = datasets.MNIST(root=".", train=True, download=True, transform=transform)
    test_ds = datasets.MNIST(root=".", train=False, download=True, transform=transform)
    train_loader = torch.utils.data.DataLoader(train_ds, batch_size=batch_size, shuffle=True)
    test_loader = torch.utils.data.DataLoader(test_ds, batch_size=batch_size, shuffle=False)
    return train_loader, test_loader

#####
# 5) Training
#####

```

```

def train_model(
    optimizer_name="gd",
    learning_rate=1e-2,
    epochs=2,
    batch_size=64,
    wandb_project=""
):
    import os
    import wandb

    model = LargeMLP().to(device)
    train_loader, test_loader = get_mnist_loaders(batch_size)
    criterion = nn.CrossEntropyLoss()

    # Possibly wandb
    do_wandb = (wandb_project != "" and os.environ.get("wandb"))
    if do_wandb:
        wandb.login(key=os.environ.get("wandb"))
        wandb.init(project=wandb_project, config={
            "optimizer_name": optimizer_name,
            "learning_rate": learning_rate,
            "epochs": epochs,
            "batch_size": batch_size
        })

    # pick optimizer
    if optimizer_name.lower() == "gd":
        opt = NaiveGradientDescent(model.parameters(), lr=learning_rate)
    else:
        opt = torch.optim.SGD(model.parameters(), lr=learning_rate)

    global_step = 0
    initial_loss_val = None

    for epoch in range(epochs):
        model.train()
        total_loss = 0.0
        for batch_idx, (data, targets) in enumerate(train_loader):
            data, targets = data.to(device), targets.to(device)

            old_params = [p.detach().clone() for p in model.parameters()]

            def closure():
                opt.zero_grad()
                out = model(data)
                loss_t = criterion(out, targets)
                return loss_t

            loss_tensor = opt.step(closure=closure)
            loss_val = loss_tensor.item()
            total_loss += loss_val

            if initial_loss_val is None:
                initial_loss_val = loss_val

            grad_norm = compute_global_grad_norm(model)

            sum_sq = 0.0
            for p, old_p in zip(model.parameters(), old_params):
                delta = (p.detach() - old_p).view(-1)

```

```

        sum_sq += delta.norm(2).item()*2
    step_size = math.sqrt(sum_sq)

    rel_improvement = 0.0
    if abs(initial_loss_val) > 1e-12:
        rel_improvement = (initial_loss_val - loss_val)/abs(initial_loss_val)

    global_step += 1
    print(f"Epoch[{epoch}/{epochs}] Step[{global_step}] "
          f"Loss={loss_val:.4f} GradNorm={grad_norm:.4f} "
          f"StepSize={step_size:.4f} RelImp={rel_improvement*100:.2f}%")

    if do_wandb:
        wandb.log({
            "train_loss": loss_val,
            "grad_norm": grad_norm,
            "step_size": step_size,
            "relative_improvement": rel_improvement
        }, step=global_step)

# Evaluate
model.eval()
correct = 0
test_loss = 0.0
with torch.no_grad():
    for data, targets in test_loader:
        data, targets = data.to(device), targets.to(device)
        out = model(data)
        l = criterion(out, targets)
        test_loss += l.item()*data.size(0)
        _, pred = out.max(1)
        correct += pred.eq(targets).sum().item()
test_loss /= len(test_loader.dataset)
accuracy = 100.*correct/len(test_loader.dataset)
epoch_loss = total_loss/len(train_loader)
print(f"Epoch {epoch+1}/{epochs} => "
      f"TrainLoss={epoch_loss:.4f} TestLoss={test_loss:.4f} Acc={accuracy:.2f}%")

if do_wandb:
    wandb.log({
        "epoch": epoch,
        "train_loss_epoch": epoch_loss,
        "test_loss": test_loss,
        "test_accuracy": accuracy
    }, step=global_step)

if do_wandb:
    wandb.finish()

return model
#####
# 6) Main - plus push to huggingface
#####
def main():
    # Train
    model = train_model(
        optimizer_name="gd",
        learning_rate=0.01,
        epochs=2,
        batch_size=16,

```

```

        wandb_project="AAH-IA__gradient-descent__"
    )

    # Save model to local file
    torch.save(model.state_dict(), "model.pth")
    print("Saved model to model.pth")

    # Now push to HF
    from huggingface_hub import HfApi
    hf_token = os.environ.get("HF_TOKEN", None)
    if not hf_token:
        print("No HF_TOKEN => can't push to HF. Please set HF_TOKEN first.")
        return

    repo_id = "shng2025/AAH-IA__gradient-descent__" # your HF repo
    api = HfApi()
    try:
        # you can also specify path_in_repo="some_dir/model.pth" if desired
        api.upload_file(
            path_or_fileobj="model.pth",
            path_in_repo="model.pth",
            repo_id=repo_id,
            token=hf_token
        )
        print(f"Uploaded 'model.pth' to {repo_id}")
    except Exception as e:
        print(f"HF upload failed: {e}")

if __name__ == "__main__":
    main()
import wandb

wandb.finish()

```

(Section 4.5: This script below implements a simple CNN on the MNIST dataset, employing a custom Newton–Raphson optimizer. It leverages PyTorch for model definition, data loading, and backward passes, and logs training progress via Weights & Biases.)

```

!pip install torch wandb huggingface_hub
#####
# 1) Imports & Basic Setup
#####
import os
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
import math

# Attempt to load secrets from colab
try:
    from google.colab import userdata
    # 1) HF_TOKEN
    hf_tok = userdata.get("HF_TOKEN")
    if hf_tok:
        os.environ["HF_TOKEN"] = hf_tok

```

```

        print("Set HF_TOKEN from colab secrets.")
# 2) W&B
wandb_key = userdata.get("wandb")
if wandb_key:
    os.environ["wandb"] = wandb_key
    print("Set wandb from colab secrets.")
except:
    pass

hf_token = os.environ.get("HF_TOKEN", None)
wandb_key = os.environ.get("wandb", None)
print("hf_token:", bool(hf_token))
print("wandb_key:", bool(wandb_key))

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

#####
# 2) Larger MLP (~9.6k params)
#####
class LargeMLP(nn.Module):
    """
    We'll downsample MNIST to 16x16 => input dim=256
    Then 2 hidden layers of 32 each => total ~9.6k parameters.

    Calculation:
    fc1: (256 * 32) + 32 = 8192 + 32 = 8224
    fc2: (32 * 32) + 32 = 1024 + 32 = 1056
    fc3: (32 * 10) + 10 = 320 + 10 = 330
    total => 8224 + 1056 + 330 = 9610
    """
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(256, 32)
        self.fc2 = nn.Linear(32, 32)
        self.fc3 = nn.Linear(32, 10)

    def forward(self, x):
        # x shape: [B,1,16,16]
        x = x.view(x.size(0), -1) # => [B,256]
        x = torch.relu(self.fc1(x)) # => [B,32]
        x = torch.relu(self.fc2(x)) # => [B,32]
        x = self.fc3(x) # => [B,10]
        return x

#####
# 2.1) Compute Global Gradient Norm
#####
def compute_global_grad_norm(model):
    """
    Sums the squared L2-norm of each param.grad, then sqrt.
    This gives the 'global' gradient norm across all parameters.
    """
    total_sq = 0.0
    for p in model.parameters():
        if p.grad is not None:
            g = p.grad.data
            total_sq += g.norm(2).item()**2
    return math.sqrt(total_sq)

```

```
#####
# 3) Naive Newton Optimizer
#####
class NaiveNewtonOptimizer(torch.optim.Optimizer):
    """
    Dense Hessian => O(N^3). Must call step(closure).
    Great for demonstration with small networks only.
    """
    def __init__(self, params, lr=1.0, tol=1e-6):
        defaults = dict(lr=lr, tol=tol)
        super().__init__(params, defaults)

    def step(self, closure=None):
        if closure is None:
            raise RuntimeError("NaiveNewtonOptimizer needs a closure returning the loss (tensor).")

        loss = closure()
        loss.backward(create_graph=True)

        for group in self.param_groups:
            lr = group['lr']
            tol = group['tol']

            for p in group['params']:
                if p.grad is None:
                    continue
                grad = p.grad.view(-1)
                if grad.norm() < tol:
                    continue

                n = grad.numel()
                # Build Hessian
                H = []
                for i in range(n):
                    g_i = grad[i]
                    p.grad = None
                    g_i.backward(retain_graph=True)
                    H_i = p.grad.view(-1).clone()
                    H.append(H_i)
                    p.grad = None
                H = torch.stack(H, dim=1) # => [n, n]

                # Solve H dx = grad
                try:
                    dx, _ = torch.solve(grad.unsqueeze(1), H)
                    dx = dx.squeeze(1)
                except RuntimeError:
                    dx = grad # fallback

                # Safely do in-place update using .data
                p.data.sub_((lr * dx).view(p.shape))

        return loss

#####
# 3) Naive Gradient Descent Optimizer
#####
class NaiveGradientDescent(torch.optim.Optimizer):
    """
    A pure "vanilla" Gradient Descent:

```



```

p <- p - lr * grad(p)

Must call step(closure) where the closure:
- zeroes grads
- does forward pass
- returns the loss (tensor)
Then we do normal .backward() for the first derivative only.
"""
def __init__(self, params, lr=0.01):
    defaults = dict(lr=lr)
    super().__init__(params, defaults)

def step(self, closure=None):
    if closure is None:
        raise RuntimeError("NaiveGradientDescent needs a closure returning the loss (tensor).")

    # 1) Recompute the forward pass
    loss = closure()
    # 2) Normal backward => first derivatives
    loss.backward()

    # 3) Update
    for group in self.param_groups:
        lr = group['lr']
        for p in group['params']:
            if p.grad is None:
                continue
            p.data.sub_(lr * p.grad)

    return loss

#####
# 4) DataLoaders with downsampling to 16x16
#####
def get_mnist_loaders(batch_size=64):
    transform = transforms.Compose([
        transforms.Resize((16,16)),
        transforms.ToTensor(),
        transforms.Normalize((0.5,), (0.5,))
    ])
    train_ds = datasets.MNIST(root=".", train=True, download=True, transform=transform)
    test_ds = datasets.MNIST(root=".", train=False, download=True, transform=transform)
    train_loader = torch.utils.data.DataLoader(train_ds, batch_size=batch_size, shuffle=True)
    test_loader = torch.utils.data.DataLoader(test_ds, batch_size=batch_size, shuffle=False)
    return train_loader, test_loader

#####
# 5) Train function
#####
def train_model(
    optimizer_name: str = "",
    learning_rate: float = 1e-2,
    epochs: int = 2,
    batch_size: int = 64,
    wandb_project: str = ""
):
    """
    We'll:
    - Create ~9.6k param MLP (16x16 input => 2 hidden layers of 32)

```

```

- Use "gd", "newton", or "sgd"
- Possibly log each step's loss to wandb
- Also log:
    1) global grad norm
    2) step size
    3) relative improvement
"""
model = LargeMLP().to(device)
train_loader, test_loader = get_mnist_loaders(batch_size)
criterion = nn.CrossEntropyLoss()

# W&B
wandb_key = os.environ.get("wandb", None)
do_wandb = (wandb_project != "") and (wandb_key is not None)
if do_wandb:
    import wandb
    wandb.login(key=wandb_key)
    wandb.init(project=wandb_project, config={
        "optimizer_name": optimizer_name,
        "learning_rate": learning_rate,
        "epochs": epochs,
        "batch_size": batch_size
    })

# pick optimizer
if optimizer_name.lower() == "gd":
    opt = NaiveGradientDescent(model.parameters(), lr=learning_rate)
elif optimizer_name.lower() == "newton":
    opt = NaiveNewtonOptimizer(model.parameters(), lr=learning_rate)
elif optimizer_name.lower() == "sgd":
    opt = torch.optim.SGD(model.parameters(), lr=learning_rate)
else:
    raise ValueError(f"Unknown optimizer {optimizer_name}")

global_step = 0
initial_loss_val = None # track initial for relative improvement

for epoch in range(epochs):
    model.train()
    total_loss = 0.0

    for batch_idx, (data, targets) in enumerate(train_loader):
        data, targets = data.to(device), targets.to(device)

        # 1) Save old parameters for step size calculation
        old_params = [p.detach().clone() for p in model.parameters()]

        def closure():
            opt.zero_grad()
            outputs = model(data)
            loss_t = criterion(outputs, targets)
            return loss_t

        # 2) Step
        loss_tensor = opt.step(closure=closure)
        loss_val = loss_tensor.item()
        total_loss += loss_val

        # 3) If first time, store initial loss
        if initial_loss_val is None:

```

```

        initial_loss_val = loss_val

# 4) Compute global grad norm
grad_norm = compute_global_grad_norm(model)

# 5) Compute step size
sum_sq = 0.0
for p, old_p in zip(model.parameters(), old_params):
    delta = (p.detach() - old_p).view(-1)
    sum_sq += delta.norm(2).item()*2
step_size = math.sqrt(sum_sq)

# 6) Compute relative improvement from the initial iteration
# (if initial_loss_val was 2.3 and now it's 1.2, improvement = ~47.8%)
rel_improvement = 0.0
if abs(initial_loss_val) > 1e-12: # avoid /0
    rel_improvement = (initial_loss_val - loss_val) / abs(initial_loss_val)

global_step += 1
print(
    f"Epoch[{epoch}/{epochs}] Step[{global_step}] "
    f"Loss={loss_val:.4f} GradNorm={grad_norm:.4f} "
    f"StepSize={step_size:.4f} RelImp={rel_improvement*100:.2f}%"
)

if do_wandb:
    import wandb
    wandb.log({
        "train_loss": loss_val,
        "grad_norm": grad_norm,
        "step_size": step_size,
        "relative_improvement": rel_improvement
    }, step=global_step)

# Evaluate
model.eval()
correct = 0
test_loss = 0.0
with torch.no_grad():
    for data, targets in test_loader:
        data, targets = data.to(device), targets.to(device)
        out = model(data)
        l = criterion(out, targets)
        test_loss += l.item() * data.size(0)
        _, pred = out.max(1)
        correct += pred.eq(targets).sum().item()
test_loss /= len(test_loader.dataset)
accuracy = 100.*correct / len(test_loader.dataset)
epoch_loss = total_loss / len(train_loader)

print(f"Epoch {epoch+1}/{epochs} => TrainLoss={epoch_loss:.4f} TestLoss={test_loss:.4f}
Acc={accuracy:.2f}%")

if do_wandb:
    import wandb
    wandb.log({
        "epoch": epoch,
        "train_loss_epoch": epoch_loss,
        "test_loss": test_loss,
        "test_accuracy": accuracy
    })

```

```

        }, step=global_step)

    if do_wandb:
        import wandb
        wandb.finish()

    return model
#####
# 6) Main
#####
def main(
    optimizer_name="newton",
    lr=1.0,
    epochs=2,
    batch_size=16,
    wandb_project="AAH-IA__newton-rhapson__",
):
    hf_token = os.environ.get("HF_TOKEN", None)
    wandb_key = os.environ.get("wandb", None)

    print("HF token:", bool(hf_token))
    print("W&B key:", bool(wandb_key))
    print(f"Using {optimizer_name} with ~9.6k param model. LR={lr}, epochs={epochs},
batch_size={batch_size}")

    model = train_model(
        optimizer_name=optimizer_name,
        learning_rate=lr,
        epochs=epochs,
        batch_size=batch_size,
        wandb_project=wandb_project
    )
    print("Done with main(). Returning model.")
    return model

# Example if we want to run immediately:
if __name__ == "__main__":
    model = main(
        optimizer_name="newton",
        lr=0.0001, # 0.0001 will be used for newton
        epochs=2,
        batch_size=16,
        wandb_project="AAH-IA__newton-rhapson__" # or AAH-IA__gradient-descent__
    )
import wandb

wandb.finish()

```

Appendix C

In this section, I present the Matplotlib (Hunter, 2007) code I wrote for generating each figure and table, while leveraging NumPy (Harris et al., 2020) and SciPy (Virtanen et al., 2020) for statistical tasks—such as computing the Mann–Whitney test.

Figure 2: 3D Plot with Contour Plot Demonstrating Function’s Landscape Features (Matplotlib)

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

def three_hump_camel(x, y):
    """Three-hump camel function"""
    return 2*x**2 - 1.05*x**4 + (x**6)/6 + x*y + y**2

# Create data points
x = np.linspace(-2, 2, 200)
y = np.linspace(-1, 1, 200)
X, Y = np.meshgrid(x, y)
Z = three_hump_camel(X, Y)

# Correct critical points from scipy
critical_points = {
    'Local Minimum 1': (-1.74755234583029, 0.873776172915144),
    'Saddle Point 1': (-1.07054229182366, 0.535271145911830),
    'Global Minimum': (0.0, 0.0),
    'Saddle Point 2': (1.07054229182366, -0.535271145911830),
    'Local Minimum 2': (1.74755234583029, -0.873776172915144)
}

# Create figure with two subplots
fig = plt.figure(figsize=(20, 8))

# 3D Surface Plot
ax1 = fig.add_subplot(121, projection='3d')

# Plot surface with lower zorder
surf = ax1.plot_surface(X, Y, Z, cmap='viridis',
                        linewidth=0.5,
                        antialiased=True,
                        alpha=1.0,
                        zorder=1) # Lower zorder for surface

# Plot critical points in 3D with higher zorder
for label, (x_cp, y_cp) in critical_points.items():
    z_cp = three_hump_camel(x_cp, y_cp)
    if 'Saddle' in label:
        ax1.plot(x_cp, y_cp, z_cp, color='red', marker='*', markersize=12,
                 linestyle='None', label='Saddle Point', zorder=10) # Higher zorder for points
    elif 'Global' in label:
```

```

        ax1.plot(x_cp, y_cp, z_cp, color='green', marker='^', markersize=12,
                 linestyle='None', label='Global Minimum', zorder=10)
    else:
        ax1.plot(x_cp, y_cp, z_cp, color='blue', marker='o', markersize=12,
                 linestyle='None', label='Local Minimum', zorder=10)

ax1.set_xlabel('x')
ax1.set_ylabel('y')
ax1.set_zlabel('f(x,y)')
ax1.set_title('Three-Hump Camel Function')

# Set custom ticks every 0.5
ax1.set_xticks(np.arange(-2, 2.5, 0.5))
ax1.set_yticks(np.arange(-1, 1.5, 0.5))
ax1.set_zticks(np.arange(0, 6, 1))

# Fix duplicate labels
handles, labels = ax1.get_legend_handles_labels()
by_label = dict(zip(labels, handles))
ax1.legend(by_label.values(), by_label.keys(), loc='upper right')

# Set the correct view angle
ax1.view_init(elev=30, azim=120)

# Adjust the plot box aspect ratio
ax1.set_box_aspect([2, 1, 1])

# 2D Contour Plot
ax2 = fig.add_subplot(122)
levels = np.linspace(np.min(Z), np.max(Z), 20)
contours = ax2.contour(X, Y, Z, levels=levels, cmap='viridis')
contourf = ax2.contourf(X, Y, Z, levels=levels, cmap='viridis', alpha=0.7)
fig.colorbar(contourf, ax=ax2)

# Plot critical points in 2D with high zorder
for label, (x_cp, y_cp) in critical_points.items():
    if 'Saddle' in label:
        ax2.plot(x_cp, y_cp, 'r*', markersize=15, label='Saddle Point', zorder=10)
    elif 'Global' in label:
        ax2.plot(x_cp, y_cp, 'g^', markersize=15, label='Global Minimum', zorder=10)
    else:
        ax2.plot(x_cp, y_cp, 'bo', markersize=10, label='Local Minimum', zorder=10)

ax2.set_xlabel('x')
ax2.set_ylabel('y')
ax2.set_title('Contour Plot of Three-Hump Camel Function')
ax2.grid(True, linestyle='--', alpha=0.3)
ax2.set_aspect('equal')

# Set custom ticks every 0.5
ax2.set_xticks(np.arange(-2, 2.5, 0.5))
ax2.set_yticks(np.arange(-1, 1.5, 0.5))

```

```

# Fix duplicate labels
handles, labels = ax2.get_legend_handles_labels()
by_label = dict(zip(labels, handles))
ax2.legend(by_label.values(), by_label.keys())

plt.tight_layout()
plt.show()

```

Figure 3 - 3D Surface Plot of Sphere Function with Red Gradient Arrows (Matplotlib)

```

import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from mpl_toolkits.mplot3d.art3d import Line3DCollection

def create_3d_gradient_plot():
    # Create figure
    fig = plt.figure(figsize=(12, 10))
    ax = fig.add_subplot(111, projection='3d')

    # Generate data for surface plot
    x = np.linspace(-2, 2, 100)
    y = np.linspace(-2, 2, 100)
    X, Y = np.meshgrid(x, y)
    Z = X**2 + Y**2

    # Create surface plot
    surf = ax.plot_surface(X, Y, Z, cmap='viridis', alpha=0.8)
    fig.colorbar(surf, label='f(x1, x2)')

    # Generate grid of points for gradient vectors
    x_points = np.linspace(-2, 2, 10)
    y_points = np.linspace(-2, 2, 10)
    X_grad, Y_grad = np.meshgrid(x_points, y_points)
    Z_grad = X_grad**2 + Y_grad**2

    # Calculate gradients at each point
    U = 2 * X_grad # ∂f/∂x1 = 2x1
    V = 2 * Y_grad # ∂f/∂x2 = 2x2
    W = np.ones_like(U) # Vertical component for visualization

    # Normalize gradient vectors
    norms = np.sqrt(U**2 + V**2 + W**2)
    mask = norms > 1e-10
    U_norm = np.zeros_like(U)
    V_norm = np.zeros_like(V)
    W_norm = np.zeros_like(W)
    U_norm[mask] = U[mask] / norms[mask]
    V_norm[mask] = V[mask] / norms[mask]
    W_norm[mask] = W[mask] / norms[mask]

    # Scale factor for arrows

```

```

scale = 0.2

# Plot gradient vectors as arrows with increased thickness
for i in range(len(x_points)):
    for j in range(len(y_points)):
        ax.quiver(X_grad[i,j], Y_grad[i,j], Z_grad[i,j],
                  U_norm[i,j]*scale, V_norm[i,j]*scale, W_norm[i,j]*scale,
                  color='red', alpha=0.7, linewidth=1, # Increased linewidth
                  arrow_length_ratio=0.4) # Increased arrow head size

# Customize the plot
ax.set_xlabel('x1', fontsize=12)
ax.set_ylabel('x2', fontsize=12)
ax.set_zlabel('f(x1, x2)', fontsize=12)
ax.set_title('3D Surface Plot of f(x1, x2) = x1^2 + x2^2\nwith Gradient Vectors',
             pad=20, fontsize=14)

# Set the viewing angle
ax.view_init(elev=30, azim=45)

# Set axis limits
ax.set_xlim([-2, 2])
ax.set_ylim([-2, 2])
ax.set_zlim([0, 8])

plt.tight_layout()
plt.show()

# Create the plot
create_3d_gradient_plot()

```

Figure 4: Comparison of Optimizers' Trajectories on Test Function Beyond 1st Iteration (Matplotlib)

```

import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from scipy.optimize import minimize
import time

def shifted_sphere(x):
    if isinstance(x, np.ndarray) and len(x.shape) == 1:
        return (x[0]-1)**2 + (x[1]+2)**2
    else:
        return (x[0]-1)**2 + (x[1]+2)**2

def shifted_sphere_grid(x, y):
    return (x-1)**2 + (y+2)**2

def shifted_sphere_gradient(x, y):
    dx = 2*(x-1)
    dy = 2*(y+2)
    return np.array([dx, dy])

```



```

def plot_optimization_comparison(starting_point, learning_rate=0.1, num_steps=20):
    fig = plt.figure(figsize=(15, 6))

    # Generate surface data
    x = np.linspace(-2, 4, 200)
    y = np.linspace(-5, 1, 200)
    X, Y = np.meshgrid(x, y)
    Z = shifted_sphere_grid(X, Y)

    # Gradient Descent
    start_time_gd = time.time()
    path_gd = [starting_point]
    current_point = starting_point.copy()

    for _ in range(num_steps):
        grad = shifted_sphere_gradient(current_point[0], current_point[1])
        current_point = current_point - learning_rate * grad
        path_gd.append(current_point.copy())

    gd_time = time.time() - start_time_gd
    path_gd = np.array(path_gd)
    path_gd_z = [shifted_sphere_grid(x, y) for x, y in path_gd]
    actual_gd_steps = len(path_gd) - 1

    # Newton-Raphson using scipy
    path_nr = []
    def callback(xk):
        path_nr.append(xk)
        return False

    start_time_nr = time.time()
    result = minimize(shifted_sphere, starting_point, method='Newton-CG',
                      jac=lambda x: shifted_sphere_gradient(x[0], x[1]),
                      callback=callback,
                      options={'maxiter': num_steps})

    nr_time = time.time() - start_time_nr

    # Just add the starting point
    path_nr = [starting_point] + path_nr
    path_nr = np.array(path_nr)
    path_nr_z = [shifted_sphere_grid(x, y) for x, y in path_nr]

    # Gradient Descent 3D plot
    ax1 = fig.add_subplot(121, projection='3d')
    surf = ax1.plot_surface(X, Y, Z, cmap='viridis', alpha=1.0, zorder=1)
    fig.colorbar(surf, ax=ax1, shrink=0.5, aspect=5)

    ax1.plot(path_gd[:, 0], path_gd[:, 1], path_gd_z, 'r.-', linewidth=2, markersize=8,
             label='Optimization Path', zorder=10)

    ax1.plot([path_gd[0, 0]], [path_gd[0, 1]], [path_gd_z[0]], 'go', markersize=8,
             label='Start', zorder=3)

```

```

ax1.plot([path_gd[-1, 0]], [path_gd[-1, 1]], [path_gd_z[-1]], 'ro', markersize=8,
        label='End', zorder=3)

ax1.view_init(elev=45, azim=275)
ax1.set_xlim([-2, 4])
ax1.set_ylim([-5, 1])
ax1.set_xlabel('x')
ax1.set_ylabel('y')
ax1.set_zlabel('f(x, y)')
ax1.set_title(f'Gradient Descent Path\n(Steps: {actual_gd_steps}, Time: {gd_time:.4f}s)')
ax1.legend()

# Newton-Raphson 3D plot
ax2 = fig.add_subplot(122, projection='3d')
surf = ax2.plot_surface(X, Y, Z, cmap='viridis', alpha=1.0, zorder=1)
fig.colorbar(surf, ax=ax2, shrink=0.5, aspect=5)

# Just plot the single step with a line
ax2.plot(path_nr[:, 0], path_nr[:, 1], path_nr_z, 'y.-', linewidth=4, markersize=12,
        label='Optimization Path', zorder=10)

# Plot start and end points
ax2.plot([path_nr[0, 0]], [path_nr[0, 1]], [path_nr_z[0]], 'go', markersize=12,
        label='Start', zorder=3)
ax2.plot([path_nr[-1, 0]], [path_nr[-1, 1]], [path_nr_z[-1]], 'yo', markersize=12,
        label='End', zorder=3)

ax2.view_init(elev=45, azim=275)
ax2.set_xlim([-2, 4])
ax2.set_ylim([-5, 1])
ax2.set_xlabel('x')
ax2.set_ylabel('y')
ax2.set_zlabel('f(x, y)')
ax2.set_title(f'Newton-Raphson Path\n(Steps: 1, Time: {nr_time:.4f}s)')
ax2.legend()

plt.tight_layout()
plt.show()

# Example usage
starting_point = np.array([2.0, 2.0])
plot_optimization_comparison(starting_point, learning_rate=0.1, num_steps=50)

```

Table 1: Benchmark Functions: Equations and 3D Visualisations (Matplotlib)

```

import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import os

# Create plots directory if it doesn't exist
os.makedirs('optimization_plots', exist_ok=True)

```

```

# Define the test functions
def ackley(x, y):
    return -20 * np.exp(-0.2 * np.sqrt(0.5 * (x**2 + y**2))) - \
        np.exp(0.5 * (np.cos(2*np.pi*x) + np.cos(2*np.pi*y))) + 20 + np.e

def rastrigin(x, y):
    return 20 + (x**2 - 10*np.cos(2*np.pi*x)) + (y**2 - 10*np.cos(2*np.pi*y))

def schwefel(x, y):
    return 418.9829*2 - x*np.sin(np.sqrt(np.abs(x))) - y*np.sin(np.sqrt(np.abs(y)))

def sphere(x, y):
    return x**2 + y**2

def rosenbrock(x, y):
    return 100*(y - x**2)**2 + (x - 1)**2

def sum_squares(x, y):
    return x**2 + y**2

# Function to generate Ackley local minima
def generate_ackley_minima(x_min, x_max, y_min, y_max, step=1.0):
    """
    Generate approximate local minima for the Ackley function within the specified range.
    Assumes minima are near integer grid points.
    """
    minima = []
    xi_values = np.arange(np.ceil(x_min), np.floor(x_max) + 1, step)
    yi_values = np.arange(np.ceil(y_min), np.floor(y_max) + 1, step)
    for xi in xi_values:
        for yi in yi_values:
            minima.append((xi, yi))
    return minima

# Function to generate Rastrigin local minima
def generate_rastrigin_minima(x_min, x_max, y_min, y_max):
    """
    Generate local minima for the Rastrigin function within the specified range.
    Assumes minima are at integer grid points.
    """
    minima = []
    xi_values = np.arange(np.ceil(x_min), np.floor(x_max) + 1, 1)
    yi_values = np.arange(np.ceil(y_min), np.floor(y_max) + 1, 1)
    for xi in xi_values:
        for yi in yi_values:
            minima.append((xi, yi))
    return minima

# Define plotting ranges for each function
function_configs = {
    'Ackley': (ackley, (-2.5, 2.5)),
    'Rastrigin': (rastrigin, (-2.5, 2.5)),
    'Schwefel': (schwefel, (-500, 500)), # Adjusted range for Schwefel

```

```

'Sphere': (sphere, (-5, 5)),
'Rosenbrock': (rosenbrock, (-2, 2)), # Narrower range for better visualization
'Sum_Squares': (sum_squares, (-1.25, 1.25))
}

# Generate critical points
critical_points = {
    'Ackley': {
        'global_min': [(0, 0)],
        'local_min': generate_ackley_minima(*function_configs['Ackley'][1],
*function_configs['Ackley'][1], step=1.0),
        'local_max': [], # No local maxima for Ackley
        'saddle': [(0.25, -0.25), (-0.25, 0.25), (0.25, 0.25), (-0.25, -0.25)] # Retained as
per user request
    },
    'Rastrigin': {
        'global_min': [(0, 0)],
        'local_min': generate_rastrigin_minima(*function_configs['Rastrigin'][1],
*function_configs['Rastrigin'][1]),
        'local_max': [], # No local maxima for Rastrigin
        'saddle': [] # Typically, no saddle points
    },
    'Schwefel': {
        'global_min': [(420.9687, 420.9687)],
        'local_min': [(-420.9687, 420.9687), (420.9687, -420.9687)],
        'local_max': [], # No local maxima for Schwefel
        'saddle': [(0, 0)]
    },
    'Sphere': {
        'global_min': [(0, 0)],
        'local_min': [],
        'local_max': [],
        'saddle': []
    },
    'Rosenbrock': {
        'global_min': [(1, 1)],
        'local_min': [],
        'local_max': [],
        'saddle': [(0, 0)]
    },
    'Sum_Squares': {
        'global_min': [(0, 0)],
        'local_min': [],
        'local_max': [],
        'saddle': []
    }
}

# Remove the global minimum from local_min if it's already listed
for func_name, points in critical_points.items():
    global_mins = set(points['global_min'])
    local_mins = set(points['local_min'])
    points['local_min'] = list(local_mins - global_mins)

```

```

# Function to create both 3D and contour plots
def create_plots(func, func_name, x_range=(-5, 5), y_range=(-5, 5), points=200):
    x = np.linspace(x_range[0], x_range[1], points)
    y = np.linspace(y_range[0], y_range[1], points)
    X, Y = np.meshgrid(x, y)
    Z = func(X, Y)

    # Get critical points for this function
    points_dict = critical_points[func_name]

    # Debugging: Print number of critical points
    print(f"{func_name} - Number of Global Minima: {len(points_dict['global_min'])}")
    print(f"{func_name} - Number of Local Minima: {len(points_dict['local_min'])}")
    print(f"{func_name} - Number of Local Maxima: {len(points_dict['local_max'])}")
    print(f"{func_name} - Number of Saddle Points: {len(points_dict['saddle'])}")

    # 3D Surface Plot
    fig = plt.figure(figsize=(12, 9))
    ax = fig.add_subplot(111, projection='3d')
    surf = ax.plot_surface(X, Y, Z, cmap='viridis', alpha=0.6, edgecolor='none')

    # Define scatter order to ensure global minima are plotted on top
    scatter_order = ['local_max', 'saddle', 'local_min', 'global_min']

    # Define colors and markers
    scatter_styles = {
        'global_min': {'color': 'red', 'marker': '*', 's': 200, 'label': 'Global Minimum'},
        'local_max': {'color': 'purple', 'marker': '^', 's': 150, 'label': 'Local Maximum'},
        'local_min': {'color': 'blue', 'marker': 'o', 's': 100, 'label': 'Local Minimum'},
        'saddle': {'color': 'green', 'marker': 'D', 's': 150, 'label': 'Saddle Point'}
    }

    # To avoid duplicate labels in the legend
    plotted_labels = set()

    for point_type in scatter_order:
        points_list = points_dict.get(point_type, [])
        if not points_list:
            continue # Skip if no points for this type
        # Extract all x, y, z coordinates for the current point type
        px, py = zip(*points_list) if points_list else ([], [])
        pz = [func(px_i, py_i) for px_i, py_i in points_list]
        style = scatter_styles.get(point_type, {})
        label = style.get('label') if style.get('label') not in plotted_labels else ""
        if px and py:
            ax.scatter(px, py, pz,
                      color=style.get('color', 'black'),
                      marker=style.get('marker', 'o'),
                      s=style.get('s', 100),
                      label=label,
                      edgecolors='black', # Enhance visibility
                      linewidths=0.5,

```

```

        depthshade=False) # Ensure markers are on top
    if label:
        plotted_labels.add(label)

ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('f(x, y)')
plt.title(f'{func_name} Function - 3D Surface')
fig.colorbar(surf, shrink=0.5, aspect=10, label='f(x, y)')
ax.legend()
plt.savefig(f'optimization_plots/{func_name.lower()}_3d.png', bbox_inches='tight', dpi=300)
plt.close()

# Contour Plot
fig, ax = plt.subplots(figsize=(12, 9))
contour = ax.contour(X, Y, Z, levels=50, cmap='viridis')
ax.clabel(contour, inline=True, fontsize=8)

# Reset plotted_labels for contour plot
plotted_labels = set()

for point_type in scatter_order:
    points_list = points_dict.get(point_type, [])
    if not points_list:
        continue # Skip if no points for this type
    # Extract all x, y coordinates for the current point type
    px, py = zip(*points_list) if points_list else ([], [])
    style = scatter_styles.get(point_type, {})
    label = style.get('label') if style.get('label') not in plotted_labels else ""
    if px and py:
        ax.scatter(px, py,
                   color=style.get('color', 'black'),
                   marker=style.get('marker', 'o'),
                   s=style.get('s', 100),
                   label=label,
                   edgecolors='black', # Enhance visibility
                   linewidths=0.5,
                   zorder=5) # Ensure markers are on top
        if label:
            plotted_labels.add(label)

plt.colorbar(contour, ax=ax, label='f(x, y)')
ax.set_xlabel('x')
ax.set_ylabel('y')
plt.title(f'{func_name} Function - Contour')
ax.legend()
plt.savefig(f'optimization_plots/{func_name.lower()}_contour.png', bbox_inches='tight',
dpi=300)
plt.close()

# Generate all plots
for func_name, (func, range_val) in function_configs.items():
    print(f"Generating plots for {func_name} function...")

```

```

create_plots(func, func_name, x_range=range_val, y_range=range_val)

print("All plots have been generated in the 'optimization_plots' folder!")

```

Figure 5: Ackley Runtime on Linear Scale (Matplotlib)

Figure 6: Ackley Runtime on Log Scale (Matplotlib)

Figure 7: Ackley Iterations vs. Dimension (Matplotlib)

```

def plot_single_run_all_three(csv_path="summary [32768].csv"):
    """
    Reads a single-run CSV with columns like:
        - function, dimension, method, iterations, runtime, ...
    Creates one figure per function, each with a row of 3 subplots side-by-side:
        (1) runtime (linear), (2) runtime (log-scale), (3) iterations (linear).
    """

    if not os.path.exists(csv_path):
        print(f"ERROR: CSV not found at: {csv_path}")
        return

    print(f>Loading CSV from: {csv_path}")
    df = pd.read_csv(csv_path)
    print("Loaded data with columns:", df.columns.tolist())
    display(df.head())

    # Filter down to dimension ≤ 32768 if desired
    df_filtered = df[df["dimension"] <= 32768].copy()

    # Unique functions
    all_funcs = df_filtered["function"].unique().tolist()
    print("Functions found:", all_funcs)

    for func in all_funcs:
        sub = df_filtered[df_filtered["function"] == func].copy()

        # Create a single figure with 3 subplots side-by-side
        fig, axes = plt.subplots(nrows=1, ncols=3, figsize=(21, 5))

        #####
        # (A) RUNTIME (LINEAR SCALE)
        #####
        sns.barplot(
            data=sub,
            x="dimension",
            y="runtime",
            hue="method",
            errorbar=None, # replaces ci=None
            ax=axes[0]
        )
        axes[0].set_title(f"Runtime (Linear) - {func}")
        axes[0].set_xlabel("Dimension")
        axes[0].set_ylabel("Runtime (seconds, linear)")

```

```

axes[0].grid(True, alpha=0.3)
axes[0].legend(title="Method")

#####
# (B) RUNTIME (LOG SCALE)
#####
sns.barplot(
    data=sub,
    x="dimension",
    y="runtime",
    hue="method",
    errorbar=None,
    ax=axes[1]
)
axes[1].set_yscale("log")
axes[1].set_title(f"Runtime (Log) - {func}")
axes[1].set_xlabel("Dimension")
axes[1].set_ylabel("Runtime (log-scale)")
axes[1].grid(True, alpha=0.3)
# Move or remove legend here if you wish (e.g., axes[1].legend().remove())

#####
# (C) ITERATIONS (LINEAR SCALE)
#####
sns.barplot(
    data=sub,
    x="dimension",
    y="iterations",
    hue="method",
    errorbar=None,
    ax=axes[2]
)
axes[2].set_title(f"Iterations - {func}")
axes[2].set_xlabel("Dimension")
axes[2].set_ylabel("Iterations")
axes[2].grid(True, alpha=0.3)
# Possibly remove legend here or keep it:
axes[2].legend(title="Method")

# Optional: a common figure title
fig.suptitle(f"[Single Run for {func}] - 3 Charts Side-by-Side", fontsize=13)

plt.tight_layout()
plt.show()

print("Done plotting single-run CSV with side-by-side subplots for each function.")

plot_single_run_all_three()

```

Figure 8: Mean Runtime vs. Dimension (Linear Scale) (Matplotlib)

Figure 9: Mean Runtime vs. Dimension (Log–Log Scale) (Matplotlib)

Figure 10: Mean Iteration Count vs. Dimension (with Standard Deviations) (Matplotlib)


```

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

def plot_average_across_functions(csv_path="summary.csv"):
    """
    1) Loads 'summary.csv' with columns: dimension, method, runtime, iterations, function...
    2) Groups by (dimension, method) => takes the *average across all functions*
       for runtime & iteration (plus std).
    3) Produces ONE figure with 3 subplots side by side:
        (A) runtime (linear dimension)
        (B) runtime (log2 dimension, log y-axis)
        (C) iteration bar chart
    => each method gets its own line/bar in each subplot.
    """

    # 1) Load data
    if not os.path.exists(csv_path):
        print(f"[ERROR] CSV not found => {csv_path}")
        return
    df = pd.read_csv(csv_path)
    print("[INFO] Loaded columns:", df.columns.tolist())

    # 2) Group by (dimension, method) => average across *all* functions
    grouped = (
        df.groupby(["dimension", "method"], dropna=False)
        .agg(
            runtime_mean = ("runtime", "mean"),
            runtime_std = ("runtime", "std"),
            iterations_mean = ("iterations", "mean"),
            iterations_std = ("iterations", "std")
        )
        .reset_index()
    )

    # Just to confirm shape & columns
    print("[INFO] After grouping across all functions => shape:", grouped.shape)
    print(grouped.head())

    # 3) Extract a list of methods & sorted dimension
    methods = sorted(grouped["method"].unique())
    dimensions = sorted(grouped["dimension"].unique())

    # 4) We'll create a single figure with 3 subplots side by side
    fig, axes = plt.subplots(nrows=1, ncols=3, figsize=(22, 6))
    fig.suptitle("Averaged Across All 5 Functions", fontsize=14, y=1.02)

    # -----
    # (A) RUNTIME (LINEAR dimension)
    # -----
    ax0 = axes[0]
    ax0.set_title("Runtime vs. Dimension (Linear)")

```

```

for m in methods:
    sub_m = grouped[grouped["method"]==m]
    xvals = sub_m["dimension"].values
    yvals = sub_m["runtime_mean"].values
    errs = sub_m["runtime_std"].values

    ax0.errorbar(
        xvals, yvals, yerr=errs,
        fmt='-o', capsize=4, label=m
    )
ax0.set_xlabel("Dimension (linear)")
ax0.set_ylabel("Runtime (sec)")
ax0.grid(alpha=0.3)
ax0.legend(title="Method")

# -----
# (B) RUNTIME (LOG2 dimension and LOG y-axis)
# -----
ax1 = axes[1]
ax1.set_title("Runtime vs. Dimension (Log-Log)")
for m in methods:
    sub_m = grouped[grouped["method"]==m]
    xvals = sub_m["dimension"].values
    yvals = sub_m["runtime_mean"].values
    errs = sub_m["runtime_std"].values

    ax1.errorbar(
        xvals, yvals, yerr=errs,
        fmt='-o', capsize=4, label=m
    )
ax1.set_xscale("log", base=2)
ax1.set_yscale("log") # Changed: Added log scale for y-axis
ax1.set_xlabel("Dimension (log2)")
ax1.set_ylabel("Runtime (log sec)") # Updated label to reflect log scale
ax1.grid(alpha=0.3)
# ax1.legend(title="Method")

# -----
# (C) ITERATIONS BAR CHART
# -----
ax2 = axes[2]
ax2.set_title("Iterations (Mean  $\pm$  Std)")

# We pivot so each method is a column => dimension on rows => we'll bar-plot side-by-side
piv = grouped.pivot(index="dimension", columns="method",
                     values=["iterations_mean", "iterations_std"])
dims_sorted = list(piv.index)
xvals = np.arange(len(dims_sorted))
bar_width = 0.6 / len(methods)

for i_m, m in enumerate(methods):
    yvals = piv[("iterations_mean", m)].values
    errs = piv[("iterations_std", m)].values

```

```

        x_off = xvals + (i_m - 0.5*len(methods))*bar_width + 0.5*bar_width

    ax2.bar(
        x_off, yvals,
        yerr=errs,
        width=bar_width,
        capsize=4,
        label=m,
        edgecolor="white"
    )

    ax2.set_xticks(xvals)
    ax2.set_xticklabels(dims_sorted)
    ax2.set_xlabel("Dimension")
    ax2.set_ylabel("Iterations (Mean)")
    ax2.grid(axis="y", alpha=0.3)
    ax2.legend(title="Method")

plt.tight_layout()
plt.show()

print("[INFO] Done => 3 total plots (avg across all 5 functions).")

# Example usage:
if __name__ == "__main__":
    plot_average_across_functions("local_results/experiment_20250117_051102/results_csv/summary.csv")

```

Figure 11: Violin Plots of Iterations per Dimension (All Functions) (Matplotlib)

```

import os
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

def violin_plots_by_function(
    csv_path="local_results/experiment_20250117_051102/results_csv/summary.csv",
    metric="runtime"
):
    """
    1) Loads your summary CSV.
    2) Creates one subplot per function in a wide figure.
        - X-axis: dimension (as categorical)
        - Y-axis: chosen metric (e.g. 'runtime' or 'iterations')
        - hue: 'method' to color code the optimizer
    3) This allows side-by-side comparison of each function's distributions
        across different dimensions & methods.

    NOTES:
    - If you only have 1 run per (dimension, method, function),
      violin plots won't show a real "distribution"
    """

```

```

    (they may look like lines or small lumps).
- Typically best if multiple runs exist to see the distribution shape.
- Each subplot shares the same y-scale for easier cross-comparison.
"""

if not os.path.exists(csv_path):
    print(f"CSV not found at: {csv_path}")
    return

# 1) Load data
df = pd.read_csv(csv_path)
print("Columns:", df.columns.tolist())
print("Sample rows:")
display(df.head())

# Convert dimension to string => treat as categorical on x-axis
df["dimension_str"] = df["dimension"].astype(str)

# Sort dimension so it appears in ascending numeric order (optional)
unique_dims = sorted(df["dimension"].unique())
df["dimension_str"] = pd.Categorical(
    df["dimension_str"],
    categories=[str(d) for d in unique_dims],
    ordered=True
)

# 2) We'll produce one subplot per unique function
funcs = sorted(df["function"].unique())
n_funcs = len(funcs)

fig, axes = plt.subplots(
    1, n_funcs,
    figsize=(6 * n_funcs, 6),
    sharey=True # so they share the same Y-axis scale
)

# If there's only 1 function, axes is not a list => make it a list
if n_funcs == 1:
    axes = [axes]

for i, func_name in enumerate(funcs):
    ax = axes[i]
    subdf = df[df["function"] == func_name].copy()

    sns.violinplot(
        data=subdf,
        x="dimension_str",
        y=metric,
        hue="method",
        density_norm="width", # <-- updated param (replaces scale="width")
        cut=0,
        split=False,
        ax=ax
    )

```

```

    )

    ax.set_title(f"{func_name} - {metric}")
    ax.set_xlabel("Dimension")
    ax.set_ylabel(metric.capitalize())

    # Hide legend on all but last subplot
    if i < n_funcs - 1:
        ax.get_legend().remove()
    else:
        ax.legend(title="Method", bbox_to_anchor=(1.05, 1), loc='upper left')

plt.tight_layout()
plt.show()

print("Done. Created violin plots for each function across dimensions & methods.")

# Example usage:
violin_plots_by_function(
    csv_path="local_results/experiment_20250117_051102/results_csv/summary.csv",
    metric="runtime"
)

violin_plots_by_function(
    csv_path="local_results/experiment_20250117_051102/results_csv/summary.csv",
    metric="iterations"
)

```

Figure 12: Newton–Raphson Iterations (Mean $\pm$ Std) Across Dimensions (Matplotlib)

Figure 13: Newton–Raphson Runtime vs. Dimension (Matplotlib)

Figure 14: Newton–Raphson Runtime Variability (Relative Std) vs. Dimension (Matplotlib)

```

import os
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker

def
plot_both_optimizers_with_runtime_bar(csv_path="local_results/experiment_20250117_051102/result
s_csv/summary.csv"):
    """
    1) Loads the summary CSV.
    2) For each optimizer ("NewtonRaphson", "GradientDescent"):
        - Filter rows
        - Group by (function, dimension) => get mean,std for runtime & iterations
        - Create:
            (A) line chart: runtime (mean  $\pm$  std) vs dimension (log2 X-axis)
            (B) bar chart: iterations (mean  $\pm$  std)
            (C) bar chart: runtime (mean  $\pm$  std)
    """

```

```

        (D) bar chart: runtime_std
        (E) bar chart: runtime_relative_std
    """

    if not os.path.exists(csv_path):
        print(f"[ERROR] CSV does not exist: {csv_path}")
        return

    df = pd.read_csv(csv_path)
    print("Loaded columns:", df.columns.tolist())

    def process_one_method(method_name):
        """Filter + group + aggregator => returns aggregated DataFrame (mean ± std)."""
        df_sub = df[df["method"]==method_name].copy()
        if df_sub.empty:
            print(f"[WARNING] No rows for method={method_name}")
            return None

        # group => mean,std
        grp = df_sub.groupby(["function","dimension"])[["runtime","iterations"]]
        ag = grp.agg(["mean","std"]).reset_index()

        # flatten columns
        # it becomes: [("function",""), ("dimension",""), ("runtime","mean"), ("runtime","std"),
        ("iterations","mean"), ("iterations","std")]
        ag.columns = [
            "function","dimension",
            "runtime_mean","runtime_std",
            "iterations_mean","iterations_std"
        ]

        # relative std
        ag["runtime_rel_std"] = np.where(
            ag["runtime_mean"].abs()>1e-15,
            (ag["runtime_std"]/ag["runtime_mean"])*100.0,
            np.nan
        )

        # rounding
        for col in
["runtime_mean","runtime_std","iterations_mean","iterations_std","runtime_rel_std"]:
            ag[col] = ag[col].round(2)

        ag = ag.sort_values(["function","dimension"])
        return ag

    # We'll do for both methods
    for method in ["NewtonRaphson","GradientDescent"]:
        ag = process_one_method(method)
        if ag is None:
            continue

    funcs = ag["function"].unique()

```

```

# (A) line chart: runtime mean±std vs dimension (log2)
plt.figure(figsize=(8,5))
ax = plt.gca()
for func in funcs:
    sub = ag[ag["function"]==func]
    xvals = sub["dimension"].values
    yvals = sub["runtime_mean"].values
    err    = sub["runtime_std"].values

    ax.errorbar(
        xvals, yvals, yerr=err,
        fmt='-o', capsize=4, label=func
    )
ax.set_title(f"{method}: Runtime (Mean±Std) vs Dimension (log2 X-axis)")
ax.set_xlabel("Dimension (log2 scale)")
ax.set_ylabel("Runtime (sec)")
ax.set_xscale("log", base=2)
ax.xaxis.set_major_locator(ticker.LogLocator(base=2))
ax.grid(alpha=0.3)
ax.legend(title="Function", bbox_to_anchor=(1.05,1), loc="upper left")
plt.tight_layout()
plt.show()

# (B) bar chart: iterations (mean±std)
plt.figure(figsize=(9,5))
ax2 = plt.gca()
ax2.set_title(f"{method}: Iterations (Mean±Std)")
sns.barplot(
    data=ag,
    x="dimension",
    y="iterations_mean",
    hue="function",
    errorbar=None,
    ax=ax2
)
# manually add error bars
bars = ax2.patches
i=0
for idx,row in ag.iterrows():
    xcenter = bars[i].get_x() + bars[i].get_width()/2
    height = bars[i].get_height()
    err_iter = row["iterations_std"]
    ax2.errorbar(
        xcenter, height, yerr=err_iter,
        ecolor='black', capsize=3
    )
    i+=1
ax2.set_xlabel("Dimension")
ax2.set_ylabel("Iterations (Mean)")
ax2.grid(alpha=0.3)
ax2.legend(title="Function")
plt.tight_layout()

```

```

plt.show()

# (C) bar chart: runtime_mean ± std
# We'll do the same approach as above, but y=runtime_mean
fig, ax3 = plt.subplots(figsize=(9,5))
ax3.set_title(f"{method}: Runtime (Mean±Std)")
sns.barplot(
    data=ag,
    x="dimension",
    y="runtime_mean",
    hue="function",
    errorbar=None,
    ax=ax3
)
bars = ax3.patches
i = 0
for idx,row in ag.iterrows():
    xcenter = bars[i].get_x() + bars[i].get_width()/2
    height = bars[i].get_height()
    err_rt = row["runtime_std"]
    ax3.errorbar(
        xcenter, height, yerr=err_rt,
        ecolor='black', capsize=3
    )
    i+=1
ax3.set_xlabel("Dimension")
ax3.set_ylabel("Runtime (Mean, sec)")
ax3.grid(alpha=0.3)
ax3.legend(title="Function")
plt.tight_layout()
plt.show()

# (D) bar chart: runtime_std
# (E) bar chart: runtime_rel_std
fig, (ax4, ax5) = plt.subplots(nrows=1, ncols=2, figsize=(14,5))

ax4.set_title(f"{method}: Runtime Std (sec)")
sns.barplot(
    data=ag,
    x="dimension",
    y="runtime_std",
    hue="function",
    errorbar=None,
    ax=ax4
)
ax4.set_xlabel("Dimension")
ax4.set_ylabel("Runtime Std (sec)")
ax4.grid(alpha=0.3)
ax4.legend(title="Function")

ax5.set_title(f"{method}: Runtime Rel Std (%)")
sns.barplot(
    data=ag,

```



```

        x="dimension",
        y="runtime_rel_std",
        hue="function",
        errorbar=None,
        ax=ax5
    )
    ax5.set_xlabel("Dimension")
    ax5.set_ylabel("Runtime Rel Std (%)")
    ax5.grid(alpha=0.3)
    ax5.legend(title="Function")

    plt.tight_layout()
    plt.show()

    print(f"\n[Done: {method}]")

    print("\nFinished both methods (Newton+Gradient) with log2 dimension on the line chart + bar
charts.")

plot_both_optimizers_with_runtime_bar()

```

Figure 15: Confusion-Matrix of Mean Runtime and Relative Std for Gradient Descent (Matplotlib)

Figure 16: Confusion-matrix of Mean Runtime and Relative Std for Newton–Raphson (Matplotlib)

```

def confusion_matrix_mean_vs_std_pct(
    csv_path="local_results/experiment_20250117_051102/results_csv/summary.csv",
    metric="runtime"
):
    """
    1) Loads 'summary.csv' (or any aggregated CSV with columns [function, dimension, method,
    <metric>...]).
    2) Groups by (function, dimension, method) => compute:
        - mean(metric)
        - std(metric)
        => then compute pct_std = 100*(std/mean).
    3) For each method, produce a single figure with 2 side-by-side heatmaps:
        - Heatmap of mean(metric) (left) with forced [0,1100] if metric=='runtime'
        - Heatmap of % std (right) with forced [0,290] if metric=='runtime'
        - Rows => function, Cols => dimension.
        - White grid lines separate cells, 2 decimal places by default.
    4) The second heatmap hides function labels on the y-axis.
    """

    import os
    import numpy as np
    import pandas as pd
    import seaborn as sns
    import matplotlib.pyplot as plt

    if not os.path.exists(csv_path):
        print(f"[ERROR] CSV not found => {csv_path}")
        return

```

```

df = pd.read_csv(csv_path)
print(f"Loaded summary CSV with columns: {df.columns.tolist()}")
if df.empty:
    print("[WARNING] DataFrame is empty; nothing to plot.")
    return

# ~~~ 1) Group => mean & std of your chosen metric ~~~
df_ag = (
    df.groupby(["function", "dimension", "method"], as_index=False)
        .agg(
            metric_mean = (metric, "mean"),
            metric_std   = (metric, "std")
        )
)

# ~~~ 2) Compute relative std => 100*(std / mean) ~~~
eps = 1e-15
df_ag["pct_std"] = np.where(
    np.abs(df_ag["metric_mean"]) > eps,
    100.0 * df_ag["metric_std"] / np.abs(df_ag["metric_mean"]),
    np.nan
)

# ~~~ 3) If metric == 'runtime', we force vmin/vmax for each heatmap ~~~
vmin_mean, vmax_mean = None, None
vmin_pct,  vmax_pct   = None, None
if metric.lower() == "runtime":
    vmin_mean, vmax_mean = 0, 1100
    vmin_pct,  vmax_pct   = 0, 290

# ~~~ 4) Pivot and plot per method ~~~
methods = sorted(df_ag["method"].unique())

for m in methods:
    sub = df_ag[df_ag["method"] == m].copy()
    if sub.empty:
        print(f"[INFO] No data for method={m}. Skipping.")
        continue

    # Pivot for mean
    pivot_mean = sub.pivot(index="function", columns="dimension", values="metric_mean")
    # Pivot for %std
    pivot_pct  = sub.pivot(index="function", columns="dimension", values="pct_std")

    # Create figure => side-by-side subplots
    fig, (ax_left, ax_right) = plt.subplots(nrows=1, ncols=2, figsize=(14, 6))

    # ~~~~~ (A) Mean ~~~~~
    sns.heatmap(
        pivot_mean,
        ax=ax_left,
        annot=True,
        fmt=".2f",
        # 2 decimals

```

```

        cmap="RdYlGn_r",          # green=low => red=high
        linecolor="white",
        linewidths=1,
        cbar=True,
        vmin=vmin_mean,          # forced if metric=='runtime'
        vmax=vmax_mean
    )
    ax_left.set_title(f"{metric} (Mean)\nmethod={m}", fontsize=12)
    ax_left.set_xlabel("Dimension")
    ax_left.set_ylabel("Function")

    # ~~~~~ (B) % Std ~~~~~
    sns.heatmap(
        pivot_pct,
        ax=ax_right,
        annot=True,
        fmt=".2f",                # 2 decimals for percentage
        cmap="RdYlGn_r",
        linecolor="white",
        linewidths=1,
        cbar=True,
        vmin=vmin_pct,           # forced if metric=='runtime'
        vmax=vmax_pct
    )
    ax_right.set_title(f"{metric} (% Std)", fontsize=12)
    ax_right.set_xlabel("Dimension")
    # hide row labels on the y-axis
    ax_right.set_ylabel(None)
    ax_right.set_yticks([])
    ax_right.set_yticklabels([])

    plt.suptitle(f"{metric.capitalize()} - Mean & %Std - {m}", fontsize=14, y=1.02)
    plt.tight_layout()
    plt.show()

    print("\nDone: Created side-by-side (Mean, %Std) heatmaps per method with forced ranges for
runtime.")

confusion_matrix_mean_vs_stdpct()

```

Table 3: Mann–Whitney by Dimension ($\leq 8192D$)

```

import os
import numpy as np
import pandas as pd
import scipy.stats as stats
from scipy.stats import mannwhitneyu

#####
# 1) Overall Mann-Whitney (Newton vs Gradient), dimension  $\leq 8192$ 
#####
def mann_whitney_runtime_overall(
    csv_path="local_results/experiment_20250117_051102/results_csv/summary.csv",

```

```

method1="NewtonRaphson",
method2="GradientDescent",
max_dim=8192
):
    """
    Loads CSV, filters dimension ≤ max_dim, aggregates all runs from each method,
    and performs ONE Mann-Whitney test on 'runtime' comparing method1 vs method2.
    """
    if not os.path.exists(csv_path):
        print(f"[ERROR] CSV not found at: {csv_path}")
        return

    df = pd.read_csv(csv_path)

    # dimension ≤ max_dim
    df = df[df["dimension"] ≤ max_dim].copy()

    arr_m1 = df.loc[df["method"]==method1, "runtime"].dropna().values
    arr_m2 = df.loc[df["method"]==method2, "runtime"].dropna().values

    if len(arr_m1) < 2 or len(arr_m2) < 2:
        print(f"\n[WARN] Not enough data for {method1} vs {method2}, dimension≤{max_dim}")
        print(f"    n_{method1}={len(arr_m1)}, n_{method2}={len(arr_m2)}")
        return

    stat, pval = mannwhitneyu(arr_m1, arr_m2, alternative='two-sided')

    print(f"\n--- (1) Overall Mann-Whitney for dimension≤{max_dim} ---")
    print(f"Comparing runtime between {method1} vs {method2}")
    print(f"U={stat:.4f}, p={pval:.6g}")
    if pval < 0.05:
        print("=> Reject H0 => distribution difference is significant.")
    else:
        print("=> Fail to reject H0 => no significant difference.")

#####
# 2) By-Dimension Mann-Whitney (Newton vs Gradient), dimension ≤8192
#####
def mann_whitney_runtime_by_dimension(
    csv_path="local_results/experiment_20250117_051102/results_csv/summary.csv",
    method1="NewtonRaphson",
    method2="GradientDescent",
    max_dim=8192
):
    """
    For each dimension ≤ max_dim:
    - gather 'runtime' array from method1 & method2
    - if each has ≥2 data points, run Mann-Whitney test
    Returns a DataFrame with columns:
    [dimension, n_m1, n_m2, U_stat, p_value, conclusion]
    """
    if not os.path.exists(csv_path):

```

```

    print(f"[ERROR] CSV not found at: {csv_path}")
    return None

df = pd.read_csv(csv_path)
df = df[df["dimension"] <= max_dim].copy()

all_dims = sorted(df["dimension"].unique())

rows = []
for dim in all_dims:
    arr_m1 = df.loc[(df["dimension"]==dim) & (df["method"]==method1),
"runtime"].dropna().values
    arr_m2 = df.loc[(df["dimension"]==dim) & (df["method"]==method2),
"runtime"].dropna().values

    if len(arr_m1) < 2 or len(arr_m2) < 2:
        rows.append({
            "dimension": dim,
            "n_m1": len(arr_m1),
            "n_m2": len(arr_m2),
            "U_stat": np.nan,
            "p_value": np.nan,
            "conclusion": "Insufficient data"
        })
        continue

    stat, pval = mannwhitneyu(arr_m1, arr_m2, alternative='two-sided')
    if pval < 0.05:
        cstr = "Significant (p<0.05)"
    else:
        cstr = "Not Sig (p≥0.05)"

    rows.append({
        "dimension": dim,
        "n_m1": len(arr_m1),
        "n_m2": len(arr_m2),
        "U_stat": stat,
        "p_value": pval,
        "conclusion": cstr
    })

return pd.DataFrame(rows)

#####
# 3) By-Function Mann-Whitney (Newton vs Gradient), dim ≤8192
#####
def mann_whitney_runtime_by_function(
    csv_path="local_results/experiment_20250117_051102/results_csv/summary.csv",
    method1="NewtonRaphson",
    method2="GradientDescent",
    max_dim=8192
):

```

```

"""
For each function, dimension ≤ max_dim, gather 'runtime' for method1 & method2,
do a Mann-Whitney test.
Returns DataFrame with [function, n_m1, n_m2, U_stat, p_value, conclusion].
"""

if not os.path.exists(csv_path):
    print(f"[ERROR] CSV not found at: {csv_path}")
    return None

df = pd.read_csv(csv_path)
df = df[df["dimension"] ≤ max_dim].copy()

funcs = sorted(df["function"].unique())
rows = []
for func in funcs:
    arr_m1 = df.loc[(df["method"]==method1)&(df["function"]==func),
"runtime"].dropna().values
    arr_m2 = df.loc[(df["method"]==method2)&(df["function"]==func),
"runtime"].dropna().values

    if len(arr_m1) < 2 or len(arr_m2) < 2:
        rows.append({
            "function": func,
            "n_m1": len(arr_m1),
            "n_m2": len(arr_m2),
            "U_stat": np.nan,
            "p_value": np.nan,
            "conclusion": "Insufficient data"
        })
        continue

    stat, pval = mannwhitneyu(arr_m1, arr_m2, alternative='two-sided')
    if pval < 0.05:
        concl = "Significant (p<0.05)"
    else:
        concl = "Not Sig (p≥0.05)"

    rows.append({
        "function": func,
        "n_m1": len(arr_m1),
        "n_m2": len(arr_m2),
        "U_stat": stat,
        "p_value": pval,
        "conclusion": concl
    })
return pd.DataFrame(rows)

#####
# 4) Kruskal-Wallis across all functions (no dim filter)
#     "runtime" grouped by each method separately
#####
def kruskal_wallis_runtime_by_functions(

```

```

csv_path="local_results/experiment_20250117_051102/results_csv/summary.csv",
method="NewtonRaphson"
):
    """
    1) Filter to chosen `method`.
    2) For each function, gather all 'runtime' => array.
    3) Kruskal-Wallis => if p<0.05 => at least one function differs significantly.
    """
    if not os.path.exists(csv_path):
        print(f"[ERROR] CSV file not found at: {csv_path}")
        return

    df = pd.read_csv(csv_path)
    sub = df[df["method"] == method].copy()
    if sub.empty:
        print(f"[WARN] No data for method='{method}'.")
        return

    funcs = sorted(sub["function"].unique())
    arrays = []
    for func in funcs:
        arr = sub.loc[sub["function"]==func, "runtime"].dropna().values
        arrays.append(arr)

    print(f"\n--- Kruskal-Wallis for '{method}' runtime across ALL functions ---")
    stat, pval = stats.kruskal(*arrays)
    print(f"Kruskal-Wallis stat={stat:.4f}, p-value={pval:.6g}")
    if pval < 0.05:
        print("=> Reject H0 => at least one function differs significantly.")
    else:
        print("=> Fail to reject H0 => no significant difference among functions.")

#####
#                               EXAMPLE USAGE
#####
if __name__ == "__main__":
    csv_file = "local_results/experiment_20250117_051102/results_csv/summary.csv"

    # (1) Overall Mann-Whitney (Newton vs Gradient), dimension ≤8192
    mann_whitney_runtime_overall(
        csv_path=csv_file,
        method1="NewtonRaphson",
        method2="GradientDescent",
        max_dim=8192
    )

    # (2) By-Dimension Mann-Whitney (Newton vs Gradient), dimension ≤8192
    dim_table = mann_whitney_runtime_by_dimension(
        csv_path=csv_file,
        method1="NewtonRaphson",
        method2="GradientDescent",
        max_dim=8192
    )

```

```

)
if dim_table is not None:
    print("\nMann-Whitney by dimension (<8192D) => Newton vs Gradient:")
    print(dim_table.to_string(index=False))

# (3) By-Function Mann-Whitney (Newton vs Gradient), dimension <8192
func_table = mann_whitney_runtime_by_function(
    csv_path=csv_file,
    method1="NewtonRaphson",
    method2="GradientDescent",
    max_dim=8192
)
if func_table is not None:
    print("\nMann-Whitney by function (<8192D) => Newton vs Gradient:")
    print(func_table.to_string(index=False))

# (4) Kruskal-Wallis across all functions (runtime), for each method (no dimension filter)
kruskal_wallis_runtime_by_functions(csv_path=csv_file, method="NewtonRaphson")
kruskal_wallis_runtime_by_functions(csv_path=csv_file, method="GradientDescent")

```

Figure 18: Confusion-matrix of Failure Rates Across Dimensions and Functions (Matplotlib)

```

import os
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec

def plot_failure_rate_side_by_side(
    csv_path="summary.csv",
    fail_col="f_final",
    max_dim=None,
    large_threshold=None
):
    """
    Plots two failure-rate heatmaps side by side for:
        - method="GradientDescent"
        - method="NewtonRaphson"
    using a single colorbar on the right.

    * "Failure" is defined as:
        - fail_col is NaN, OR
        - fail_col >= large_threshold (if large_threshold is not None).

    Rows => function, Cols => dimension,
    cell value => fraction of runs that "failed."

    Parameters
    -----
    csv_path : str
        Path to summary CSV file, containing columns: method, dimension, function, etc.
    """

```



```

fail_col : str
    Column name used to detect "failure". By default "f_final".
max_dim : int or None
    If set, filter to dimension ≤ max_dim. Otherwise include all dims.
large_threshold : float or None
    If not None, treat fail_col ≥ this threshold as a failure as well.
"""

if not os.path.exists(csv_path):
    print(f"[ERROR] CSV not found: {csv_path}")
    return

df = pd.read_csv(csv_path)
print(f"[INFO] Loaded CSV from: {csv_path}, shape={df.shape}")

# We'll define a helper that, for a given method name,
# returns a pivoted DataFrame of [function x dimension] => fail_rate
def compute_fail_pivot(df_input, method_name):
    sub = df_input[df_input["method"] == method_name].copy()
    if max_dim is not None:
        sub = sub[sub["dimension"] ≤ max_dim

    if sub.empty:
        return None # no data for that method

    # Mark failure
    is_fail = sub[fail_col].isna()
    if large_threshold is not None:
        is_fail = is_fail | (sub[fail_col] ≥ large_threshold)
    sub["failed"] = is_fail.astype(bool)

    # Group => fail_rate
    gp = (
        sub.groupby(["function", "dimension"], as_index=False) ["failed"]
        .mean()
        .rename(columns={"failed": "fail_rate"})
    )

    # Pivot
    pivot_data = gp.pivot(index="function", columns="dimension", values="fail_rate")

    return pivot_data

pivot_gd = compute_fail_pivot(df, "GradientDescent")
pivot_nw = compute_fail_pivot(df, "NewtonRaphson")

# If both are None => no data => just return
if pivot_gd is None and pivot_nw is None:
    print("[INFO] No data for either GradientDescent or NewtonRaphson. Exiting.")
    return

# ~~~~~
# Build annotation DataFrames => e.g. "12.3%"

```

```

# ~~~~~
def build_annot_df(pivot_data):
    # convert pivot_data to object so we can store strings
    ann = pivot_data.copy().astype(object)
    for r in ann.index:
        for c in ann.columns:
            val = ann.at[r, c]
            if pd.isnull(val):
                ann.at[r, c] = ""
            else:
                ann.at[r, c] = f"{val*100:.1f}%" # e.g. 0.123 => "12.3%"
    return ann

anno_gd = build_annot_df(pivot_gd) if pivot_gd is not None else None
anno_nw = build_annot_df(pivot_nw) if pivot_nw is not None else None

# ~~~~~
# Create figure with 1 row, 3 columns => left=gd, mid=nw, right=colorbar
# ~~~~~
fig = plt.figure(figsize=(11.5, 5))
gs = gridspec.GridSpec(nrows=1, ncols=3, width_ratios=[1.2,1,0.08]) # narrower colorbar

ax_gd = fig.add_subplot(gs[0,0]) # left
ax_nw = fig.add_subplot(gs[0,1]) # middle
ax_cb = fig.add_subplot(gs[0,2]) # colorbar

vmin, vmax = 0.0, 1.0 # fail_rate is from 0..1

# ~~~~~
# (A) GradientDescent
# ~~~~~
if pivot_gd is not None:
    mesh_gd = sns.heatmap(
        pivot_gd,
        ax=ax_gd,
        annot=anno_gd, fmt="",
        cmap="RdYlGn_r",
        vmin=vmin, vmax=vmax,
        cbar=False,
        linecolor="white", linewidths=1
    )
    ax_gd.set_title("GradientDescent", fontsize=12)
    ax_gd.set_xlabel("Dimension")
    ax_gd.set_ylabel("Function")
else:
    # empty or no data => blank
    ax_gd.set_title("GradientDescent\n(No Data)")
    ax_gd.axis("off")

# ~~~~~
# (B) NewtonRaphson
# ~~~~~
if pivot_nw is not None:

```

```

mesh_nw = sns.heatmap(
    pivot_nw,
    ax=ax_nw,
    annot=anno_nw, fmt="",
    cmap="RdYlGn_r",
    vmin=vmin, vmax=vmax,
    cbar=False,
    linecolor="white", linewidths=1
)
ax_nw.set_title("NewtonRaphson", fontsize=12)
ax_nw.set_xlabel("Dimension")

# Hide y-label & ticks => to avoid redundancy
ax_nw.set_ylabel(None)
ax_nw.set_yticks([])
ax_nw.set_yticklabels([])
else:
    ax_nw.set_title("NewtonRaphson\n(No Data)")
    ax_nw.axis("off")

# ~~~~~
# (C) single colorbar
# ~~~~~
# We can just pick one of the existing heatmap references for the color scale
# e.g. prefer mesh_gd if not None, else mesh_nw.
if pivot_gd is not None:
    ref_mesh = mesh_gd
elif pivot_nw is not None:
    ref_mesh = mesh_nw
else:
    ref_mesh = None

if ref_mesh is not None:
    cbar = fig.colorbar(
        ref_mesh.collections[0],
        cax=ax_cb,
        orientation="vertical"
    )
    cbar.set_label("Failure Rate", rotation=90)

# ~~~~~
# Overall title, layout
# ~~~~~
# e.g. "Failure Rate: f_final, threshold≥1e8" or so
thresh_str = f">{large_threshold}" if large_threshold is not None else "(none)"
suptitle = f"Failure Rate by function/dim    (fail_col='{fail_col}', threshold={thresh_str})"
fig.suptitle(suptitle, fontsize=13, y=1.02)
plt.tight_layout()
plt.show()

# ~~~~~
# Example usage:

```

```

if __name__ == "__main__":
    CSV = "local_results/experiment_20250117_051102/results_csv/summary.csv"
    plot_failure_rate_side_by_side(
        csv_path=CSV,
        fail_col="f_final",
        max_dim=131072,
        large_threshold=1e8
    )

```

Figure 19: Confusion-Matrix of Gradient Descent's Relative Function Value Improvement (Matplotlib)

Figure 20: Confusion-Matrix of Newton–Raphson's Relative Function Value Improvement (Matplotlib)

```

import os
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec

def confusion_for_one_rel_metric(
    csv_path="local_results/experiment_20250117_051102/results_csv/summary.csv",
    method="GradientDescent",
    rel_col="relative_improvement_f"
):
    """
    Creates double heatmap visualization for relative metrics with specific ranges.
    Note: Relative metrics are computed only from successful runs (finite values).
    Failed runs are excluded from means but shown as black cells when all runs fail.
    """
    if not os.path.exists(csv_path):
        print(f"[ERROR] CSV not found: {csv_path}")
        return

    df = pd.read_csv(csv_path)
    print(f"[INFO] Creating double heatmaps for method='{method}' & metric='{rel_col}'")

    if rel_col not in df.columns:
        print(f"[ERROR] Column '{rel_col}' not in CSV. Columns:", df.columns.tolist())
        return

    # Filter by method
    df_m = df[df["method"] == method].copy()
    if df_m.empty:
        print(f"[WARNING] No rows for method='{method}'. Nothing to plot.")
        return

    # Custom aggregation function that handles partial data
    def robust_aggregate(group):
        valid_data = group[~np.isinf(group[rel_col])]
        n_total = len(group)
        n_valid = len(valid_data)

```

```

    if n_valid == 0:
        return pd.Series({
            'mean': np.nan,
            'std': np.nan,
            'std_pct': np.nan,
            'valid_ratio': 0
        })

    mean_val = valid_data[rel_col].mean()
    std_val = valid_data[rel_col].std()

    # Calculate relative std% only if mean is not too close to zero
    eps = 1e-14
    std_pct = (100.0 * std_val / abs(mean_val)) if abs(mean_val) > eps else np.nan

    return pd.Series({
        'mean': mean_val,
        'std': std_val,
        'std_pct': std_pct,
        'valid_ratio': n_valid/n_total
    })

# Group and aggregate
grouped = (df_m.groupby(['function', 'dimension'])
            .apply(robust_aggregate)
            .reset_index())

# Print debug info
print(f"\nDEBUG for {method} - {rel_col}:")
print(f"Data shape after grouping: {grouped.shape}")

# Pivot for plotting
pivot_mean = grouped.pivot(index='function', columns='dimension', values='mean')
pivot_std_pct = grouped.pivot(index='function', columns='dimension', values='std_pct')
pivot_valid = grouped.pivot(index='function', columns='dimension', values='valid_ratio')

# Print valid ratio info
print("\nValid data ratio range:")
print(pivot_valid.min().min(), "to", pivot_valid.max().max())

# Setup plot
fig_width = 12 # Increased to accommodate two colorbars
fig = plt.figure(figsize=(fig_width, 5))

widths = [1, 0.05, 1, 0.05] # Two main plots, each with its own colorbar
gs = gridspec.GridSpec(1, 4, width_ratios=widths)

def format_value(x, is_percent=False):
    if pd.isnull(x):
        return "NaN"
    if is_percent:
        if abs(x) >= 10000:
            return f"{x:.1E}%"

```

```

        return f"{x:.1f}%"
    if abs(x) < 0.01 or abs(x) > 1000:
        return f"{x:.1E}"
    return f"{x:.1f}"

def draw_heatmap(pos, pos_cb, data, title, vmin, vmax, is_percent=False, show_ylabel=True,
cmap="RdYlGn") :
    ax = fig.add_subplot(gs[0, pos])
    ax_cb = fig.add_subplot(gs[0, pos_cb])

    # Prepare annotations
    annot = pd.DataFrame(index=data.index, columns=data.columns)
    for i in data.index:
        for c in data.columns:
            annot.at[i, c] = format_value(data.at[i, c], is_percent)

    # Create mask for NaN values and invalid data
    mask = data.isna() | (pivot_valid == 0)

    # Draw heatmap
    mesh = sns.heatmap(
        data,
        ax=ax,
        annot=annot,
        fmt='',
        cmap=cmap,
        vmin=vmin,
        vmax=vmax,
        cbar=True,
        cbar_ax=ax_cb,
        mask=mask,
        linecolor="white",
        linewidths=1
    )

    # Set black background for masked cells
    ax.set_facecolor('black')

    # Add "NaN" text over ALL black boxes
    for i, idx in enumerate(data.index):
        for j, col in enumerate(data.columns):
            if mask.at[idx, col]: # If cell is masked (black)
                ax.text(j + 0.5, i + 0.5, 'NaN',
                        color='white',
                        ha='center',
                        va='center',
                        fontsize=10)

    ax.set_title(title, fontsize=11, pad=10)
    ax.set_xlabel("Dimension", fontsize=10)

    if show_ylabel:
        ax.set_ylabel("Function", fontsize=10)

```

```

        else:
            ax.set_ylabel("")
            ax.set_yticks([])

        return mesh

# Set ranges based on metric type
if rel_col == "relative_improvement_f":
    mean_vmin, mean_vmax = -1.0, 1.0 # Red to Green
    std_vmax = 450 # Green to Red
else: # relative_improvement_dist
    mean_vmin, mean_vmax = -1.0, 1.0 # Red to Green
    std_vmax = 1000 # Green to Red

# Draw mean heatmap
mesh1 = draw_heatmap(0, 1, pivot_mean, f"Mean({rel_col})",
                    vmin=mean_vmin, vmax=mean_vmax,
                    show_ylabel=True)

# Draw std% heatmap
mesh2 = draw_heatmap(2, 3, pivot_std_pct, f"Std%({rel_col})",
                    vmin=0, vmax=std_vmax, is_percent=True,
                    show_ylabel=False, cmap="RdYlGn_r")

plt.suptitle(f"{method}: {rel_col}", fontsize=13, y=1.02)
plt.subplots_adjust(top=0.85, right=0.98, left=0.07, bottom=0.15, wspace=0.2)
plt.show()

# Example usage:
csv_file = "local_results/experiment_20250117_051102/results_csv/summary.csv"

# 1) Relative improvement in function value - GradientDescent
confusion_for_one_rel_metric(
    csv_path=csv_file,
    method="GradientDescent",
    rel_col="relative_improvement_f"
)

# 2) Relative improvement in function value - NewtonRaphson
confusion_for_one_rel_metric(
    csv_path=csv_file,
    method="NewtonRaphson",
    rel_col="relative_improvement_f"
)

# 3) Distance to minimum - GradientDescent
confusion_for_one_rel_metric(
    csv_path=csv_file,
    method="GradientDescent",
    rel_col="relative_improvement_dist"
)

# 4) Distance to minimum - NewtonRaphson

```

```

confusion_for_one_rel_metric(
    csv_path=csv_file,
    method="NewtonRaphson",
    rel_col="relative_improvement_dist"
)

```

Figure 21: Confusion-Matrix of Gradient Descent's Gradient Cosine Similarity (Matplotlib)

Figure 22: Confusion-Matrix of Newton Raphson's Gradient Cosine Similarity (Matplotlib)

Figure 23: Confusion-Matrix of Gradient Descent Step Sizes (Matplotlib)

Figure 24: Confusion-Matrix of Newton Raphson Step Sizes (Matplotlib)

```

import os
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec

def heatmap_metric_analysis_v3(
    csv_path: str,
    method: str,
    metric_base: str,
    mean_cmap="RdYlGn",      # Default colormap (will be overridden based on metric)
    std_cmap="RdYlGn_r",    # Red for higher variation
    mean_range=None,        # Custom range for mean values (min, max)
    std_range=None,         # Custom range for std values (min, max)
    colorbar_width=0.05,
    wspace=0.15
):
    """
    Creates a visualization of metric analysis with customizable color ranges.

    Args:
        csv_path: Path to the CSV file
        method: Method name
        metric_base: Base name of metric
        mean_cmap: Colormap for mean values
        std_cmap: Colormap for standard deviation
        mean_range: Tuple of (min, max) for mean colorbar range
        std_range: Tuple of (min, max) for std colorbar range
        colorbar_width: Width ratio for colorbar
        wspace: Width space between subplots
    """
    # [Previous data loading and processing code remains exactly the same until the plotting
    part]

    if not os.path.exists(csv_path):
        print(f"[ERROR] CSV file not found: {csv_path}")
        return

    # Load and filter data
    df = pd.read_csv(csv_path)
    df_m = df[df["method"] == method].copy()

```



```

# Construct full column names
mean_col = f"{metric_base}_mean"
std_col = f"{metric_base}_std"

if df_m.empty:
    print(f"[WARNING] No data for method='{method}'. Nothing to plot.")
    return

# Custom aggregation function that handles partial data
def robust_aggregate(group):
    # Filter out infinite values
    valid_data = group[
        (~group[mean_col].isna()) &
        (~np.isinf(group[mean_col])) &
        (~np.isinf(group[std_col]))
    ]
    n_total = len(group)
    n_valid = len(valid_data)

    if n_valid == 0:
        return pd.Series({
            'mean': np.nan,
            'std': np.nan,
            'std_pct': np.nan,
            'valid_ratio': 0
        })

    mean_val = valid_data[mean_col].mean()
    std_val = valid_data[std_col].mean()

    # Calculate relative std% only if mean is not too close to zero
    eps = 1e-14
    std_pct = (100.0 * std_val / abs(mean_val)) if abs(mean_val) > eps else np.nan

    return pd.Series({
        'mean': mean_val,
        'std': std_val,
        'std_pct': std_pct,
        'valid_ratio': n_valid/n_total
    })

# Group and aggregate
grouped = (df_m.groupby(['function', 'dimension'])
            .apply(robust_aggregate)
            .reset_index())

# Pivot for plotting
pivot_mean = grouped.pivot(index='function', columns='dimension', values='mean')
pivot_std_pct = grouped.pivot(index='function', columns='dimension', values='std_pct')
pivot_valid = grouped.pivot(index='function', columns='dimension', values='valid_ratio')

# Setup plot

```

```

fig_width = 15
fig = plt.figure(figsize=(fig_width, 5))

widths = [1, 0.05, 1, 0.05]
gs = gridspec.GridSpec(1, 4, width_ratios=widths)

def format_value(x, is_percent=False):
    if pd.isnull(x):
        return "NaN"
    if is_percent:
        if abs(x) >= 10000:
            return f"{x:.1E}%"
        return f"{x:.1f}%"
    if abs(x) < 0.01 or abs(x) > 1000:
        return f"{x:.1E}"
    return f"{x:.3f}"

def draw_heatmap(pos, pos_cb, data, title, cmap, value_range=None, is_percent=False,
show_ylabel=True):
    ax = fig.add_subplot(gs[0, pos])
    ax_cb = fig.add_subplot(gs[0, pos_cb])

    vmin, vmax = value_range if value_range else (None, None)

    # Prepare annotations
    annot = pd.DataFrame(index=data.index, columns=data.columns)
    for i in data.index:
        for c in data.columns:
            annot.at[i, c] = format_value(data.at[i, c], is_percent)

    # Create mask for NaN values and invalid data
    mask = data.isna() | (pivot_valid == 0)

    # Draw heatmap
    mesh = sns.heatmap(
        data,
        ax=ax,
        annot=annot,
        fmt='',
        cmap=cmap,
        vmin=vmin,
        vmax=vmax,
        cbar=True,
        cbar_ax=ax_cb,
        mask=mask,
        linecolor="white",
        linewidths=1
    )

    # Set black background for masked cells
    ax.set_facecolor('black')

    # Add "NaN" text over masked cells

```

```

    for i, idx in enumerate(data.index):
        for j, col in enumerate(data.columns):
            if mask.at[idx, col]:
                ax.text(j + 0.5, i + 0.5, 'NaN',
                        color='white',
                        ha='center',
                        va='center',
                        fontsize=10)

    ax.set_title(title, fontsize=11, pad=10)
    ax.set_xlabel("Dimension", fontsize=10)

    if show_ylabel:
        ax.set_ylabel("Function", fontsize=10)
    else:
        ax.set_ylabel("")
        ax.set_yticks([])

    return mesh

# Draw mean heatmap with custom colormap and range
mesh1 = draw_heatmap(0, 1, pivot_mean, f"Mean({metric_base})",
                    cmap=mean_cmap,
                    value_range=mean_range,
                    show_ylabel=True)

# Draw std% heatmap with custom colormap and range
mesh2 = draw_heatmap(2, 3, pivot_std_pct, f"Std%({metric_base})",
                    cmap=std_cmap,
                    value_range=std_range,
                    is_percent=True,
                    show_ylabel=False)

plt.suptitle(f"{method}: {metric_base}", fontsize=13, y=1.02)
plt.subplots_adjust(top=0.85, right=0.98, left=0.07, bottom=0.15, wspace=0.2)
plt.show()

# Example usage with all methods and metrics
csv_file = "per_iter_metrics_aggregated.csv"

# Define methods and metrics
methods = ["GradientDescent", "NewtonRaphson", "BFGS", "TrustRegion"]
metrics = ["grad_cosine_sim", "step_size"]

# Define specific settings for each metric
metric_settings = {
    "grad_cosine_sim": {
        "mean_cmap": "RdYlGn",      # Normal (green = 1.00)
        "mean_range": (0.00, 1.00),
        "std_range": (0, 1000)     # 0% to 1000%
    },
    "step_size": {
        "mean_cmap": "RdYlGn_r",    # Reversed (green = 0.0)

```

```

        "mean_range": (0.0, 0.1),
        "std_range": (0, 1500)      # 0% to 1500%
    }
}

# Generate all plots
for metric in metrics:
    for method in methods:
        print(f"\nGenerating plot for {method} - {metric}")
        heatmap_metric_analysis_v3(
            csv_path=csv_file,
            method=method,
            metric_base=metric,
            mean_cmap=metric_settings[metric]["mean_cmap"],
            mean_range=metric_settings[metric]["mean_range"],
            std_range=metric_settings[metric]["std_range"]
        )

```

Table 4: Mann–Whitney Test Results for Gradient Cosine Similarity

Table 5: Mann–Whitney Test Results for Step Size

```

#####
#           Mann-Whitney Helpers (step_size_mean / grad_cosine_sim_mean)
#####
import numpy as np
import pandas as pd
import scipy.stats as stats
import os

def mannwhitney_itercol_by_dimension(
    df: pd.DataFrame,
    analysis_col: str,
    method1: str,
    method2: str,
    max_dim: int = 8192
) -> pd.DataFrame:
    """
    For each dimension ≤ max_dim, compare 'analysis_col' between method1 & method2
    using a two-sided Mann-Whitney test.
    Returns: DataFrame with [dimension, n_m1, n_m2, U_stat, p_value, conclusion].
    """
    # Filter dimension ≤ max_dim
    df = df[df["dimension"] ≤ max_dim].copy()

    dims = sorted(df["dimension"].unique())
    rows = []
    for d in dims:
        vals_m1 = df[(df["method"]==method1) &
(df["dimension"]==d)][analysis_col].dropna().values
        vals_m2 = df[(df["method"]==method2) &
(df["dimension"]==d)][analysis_col].dropna().values

        n1, n2 = len(vals_m1), len(vals_m2)

```

```

        if n1 < 2 or n2 < 2:
            rows.append({
                "dimension": d,
                "n_m1": n1,
                "n_m2": n2,
                "U_stat": np.nan,
                "p_value": np.nan,
                "conclusion": "Insufficient data"
            })
            continue

        # Two-sided Mann-Whitney
        stat, pval = stats.mannwhitneyu(vals_m1, vals_m2, alternative="two-sided")
        if pval < 0.05:
            concl = "Significant (p<0.05)"
        else:
            concl = "Not Sig (p≥0.05)"

        rows.append({
            "dimension": d,
            "n_m1": n1,
            "n_m2": n2,
            "U_stat": stat,
            "p_value": pval,
            "conclusion": concl
        })

    return pd.DataFrame(rows)

def mannwhitney_itercol_by_function(
    df: pd.DataFrame,
    analysis_col: str,
    method1: str,
    method2: str,
    max_dim: int = 8192
) -> pd.DataFrame:
    """
    For each function (dimension ≤ max_dim), compare 'analysis_col' with a two-sided
    Mann-Whitney test. Returns: DataFrame [function, n_m1, n_m2, U_stat, p_value, conclusion].
    """
    df = df[df["dimension"] ≤ max_dim].copy()

    funcs = sorted(df["function"].unique())
    rows = []
    for f_name in funcs:
        vals_m1 = df[(df["method"]==method1) &
(df["function"]==f_name)][analysis_col].dropna().values
        vals_m2 = df[(df["method"]==method2) &
(df["function"]==f_name)][analysis_col].dropna().values

        n1, n2 = len(vals_m1), len(vals_m2)
        if n1 < 2 or n2 < 2:

```

```

        rows.append({
            "function": f_name,
            "n_m1": n1,
            "n_m2": n2,
            "U_stat": np.nan,
            "p_value": np.nan,
            "conclusion": "Insufficient data"
        })
        continue

    stat, pval = stats.mannwhitneyu(vals_m1, vals_m2, alternative='two-sided')
    conclusion = "Significant (p<0.05)" if pval < 0.05 else "Not Sig (p≥0.05)"
    rows.append({
        "function": f_name,
        "n_m1": n1,
        "n_m2": n2,
        "U_stat": stat,
        "p_value": pval,
        "conclusion": conclusion
    })

return pd.DataFrame(rows)

#####
#    Kruskal-Wallis Helper (step_size_mean / grad_cosine_sim_mean by function)
#####
def kruskal_itercol_across_functions(df: pd.DataFrame, analysis_col: str, method: str):
    """
    Gather 'analysis_col' from each function for the chosen method, then do
    Kruskal-Wallis across these multiple function-based groups.
    """
    sub = df[df["method"] == method].copy()
    funcs = sorted(sub["function"].unique())

    data_arrays = []
    for f_name in funcs:
        arr = sub.loc[sub["function"]==f_name, analysis_col].dropna().values
        data_arrays.append(arr)

    print(f"\n--- Kruskal-Wallis on '{analysis_col}' across ALL functions (method={method})
    ---")
    if len(data_arrays) < 2:
        print("Not enough groups for Kruskal-Wallis.")
        return

    stat, pval = stats.kruskal(*data_arrays)
    print(f"Kruskal-Wallis statistic={stat:.4f}, p-value={pval:.6g}")
    if pval < 0.05:
        print("=> Reject H0 => at least one function differs significantly.")
    else:
        print("=> Not Sig => no difference at 5% level.")
def main_itercol_analysis(csv_path="per_iter_metrics_aggregated.csv"):

```

```

if not os.path.exists(csv_path):
    print(f"[ERROR] File not found: {csv_path}")
    return

df = pd.read_csv(csv_path)
print("Loaded iteration metrics CSV with columns:", df.columns.tolist())

# We'll do the same steps for each of these two columns:
columns_to_check = ["step_size_mean", "grad_cosine_sim_mean"]

for col_name in columns_to_check:
    print(f"\n===== {col_name.upper()} =====")

    # A) Mann-Whitney by dimension (≤8192)
    df_dim = mannwhitney_itercol_by_dimension(
        df=df,
        analysis_col=col_name,
        method1="NewtonRaphson",
        method2="GradientDescent",
        max_dim=8192
    )
    print(f"\n(A) Mann-Whitney for '{col_name}' by dimension ≤8192:")
    print(df_dim.to_string(index=False))

    # B) Mann-Whitney by function (≤8192)
    df_func = mannwhitney_itercol_by_function(
        df=df,
        analysis_col=col_name,
        method1="NewtonRaphson",
        method2="GradientDescent",
        max_dim=8192
    )
    print(f"\n(B) Mann-Whitney for '{col_name}' by function ≤8192:")
    print(df_func.to_string(index=False))

    # C) Kruskal-Wallis across all functions (method=Newton, method=Gradient)
    kruskal_itercol_across_functions(df, col_name, "NewtonRaphson")
    kruskal_itercol_across_functions(df, col_name, "GradientDescent")

if __name__ == "__main__":
    # Just run the analysis for step_size_mean & grad_cosine_sim_mean
    main_itercol_analysis()

```